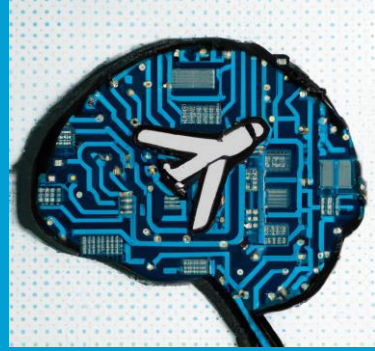
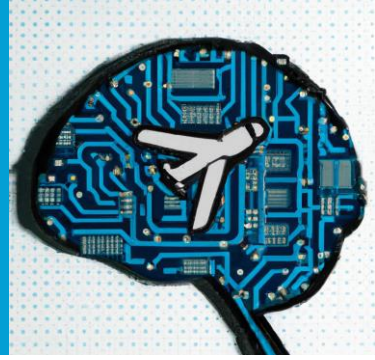




Deep neural networks

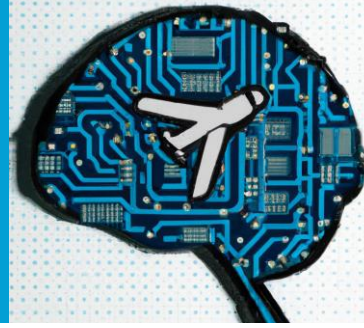
Learning objectives



Students:

- Can list and explain challenges of training deep neural networks.
- Can list and apply solutions to tackle these challenges.
- Can explain how convolution and deconvolution works and why it is important for deep learning.

Recommended reading



“Deep learning: Foundations and concepts”, by Bishop, C. M., & Bishop, H. (2023). Available via TU Delft [here](#).

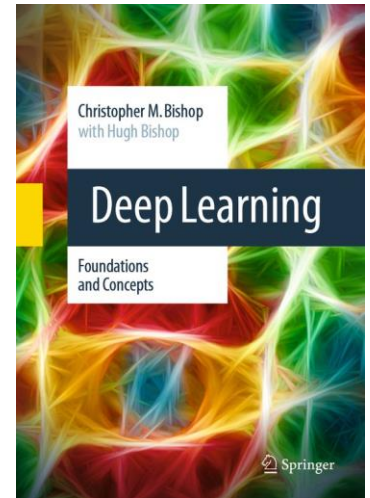
Sections 7.3.3 & 7.4

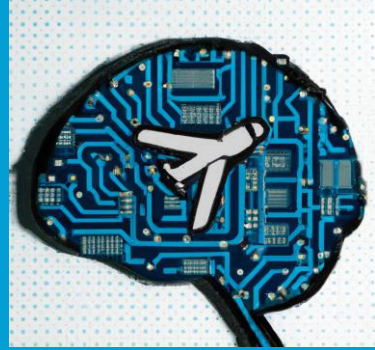
Section 9.6

Interactive reader on github:

<https://github.com/guidoAI/DNNs>

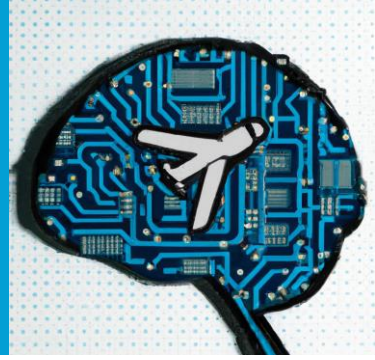
<https://mybinder.org/v2/gh/guidoAI/DNNs/master>





PyTorch

Neural network packages

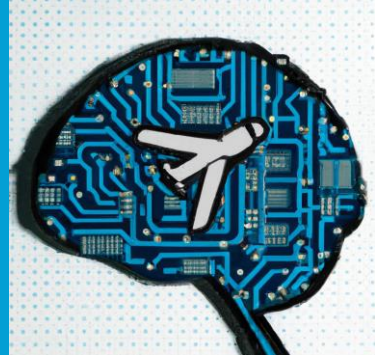


A non-exclusive list of neural network libraries in Python:

- Theano
- TensorFlow
- Keras
- PyTorch

Importantly, these libraries perform automated differentiation to calculate the gradients necessary for backpropagation.

Neural network packages

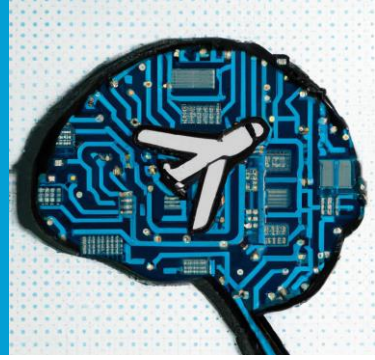


A non-exclusive list of neural network libraries in Python:

- Theano
- TensorFlow
- Keras
- **PyTorch**

Importantly, these libraries perform automated differentiation to calculate the gradients necessary for backpropagation.

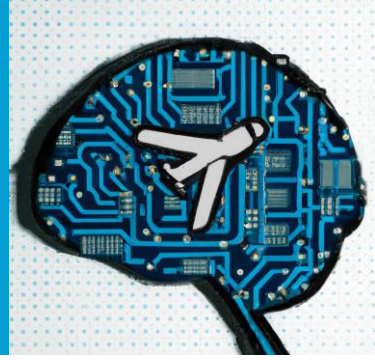
PyTorch



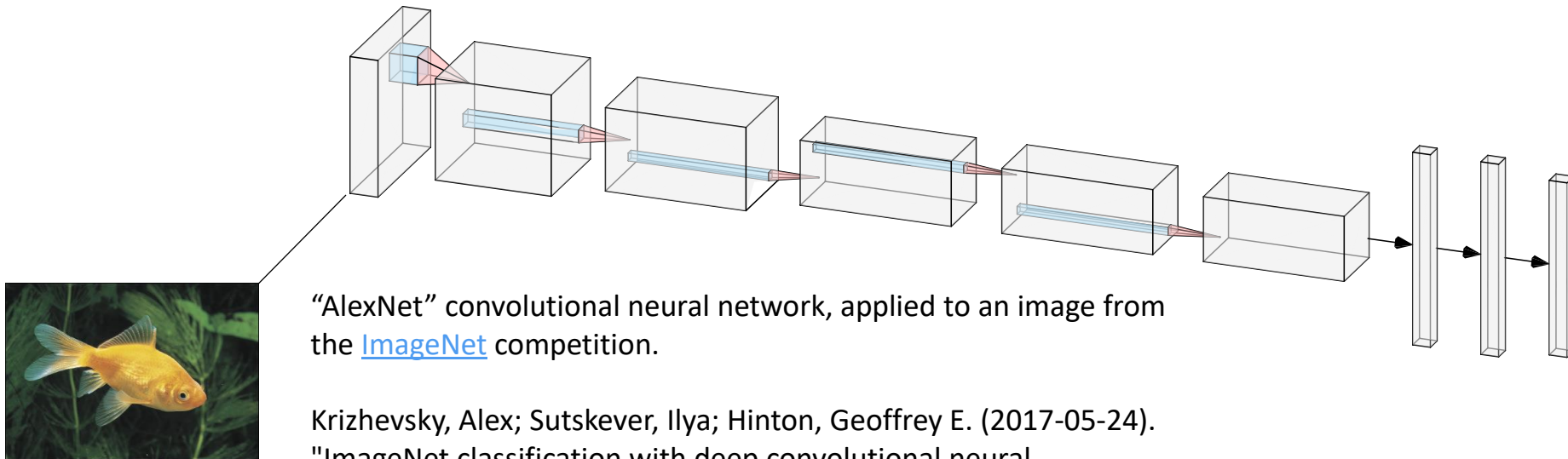
- Python-based neural network library, initially released in 2016 by Meta AI.
- Flexible framework that affords all tools for modern deep learning approaches.
- Ample tutorials are available online:
 - PyTorch [tutorials](#)
 - [60-minute PyTorch Blitz](#)
 - An example tutorial video:



Core concept: Tensor



- For use in PyTorch Tensors can be considered as multi-dimensional arrays.
- For MLPs matrices and vectors suffice, but deep neural networks will process 3- (or higher) dimensional inputs such as images.

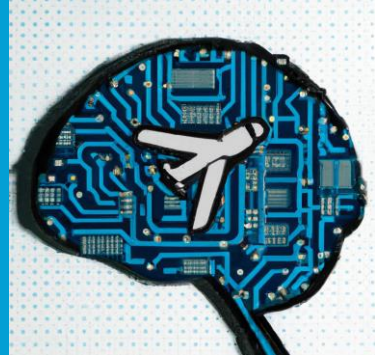


"AlexNet" convolutional neural network, applied to an image from the [ImageNet](#) competition.

Krizhevsky, Alex; Sutskever, Ilya; Hinton, Geoffrey E. (2017-05-24). "ImageNet classification with deep convolutional neural networks". Communications of the ACM. 60 (6): 84–90.



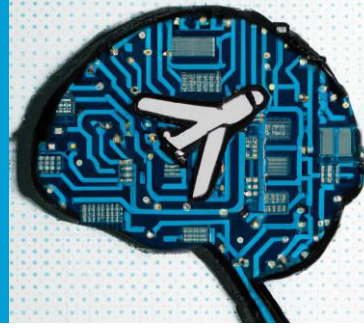
Regression example: training data set



The torch module can be used similarly to numpy:

```
# Create Tensors to hold input and outputs.  
limit = 3  
n_steps = 1000  
x = torch.linspace(-limit, limit, n_steps)  
x = torch.reshape(x, (n_steps, 1))  
y = x**2
```

Regression example: creating the MLP



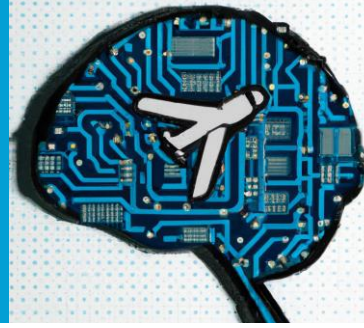
Create a neural network model with a hidden layer that has a sigmoid activation function and a single output with linear activation function:

```
# Make a neural network model for the MLP with sigmoid activation functions in the hidden layer, and linear on the output
```

```
n_hidden_neurons = 10
```

```
model = torch.nn.Sequential(torch.nn.Linear(1, n_hidden_neurons),  
torch.nn.Sigmoid(), torch.nn.Linear(n_hidden_neurons, 1))
```

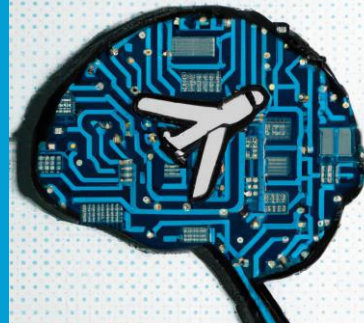
Regression example: Optimization setup



Set up the loss function and the optimizer:

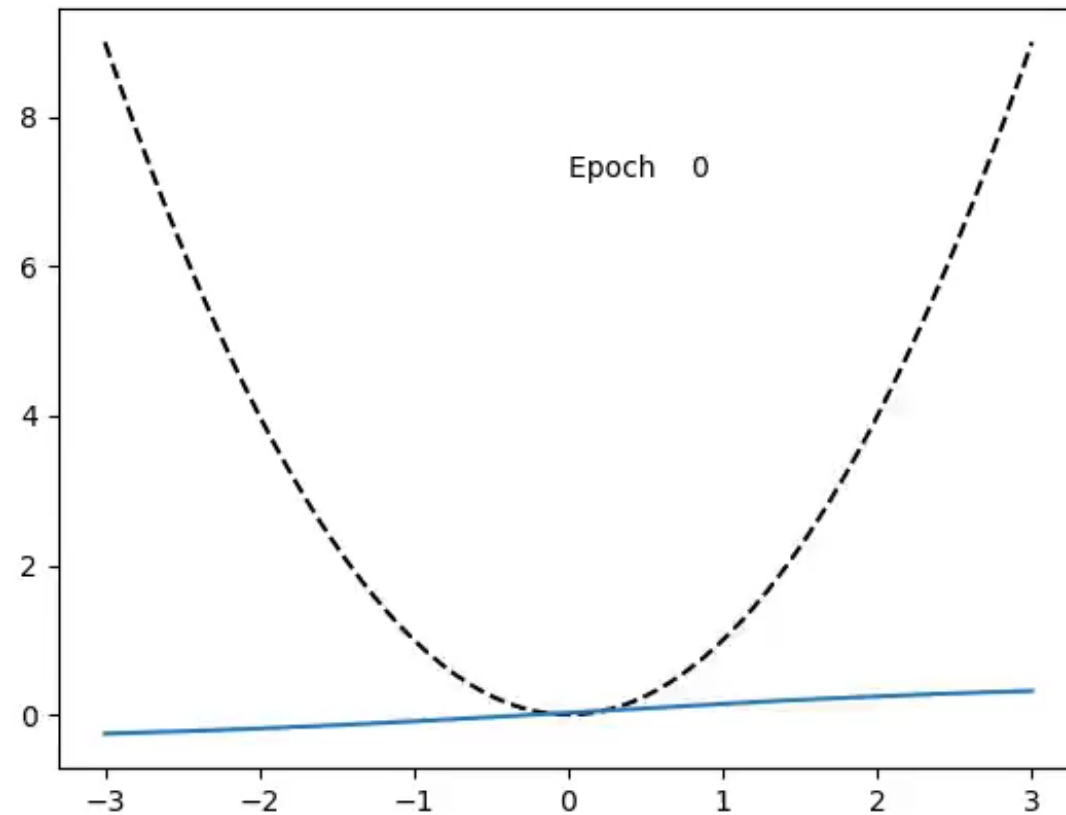
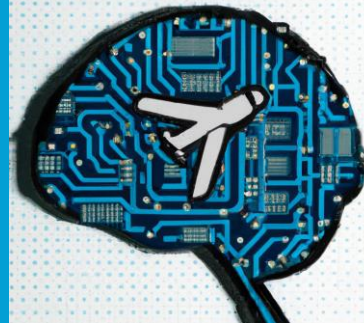
```
# We will use Mean Squared Error (MSE) as our loss function.  
loss_fn = torch.nn.MSELoss(reduction='sum')  
  
# We use stochastic gradient descent for the optimization:  
learning_rate = 1e-5  
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Regression example: perform training

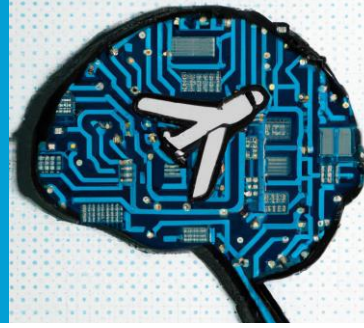


```
for t in range(n_epochs):  
    # Forward pass  
    y_pred = model(x)  
  
    # Compute and print loss. We pass Tensors containing the predicted and  
    # true values of y, and the loss function returns a Tensor containing  
    # the loss.  
    loss = loss_fn(y_pred, y)  
    if t % 100 == 0:  
        print(t, loss.item())  
  
    # Backpropagation  
    loss.backward()  
    optimizer.step()  
    optimizer.zero_grad()
```

Progress over training



Automatic differentiation



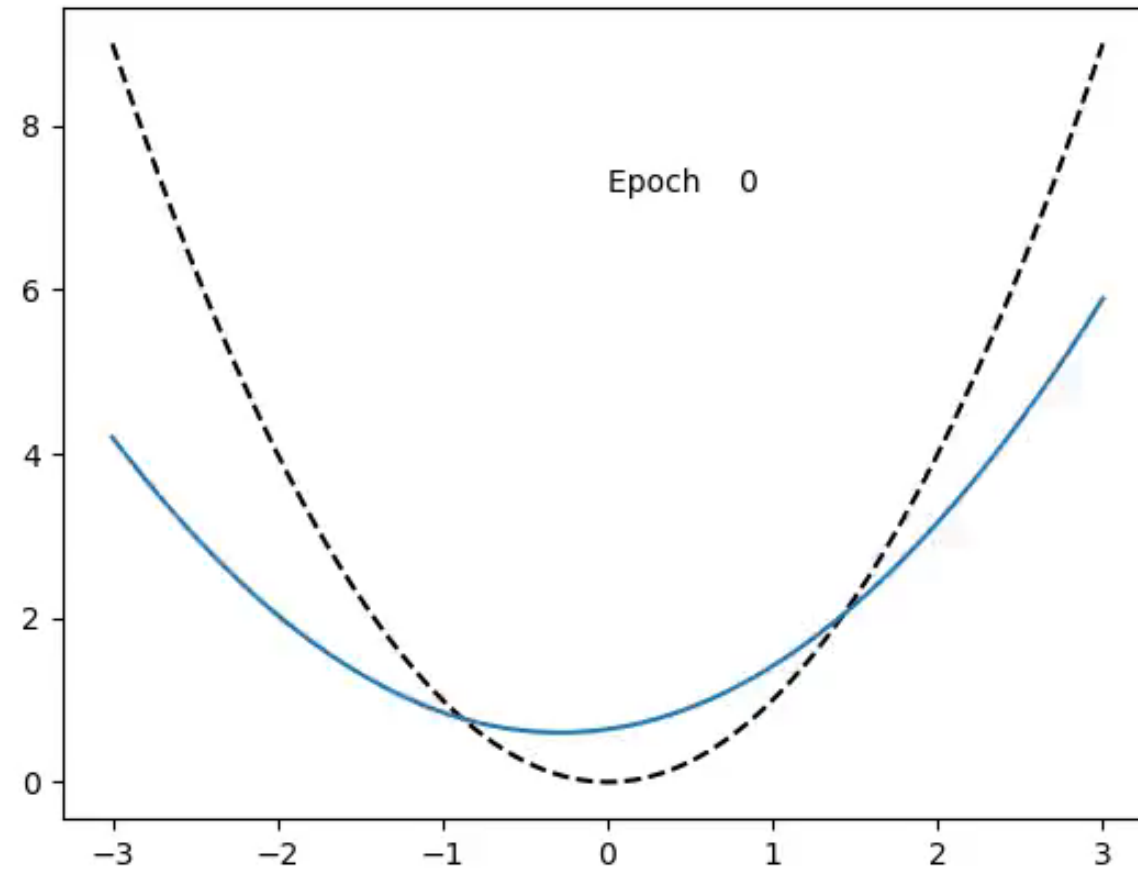
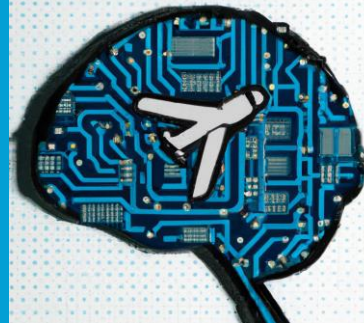
We can use any function for optimization:

```
# Define a module for a quadratic function:
class Quadratic(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.a = torch.nn.Parameter(torch.randn(()))
        self.b = torch.nn.Parameter(torch.randn(()))
        self.c = torch.nn.Parameter(torch.randn(()))

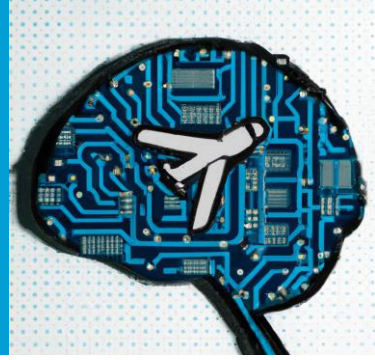
    def forward(self, x):
        return self.a * x ** 2 + self.b * x + self.c

model = Quadratic()
```

Progress over training

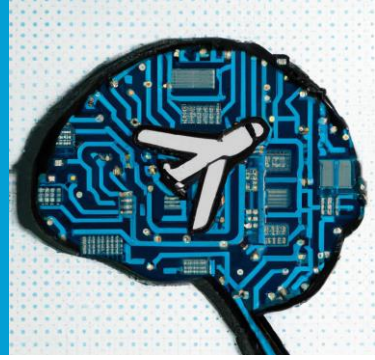


Question:



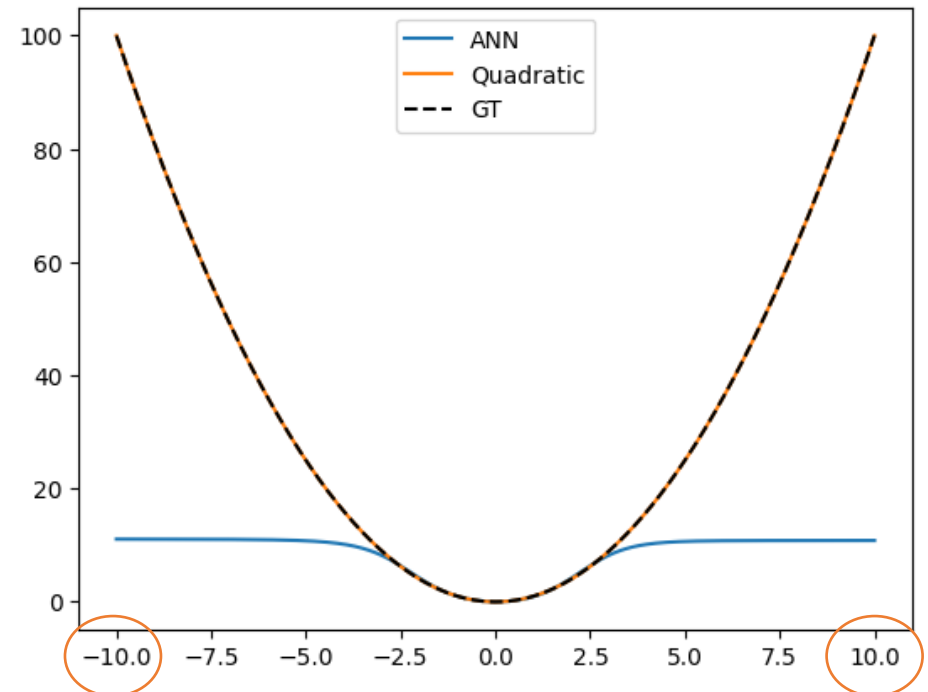
What function will generalize better outside of the training domain,
 $x \in [-3, 3]$?

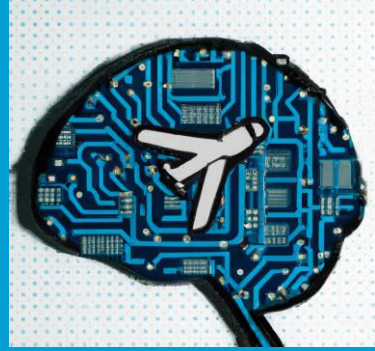
Question:



What function will generalize better outside of the training domain, $x \in [-3, 3]$?

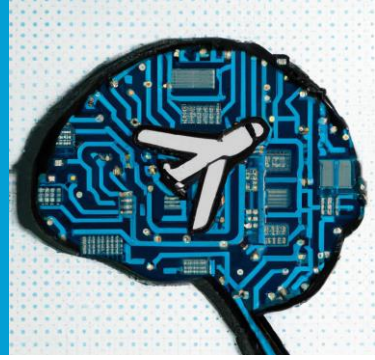
The custom quadratic model has a more suitable **inductive bias** for this specific problem.





Deep learning methods

Developments enabling deep learning

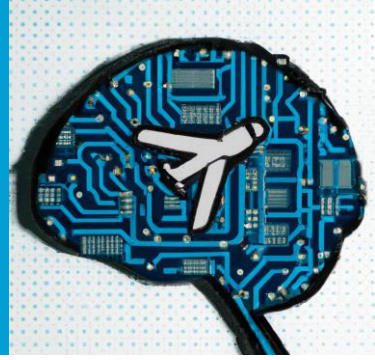


- Hardware: Graphical Processor Units (GPUs)
- More data: Digitalization led to larger data sets
- Better learning methods & model structures



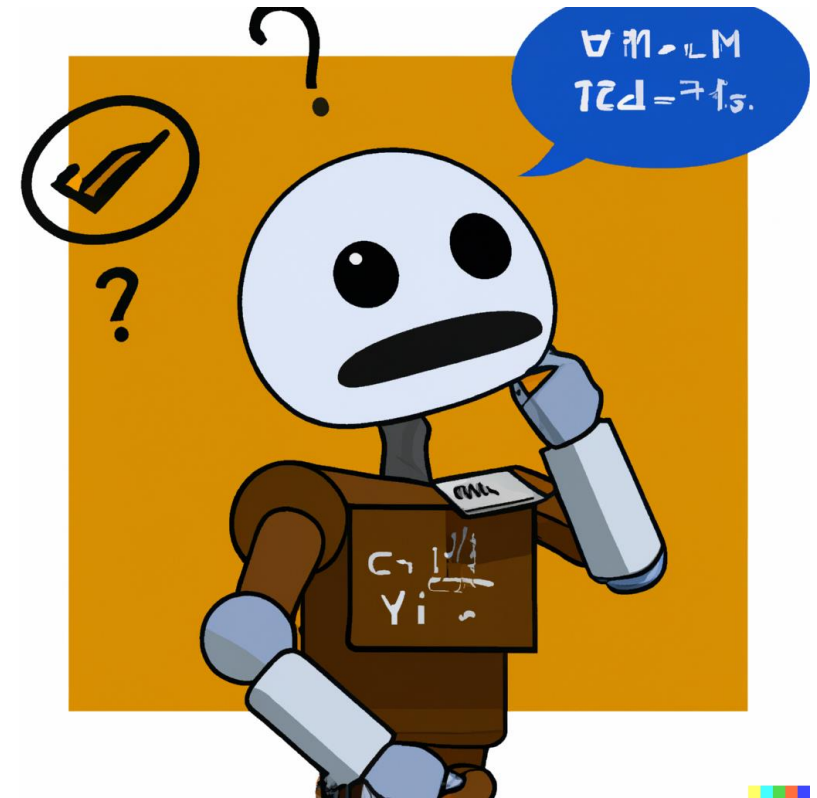
NVidia GPU

Difficulties of learning neural networks

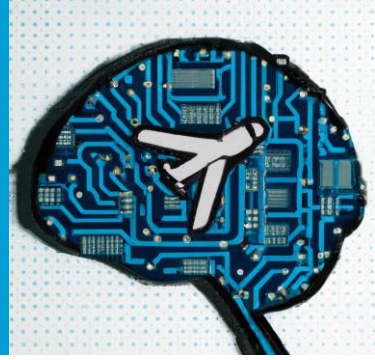


Learning problems:

- The gradient used for learning can vanish or blow up over the layers.
- Hyperparameters like the learning rate can be hard and costly to tune.
- Trained networks may overfit and generalize badly to the test set.
- Network activations can be too large or too small.
- ...

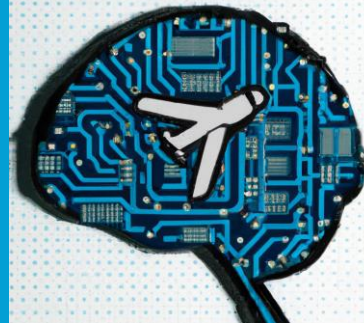


DALL-E drawing of a robot having problems learning



Learning progress

Rectified Linear Unit (ReLU)

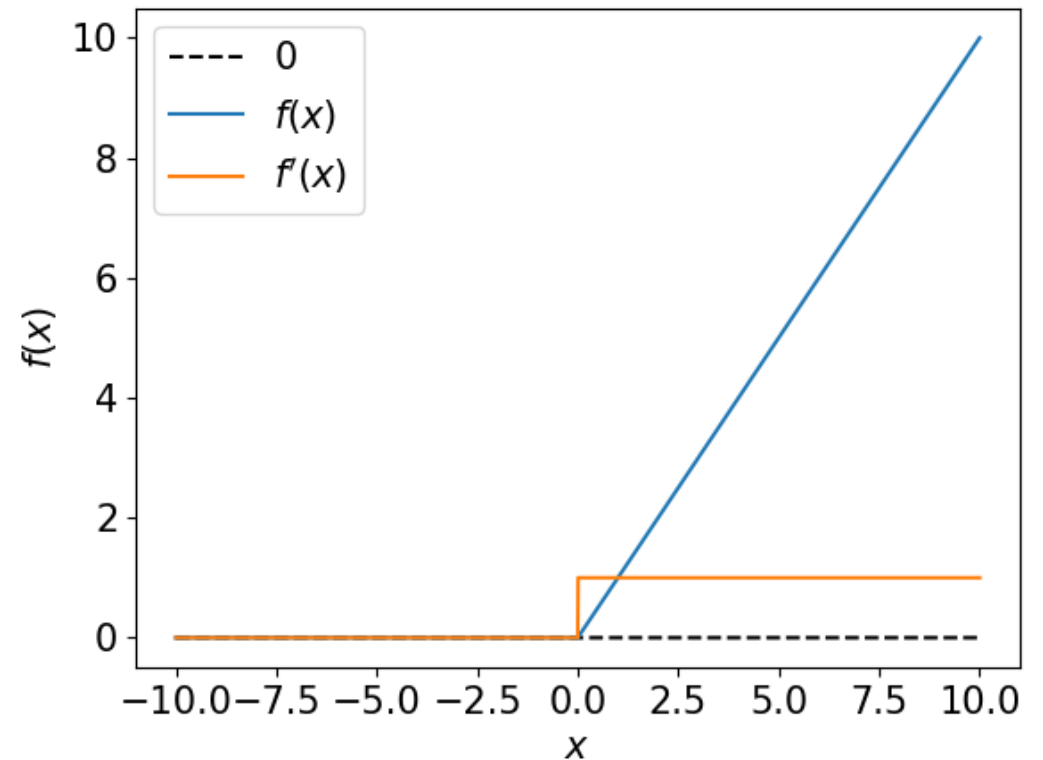


ReLU:

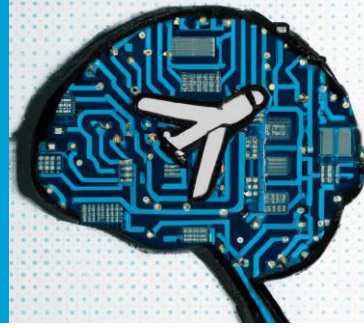
$$f(x) = \max(0, x)$$

Derivative at 0 is undefined,
but assumed to be:

$$f'(x) = (x > 0)$$



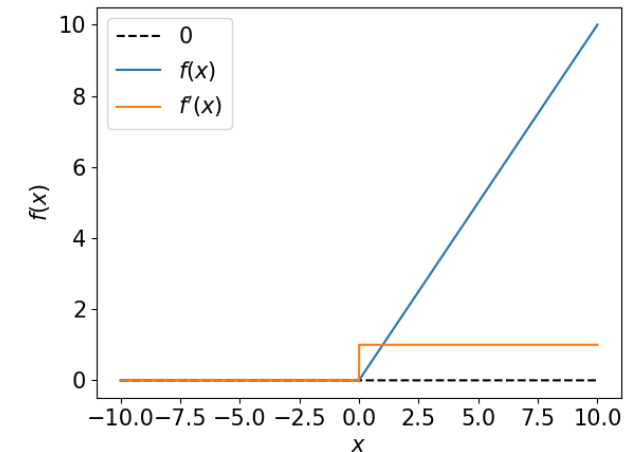
Rectified Linear Unit (ReLU)



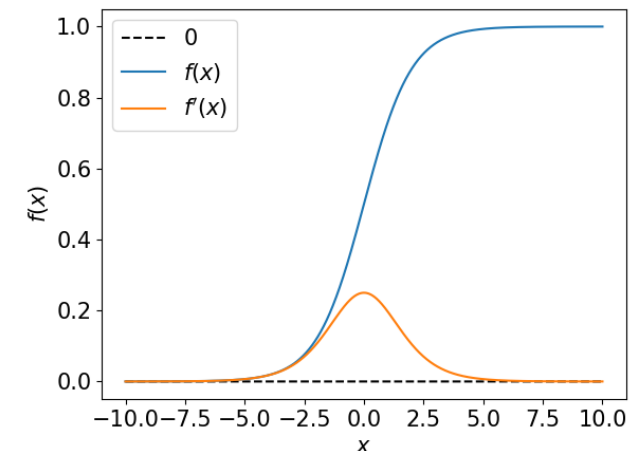
The sigmoid function has a much narrower area in which the derivative $f'(x) \neq 0$.

ReLU is quicker to compute and keeps its gradient also for large values – maintaining learning.

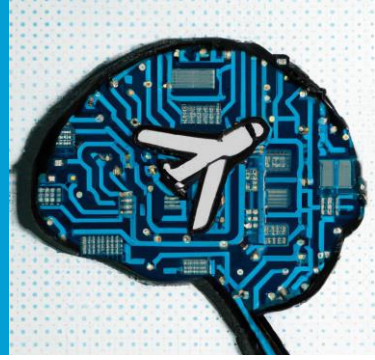
ReLU



Sigmoid

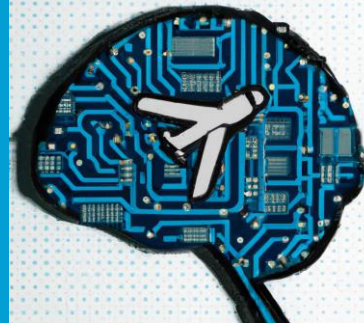


Optimization with gradient descent

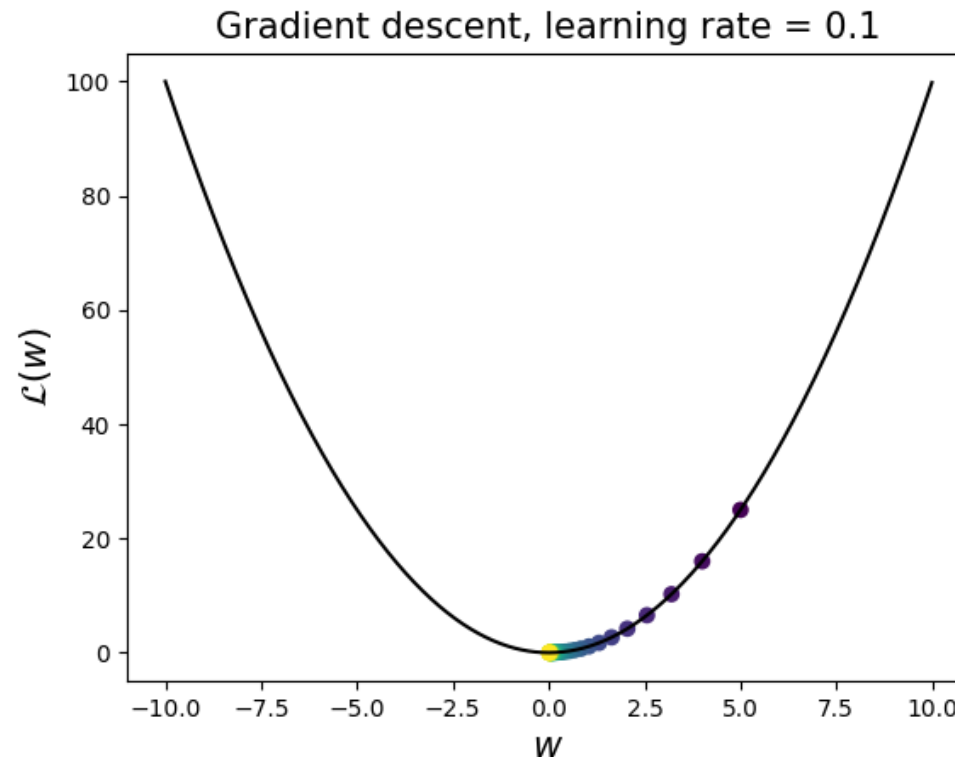


The learning rate is an important hyperparameter that can be difficult to tune.

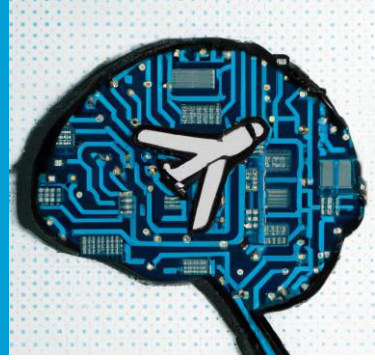
Optimization with gradient descent



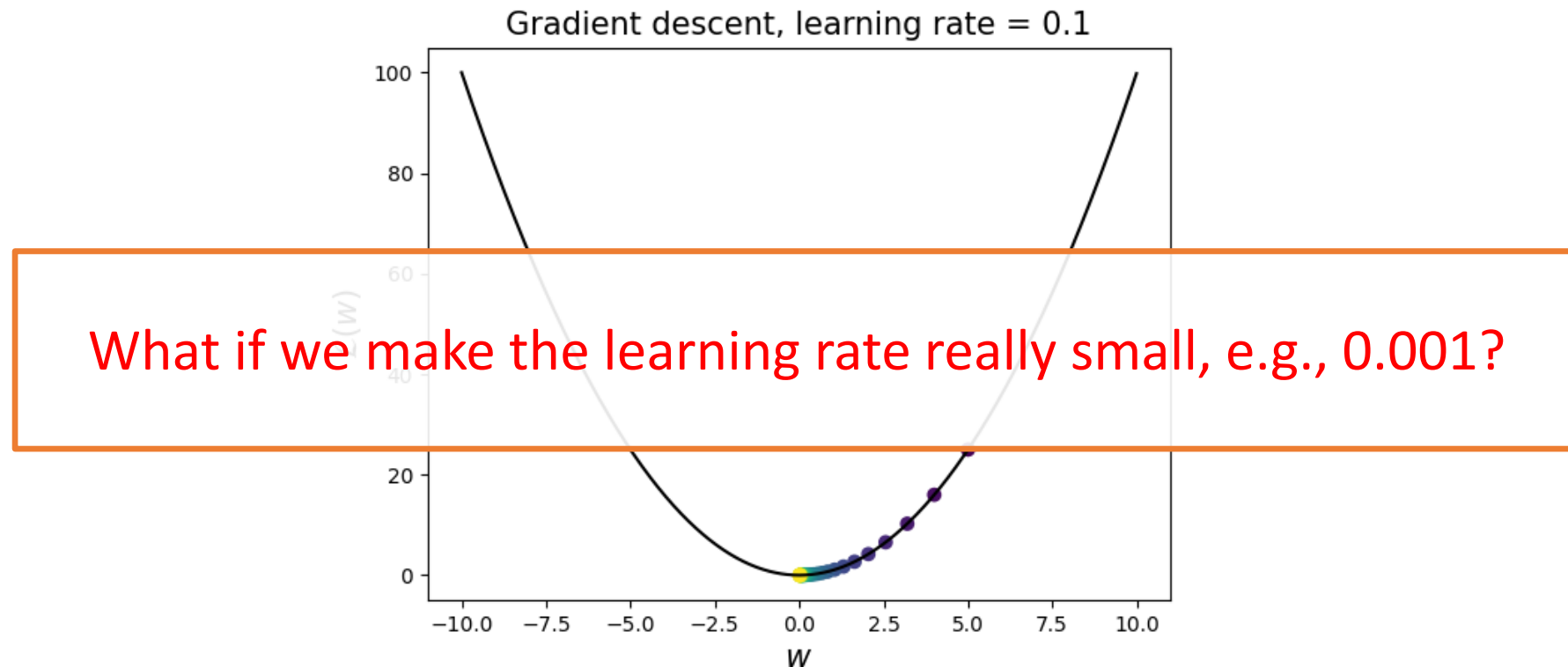
The learning rate is an important hyperparameter that can be difficult to tune.



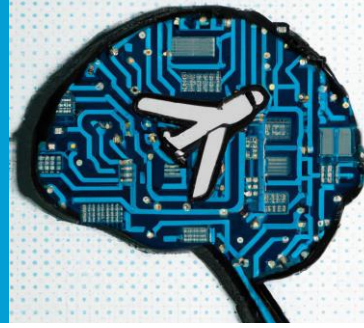
Optimization with gradient descent



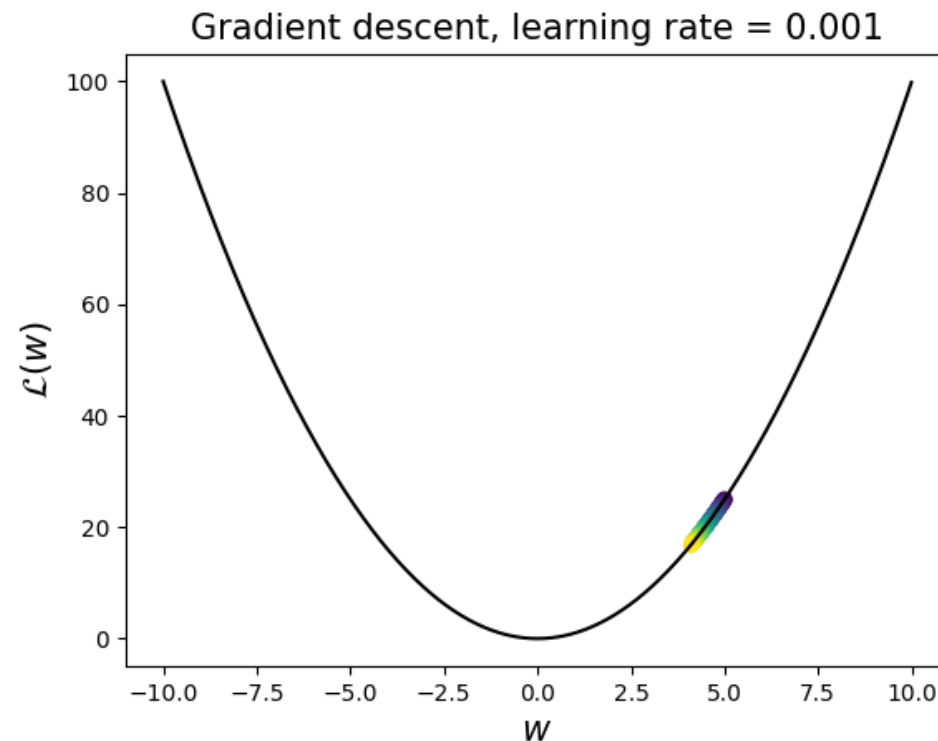
The learning rate is an important hyperparameter that can be difficult to tune.



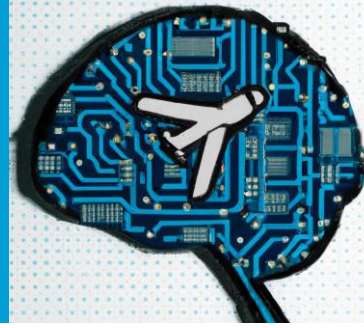
Optimization with gradient descent



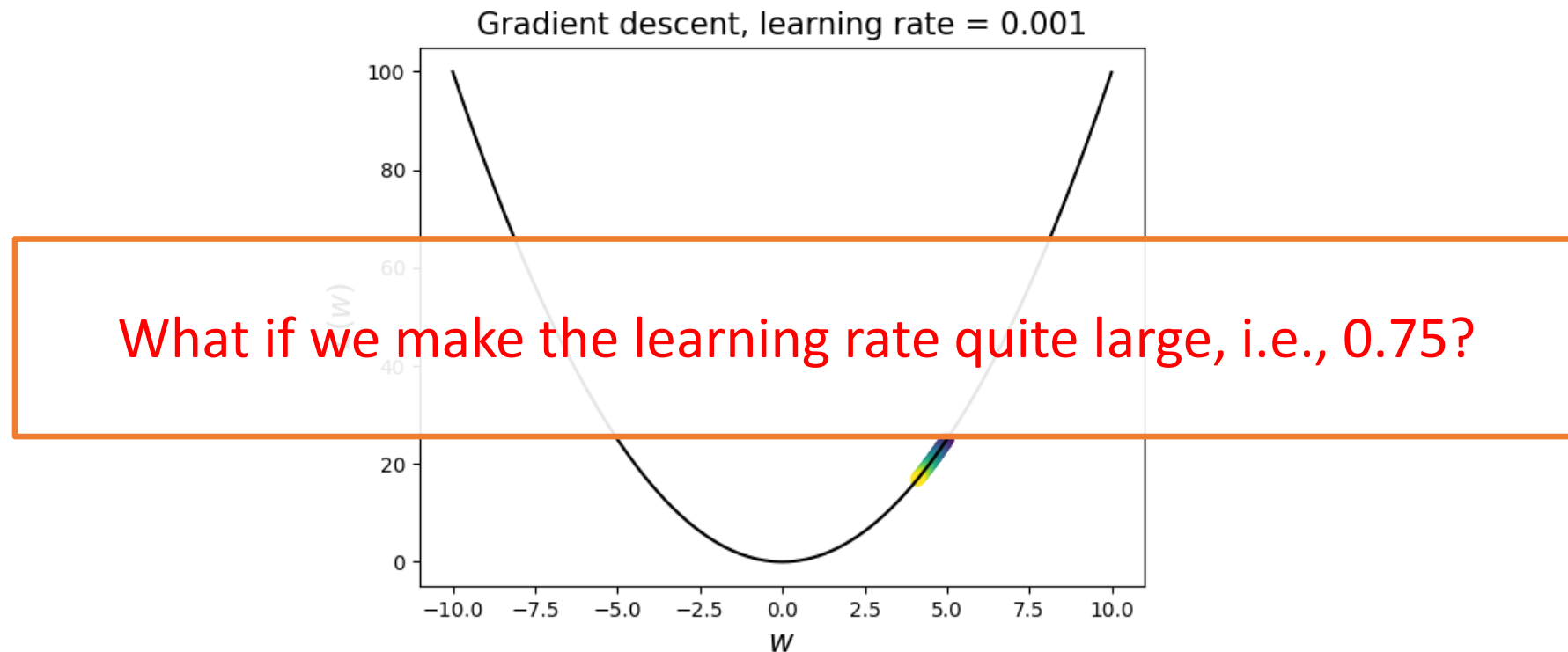
The learning rate is an important hyperparameter that can be difficult to tune.



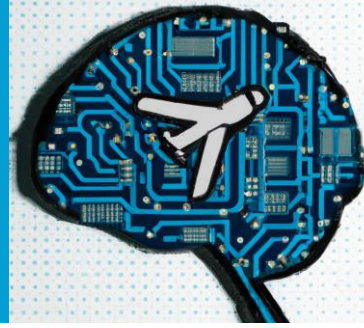
Optimization with gradient descent



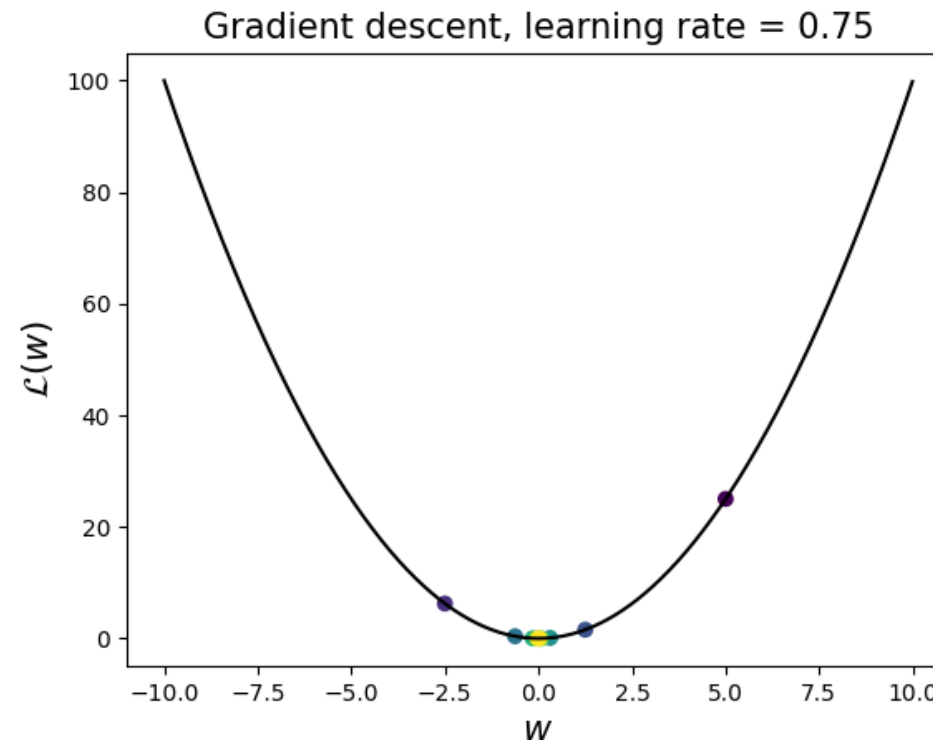
The learning rate is an important hyperparameter that can be difficult to tune.



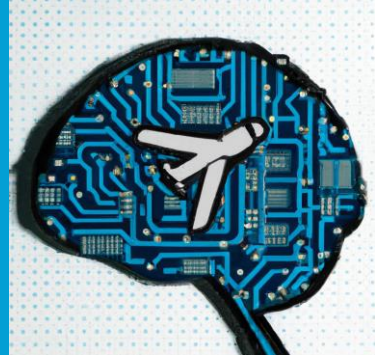
Optimization with gradient descent



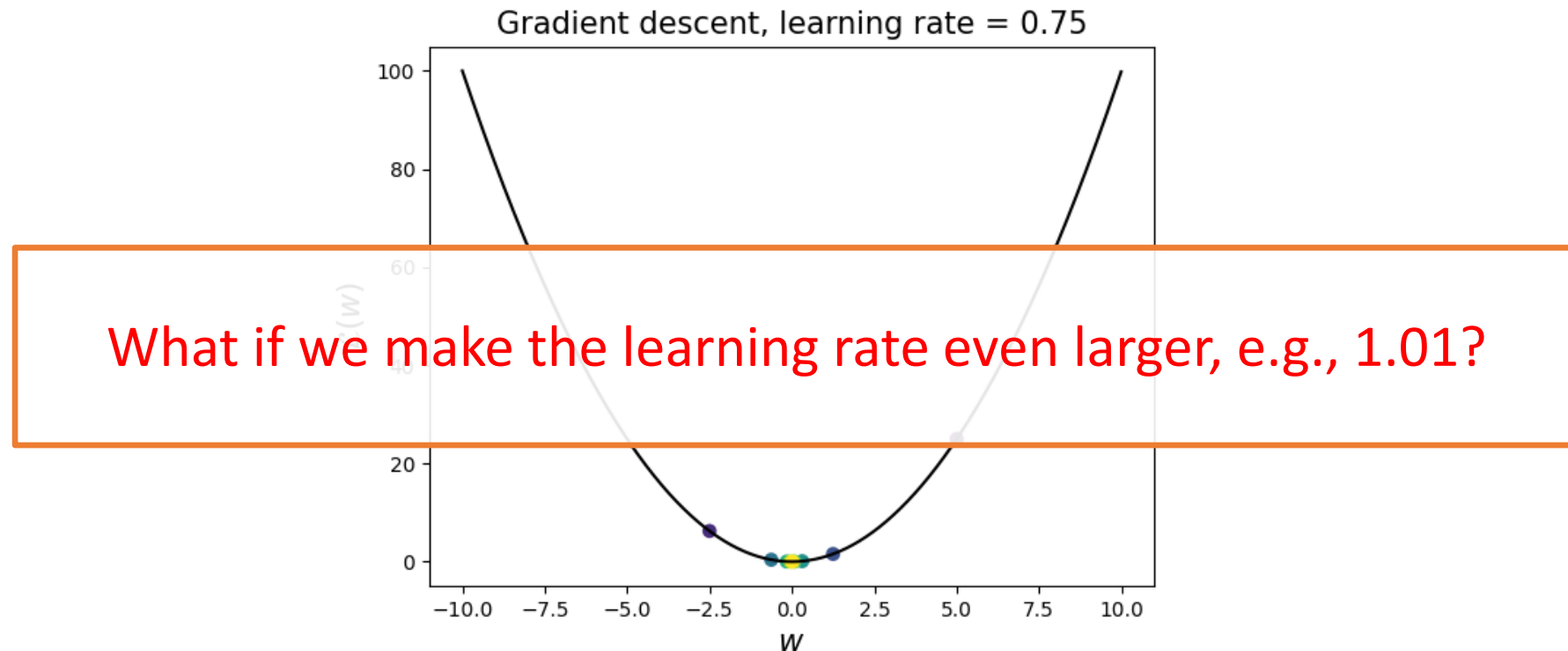
The learning rate is an important hyperparameter that can be difficult to tune.



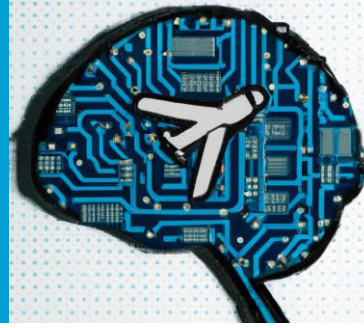
Optimization with gradient descent



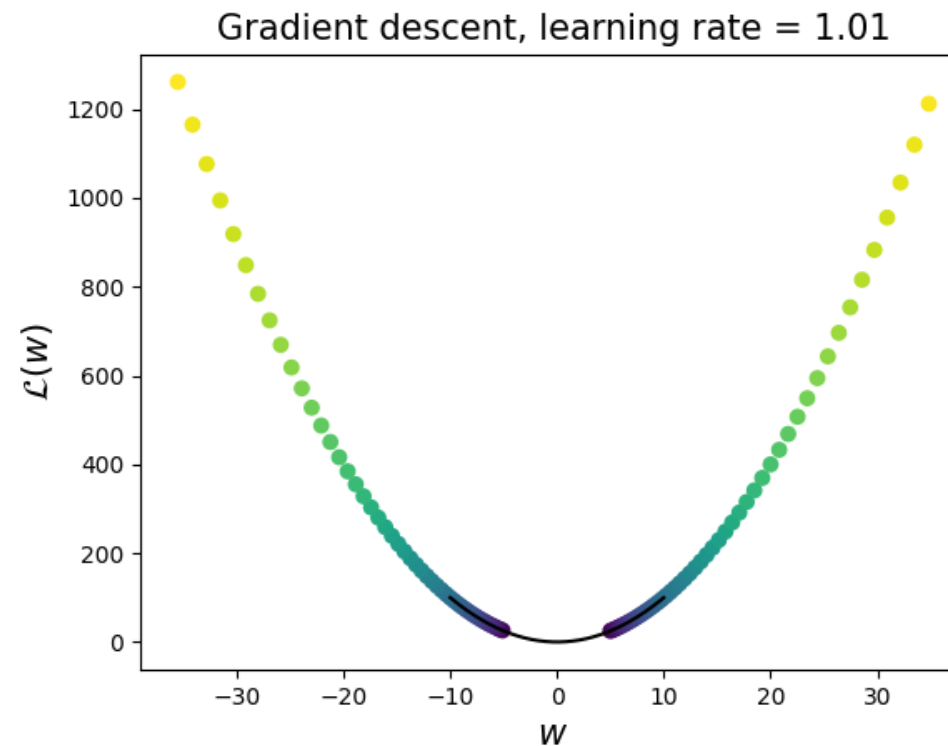
The learning rate is an important hyperparameter that can be difficult to tune.



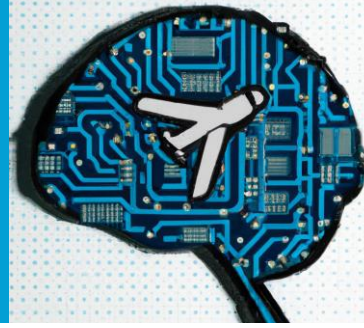
Optimization with gradient descent



The learning rate is an important hyperparameter that can be difficult to tune.



Adaptive optimization algorithms

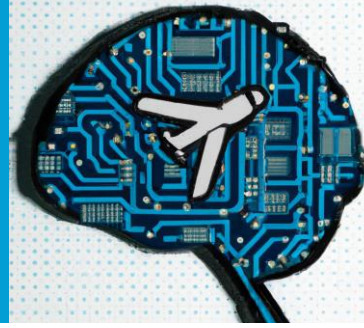


Vanilla backpropagation can take a long time when the gradient is small and can overshoot when the gradient is large.

Typical solutions include:

- Introducing momentum: $\mathbf{g}_t \leftarrow \rho \mathbf{g}_{t-1} + (1 - \rho) \hat{\mathbf{g}}_t$
- Using curvature for quicker search: $\mathbf{H}(f(\mathbf{x})) = \mathbf{J}(\nabla f(\mathbf{x}))$

Adaptive optimization algorithms



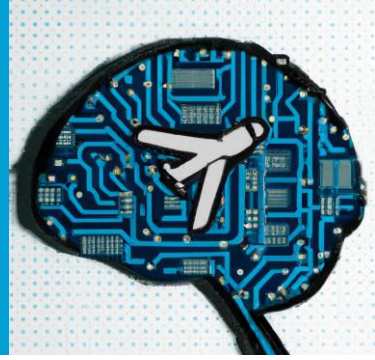
More recent adaptive optimization algorithms for deep neural networks include:

- RMSProp¹
- Adam²

1. Hinton, G., Srivastava, N., & Swersky, K. (2012). Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. Cited on, 14(8), 2.

2. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.

Adaptive optimization algorithms



More recent adaptive optimization algorithms for deep neural networks include:

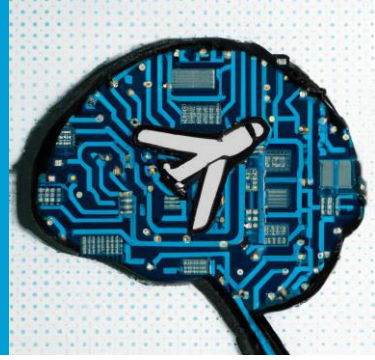
- RMSProp¹
- Adam²

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
optimizer = torch.optim.RMSProp(model.parameters(), lr=learning_rate)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

1. Hinton, G., Srivastava, N., & Swersky, K. (2012). Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. Cited on, 14(8), 2.

2. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.

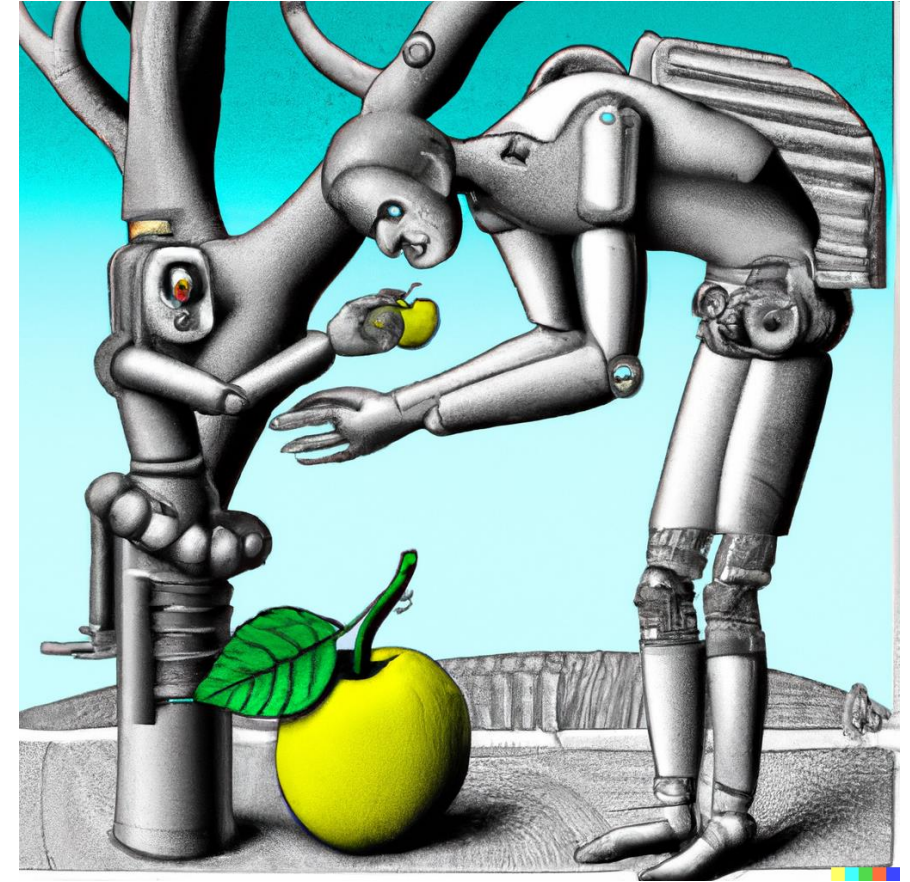
Adaptive moments (Adam)



Estimates a first ($\mathbf{s}$) and second ($\mathbf{r}$) moment based on the gradient $\mathbf{g}$:

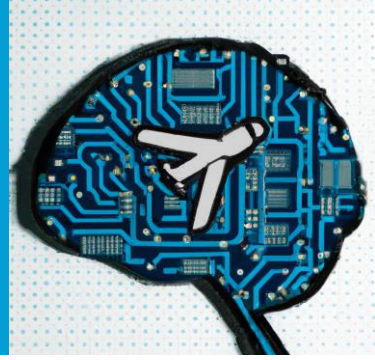
$$\begin{aligned}\mathbf{s} &\leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g} \\ \mathbf{r} &\leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}\end{aligned}$$

, where $\odot$ is the elementwise product, and ρ_1 and ρ_2 determine how much momentum the algorithm has.



DALL-E drawing of a robotic Adam

Adaptive moments (Adam)

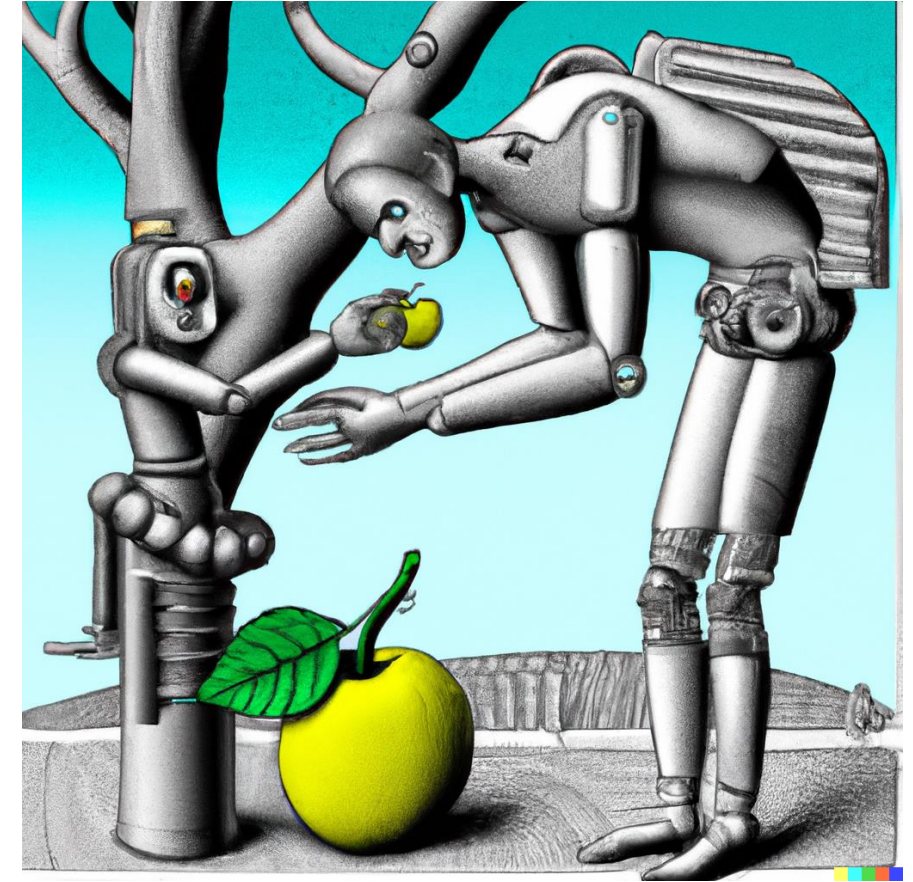


These moments are then corrected for bias:

$$\hat{s} \leftarrow \frac{s}{1 - \rho_1^t}$$

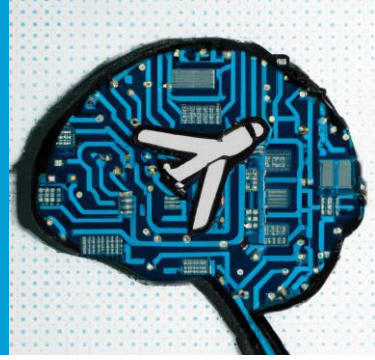
$$\hat{r} \leftarrow \frac{r}{1 - \rho_2^t}$$

, where t corresponds to the number of past updates.



DALL-E drawing of a robotic Adam

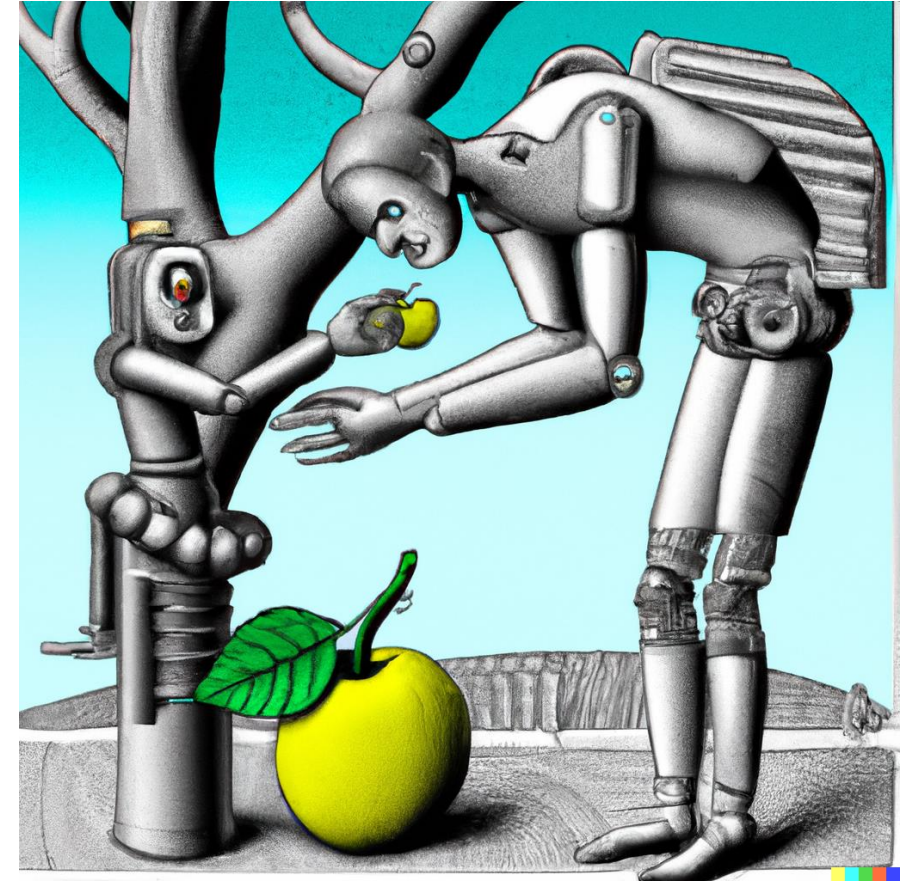
Adaptive moments (Adam)



The update of the parameters θ then becomes:

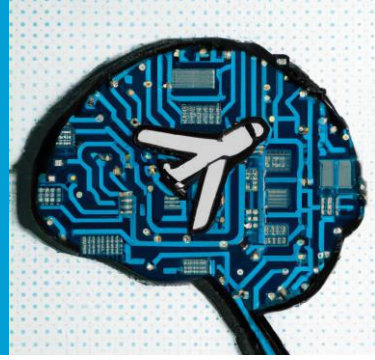
$$\Delta\theta = -\alpha \frac{\hat{s}}{\sqrt{\hat{r}} + \delta}$$

, where α is the learning rate and δ another small number that prevents dividing by zero.



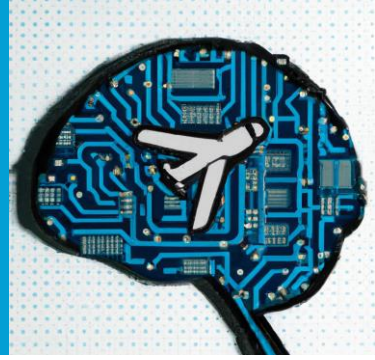
DALL-E drawing of a robotic Adam

Adaptive moments (Adam)

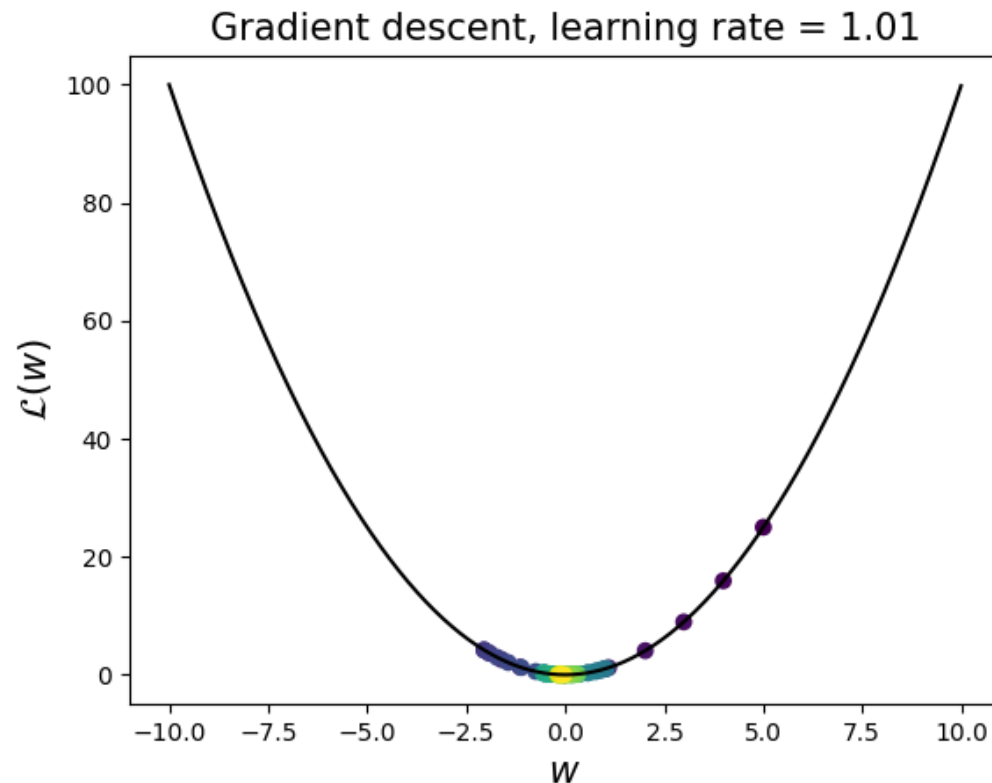


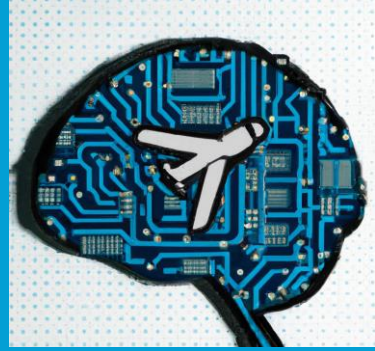
The same problem with learning rate = 1.01:

Adaptive moments (Adam)



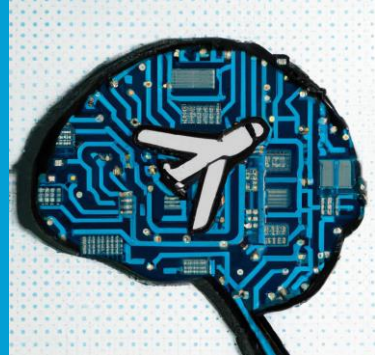
The same problem with learning rate = 1.01:





Regularization

Ensemble learning

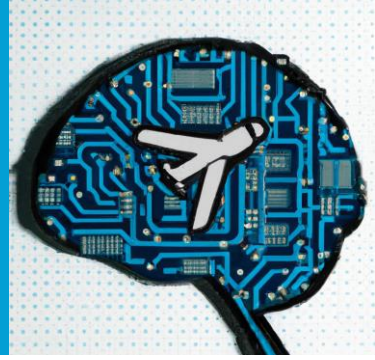


Training multiple neural networks on different parts of the training set, and then vote.

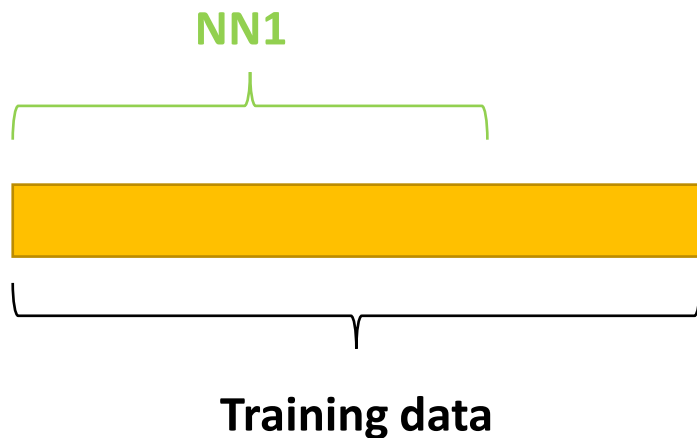


DALL-E impression of robots voting

Ensemble learning

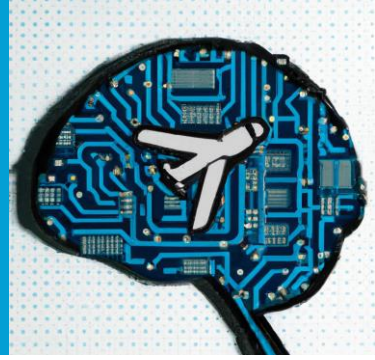


Training multiple neural networks on different parts of the training set, and then vote.

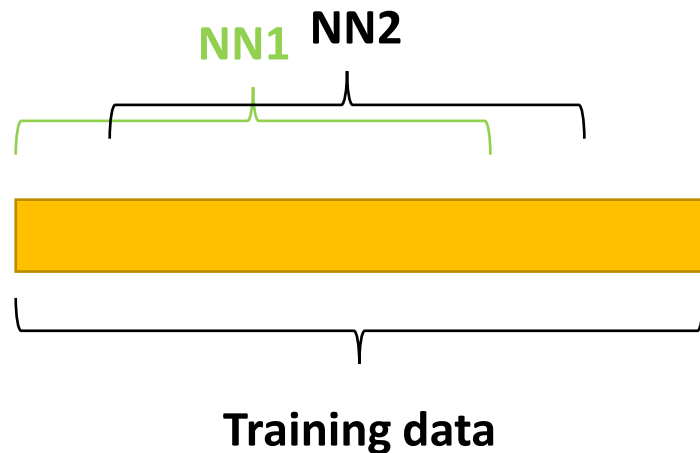


DALL-E impression of robots voting

Ensemble learning

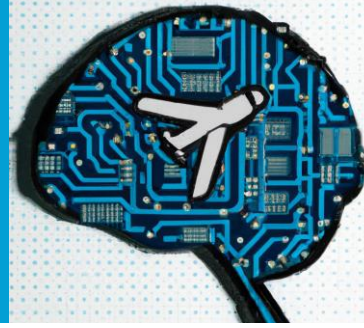


Training multiple neural networks on different parts of the training set, and then vote.

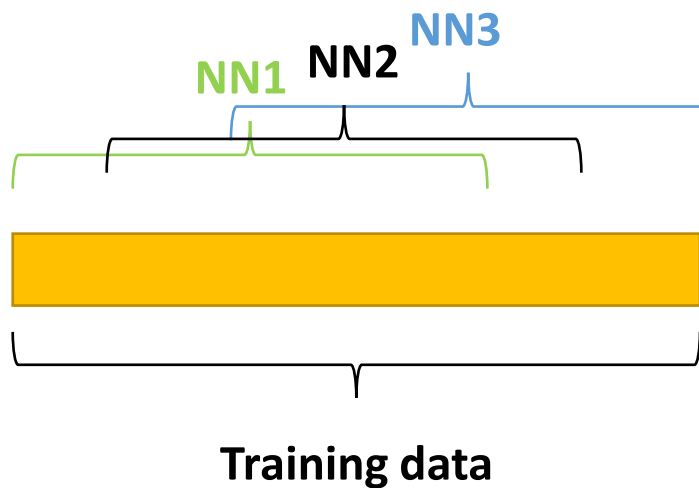


DALL-E impression of robots voting

Ensemble learning

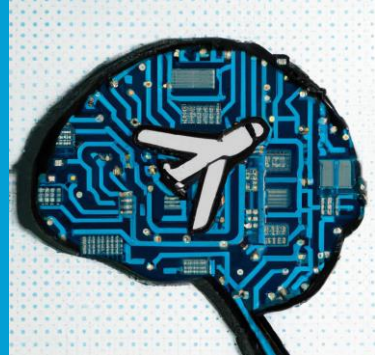


Training multiple neural networks on different parts of the training set, and then vote.

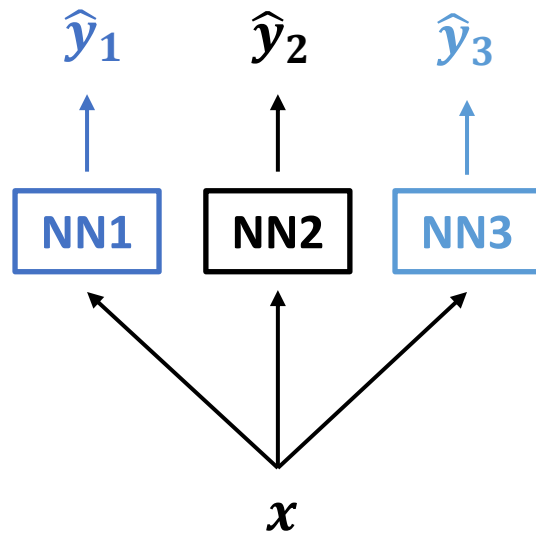


DALL-E impression of robots voting

Ensemble learning

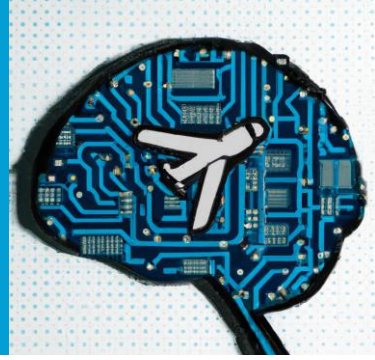


Training multiple neural networks on different parts of the training set, and then vote.



DALL-E impression of robots voting

Ensemble learning

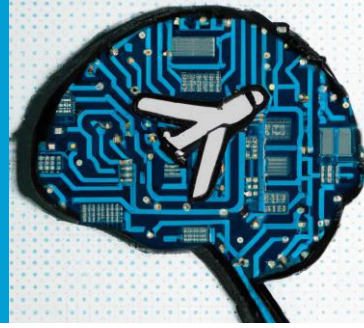


Same overfitting example as before, 1 network:

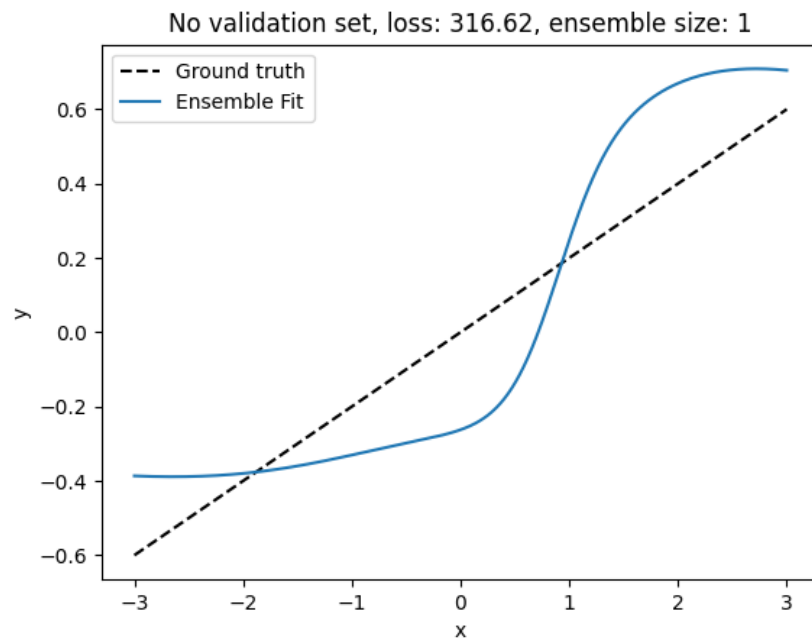


DALL-E impression of robots voting

Ensemble learning

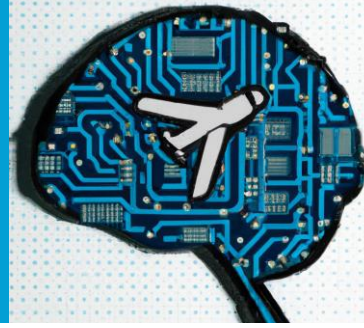


Same overfitting example as before, 1 network:

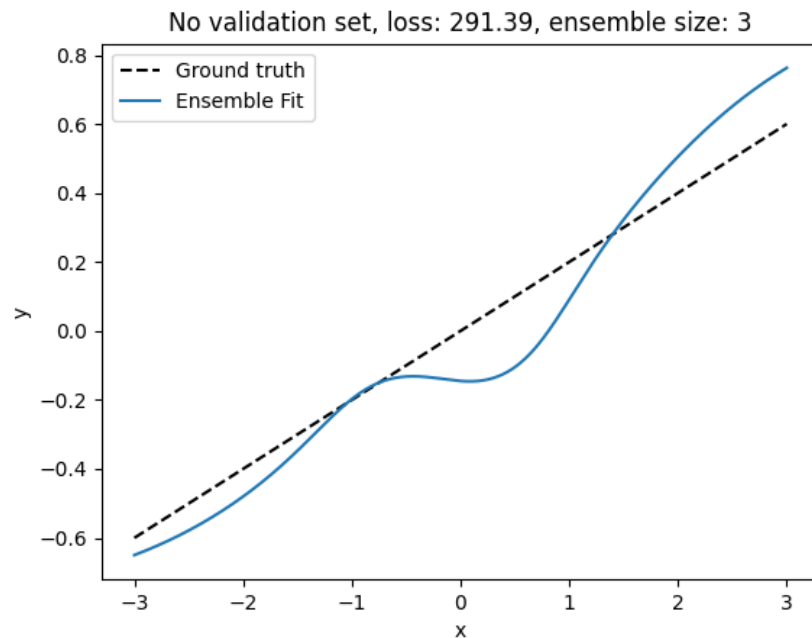


DALL-E impression of robots voting

Ensemble learning

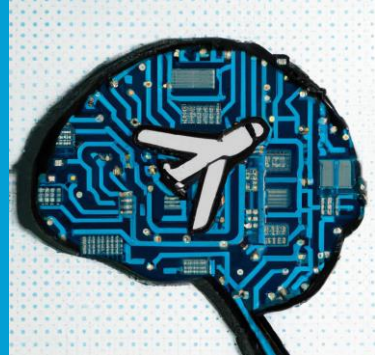


Same overfitting example as before, 1 network:

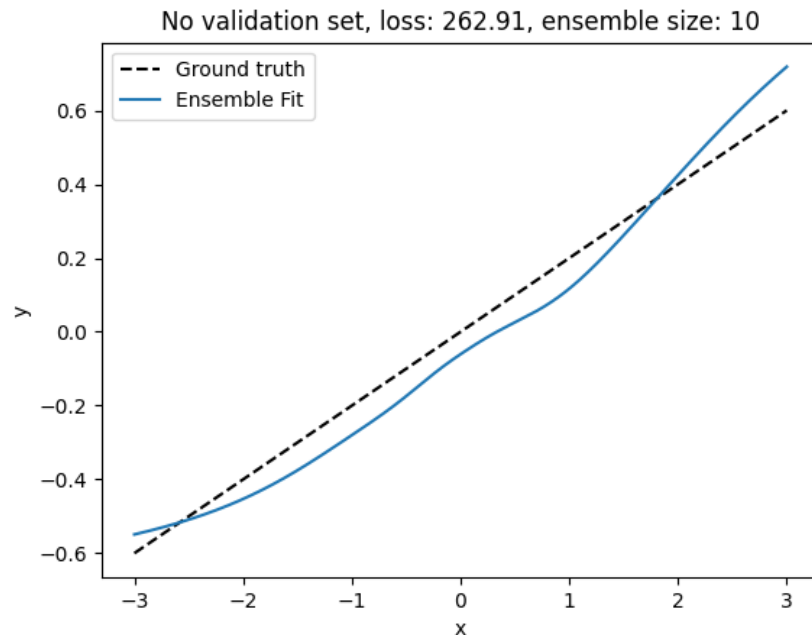


DALL-E impression of robots voting

Ensemble learning

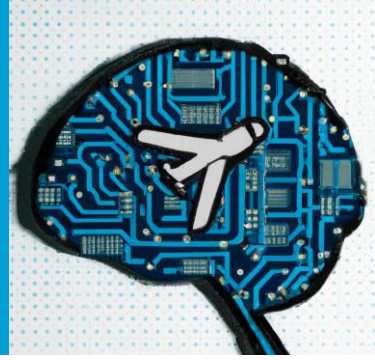


Same overfitting example as before, 1 network:



DALL-E impression of robots voting

Ensemble learning



Advantages:

- ...
- ...

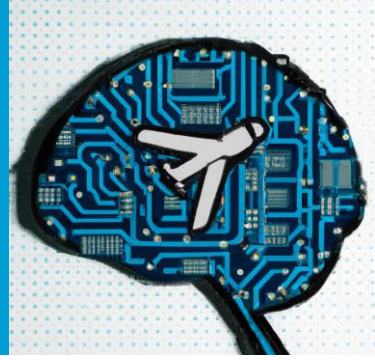
Disadvantage:

- ...



DALL-E impression of robots voting

Ensemble learning



Advantages:

- Helps prevent overfitting, as all networks see different data.
- Allows evaluation of uncertainty (variance of the votes).

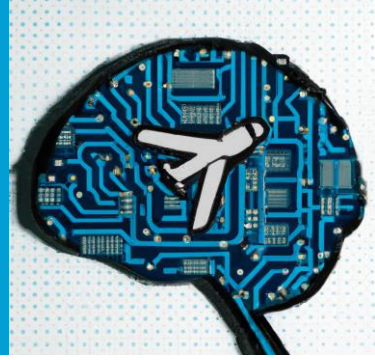
Disadvantage:

- Increases training and runtime.



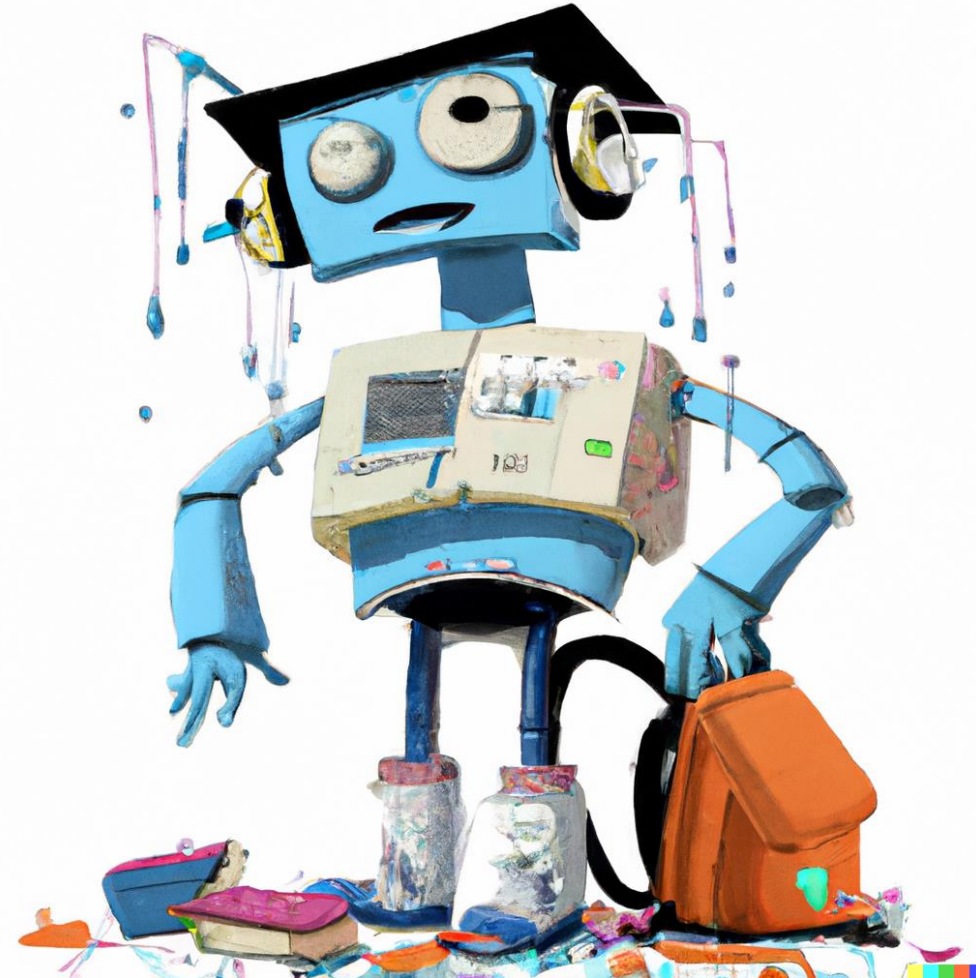
DALL-E impression of robots voting

Dropout



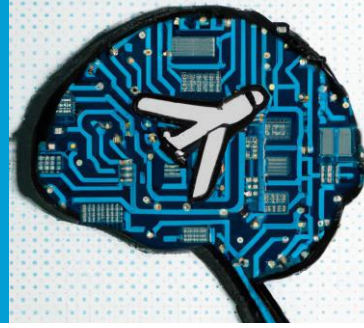
Setting a proportion of hidden neurons to zero.

Normally done during training for learning robust hidden neuron representations.



DALL-E drawing of a robot high school dropout

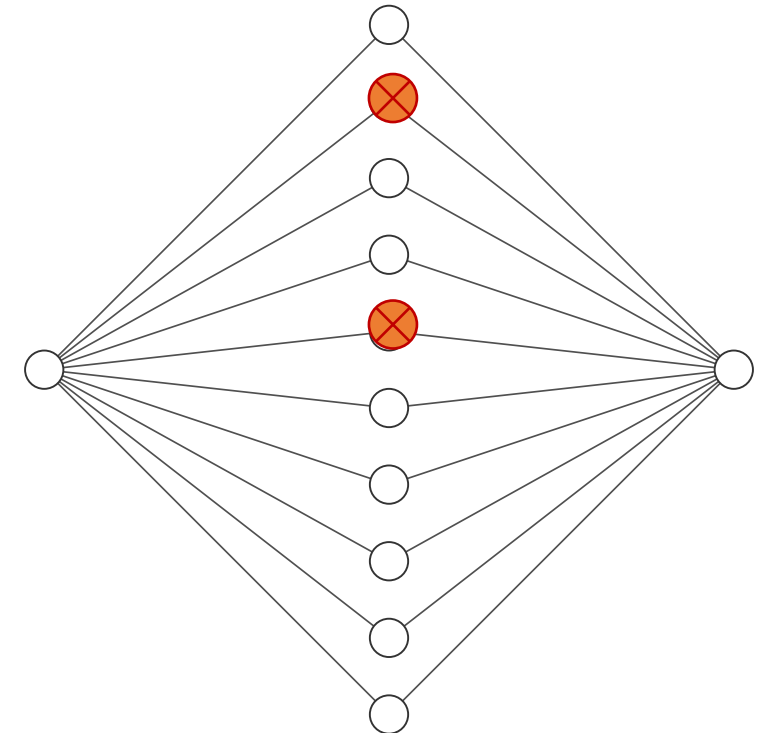
Dropout



Setting a proportion of hidden neurons to zero.

Normally done during training for learning robust hidden neuron representations.

Dropout $p = 0.2$

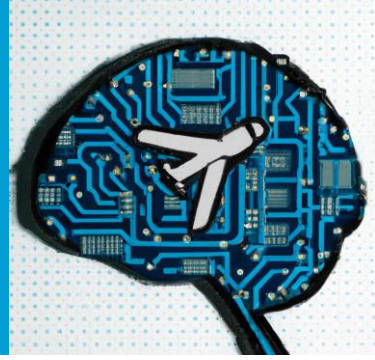


Input Layer $\in \mathbb{R}^1$

Hidden Layer $\in \mathbb{R}^{10}$

Output Layer $\in \mathbb{R}^1$

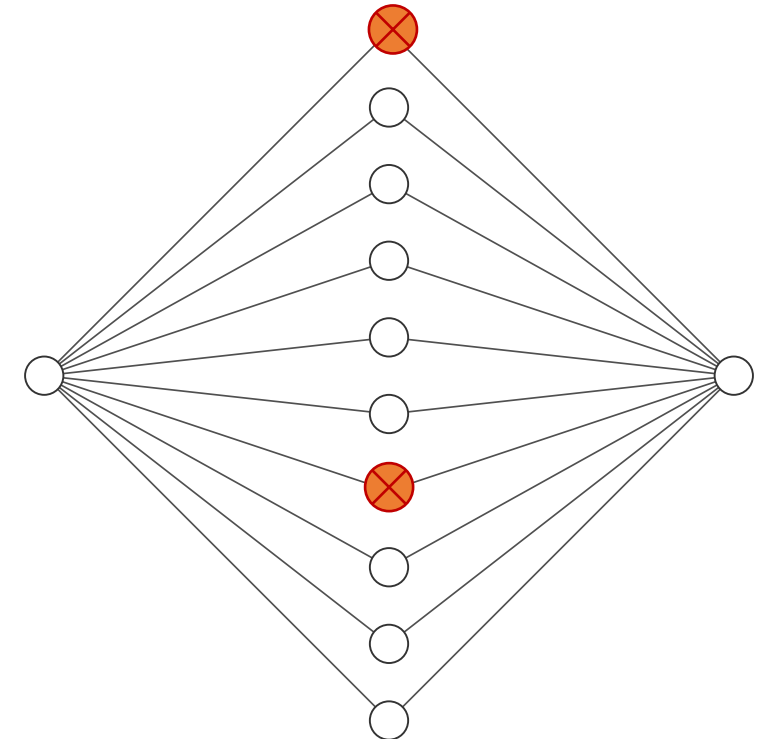
Dropout



Setting a proportion of hidden neurons to zero.

Normally done during training for learning robust hidden neuron representations.

Dropout $p = 0.2$

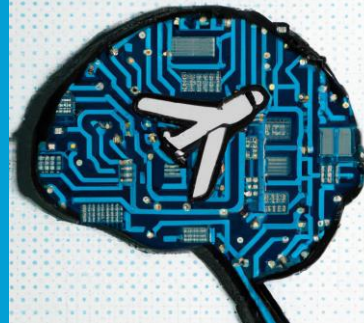


Input Layer $\in \mathbb{R}^1$

Hidden Layer $\in \mathbb{R}^{10}$

Output Layer $\in \mathbb{R}^1$

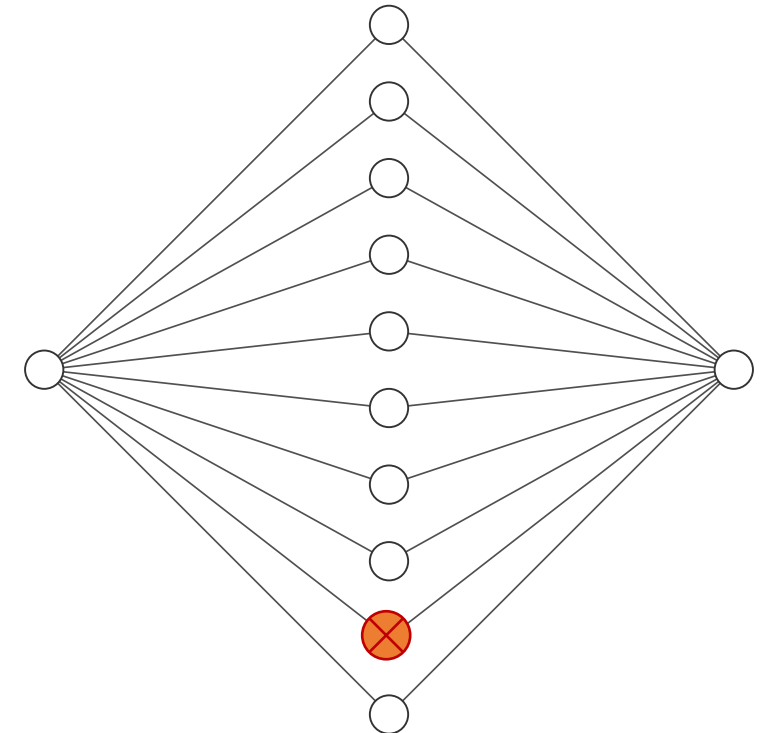
Dropout



Setting a proportion of hidden neurons to zero.

Normally done during training for learning robust hidden neuron representations.

Dropout $p = 0.2$

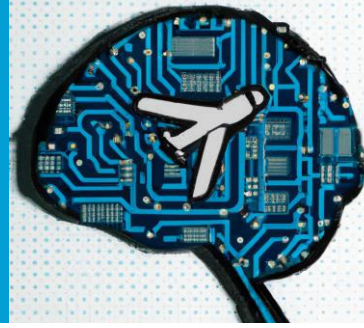


Input Layer $\in \mathbb{R}^1$

Hidden Layer $\in \mathbb{R}^{10}$

Output Layer $\in \mathbb{R}^1$

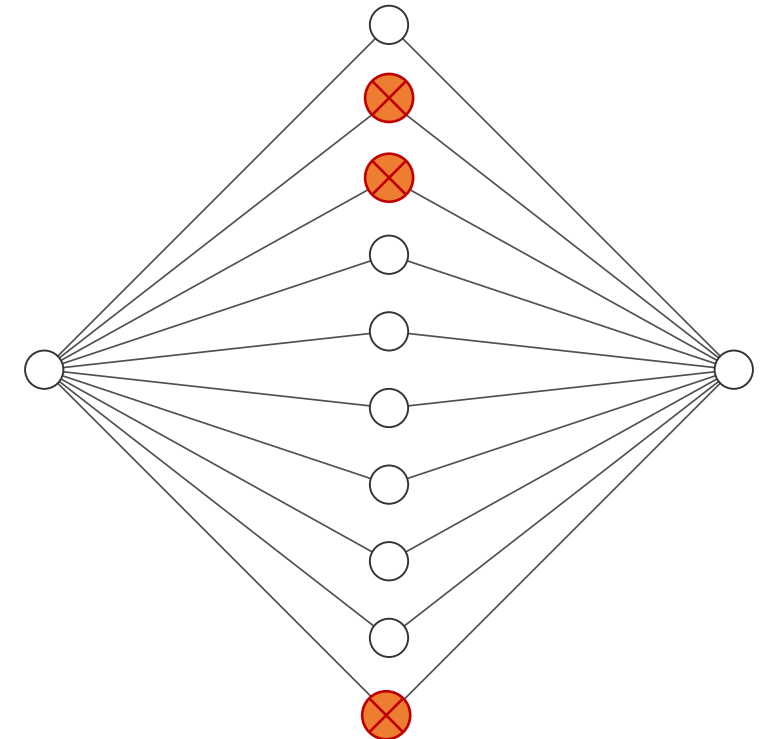
Dropout



Setting a proportion of hidden neurons to zero.

Normally done during training for learning robust hidden neuron representations.

Dropout $p = 0.2$

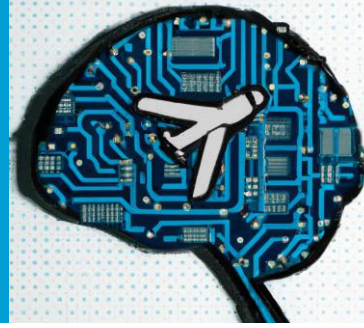


Input Layer $\in \mathbb{R}^1$

Hidden Layer $\in \mathbb{R}^{10}$

Output Layer $\in \mathbb{R}^1$

Dropout



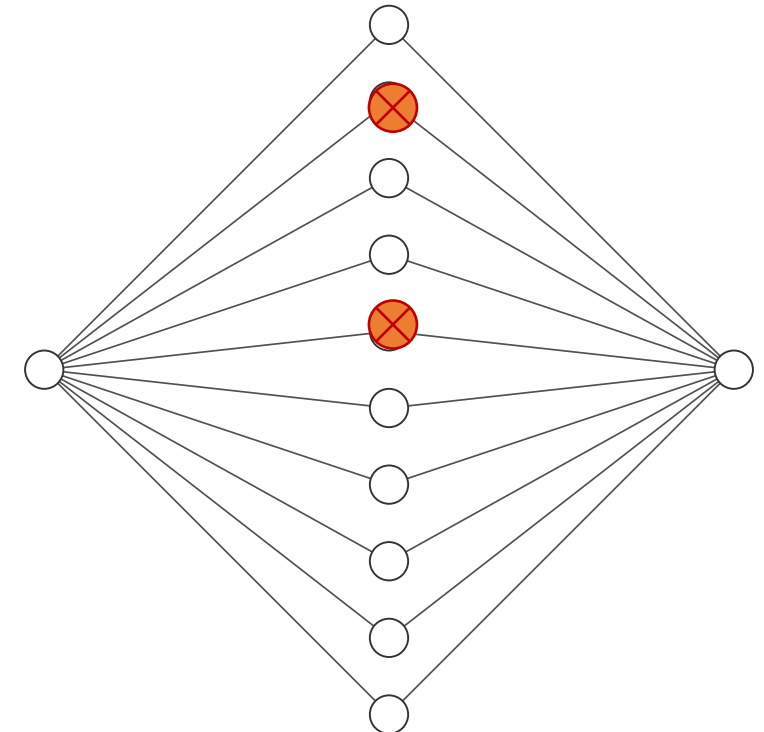
Setting a proportion of hidden neurons to zero.

Normally done during training for learning robust hidden neuron representations.

Code:

```
model = torch.nn.Sequential(  
    torch.nn.Linear(1, n_hidden_neurons),  
    torch.nn.Sigmoid(), torch.nn.Dropout(p=0.2),  
    torch.nn.Linear(n_hidden_neurons, 1)  
)
```

Dropout $p = 0.2$

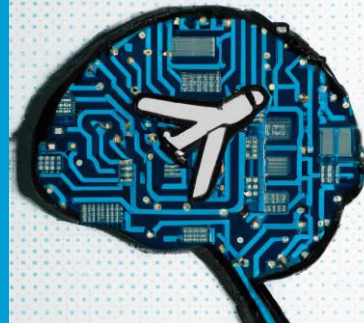


Input Layer $\in \mathbb{R}^1$

Hidden Layer $\in \mathbb{R}^{10}$

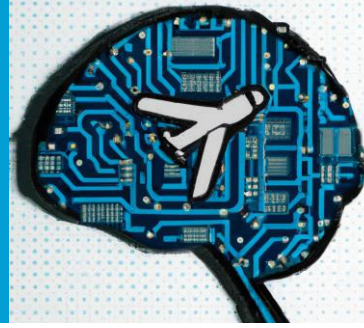
Output Layer $\in \mathbb{R}^1$

Dropout



- During training, weights are multiplied with $\frac{1}{1-p}$.
- For inference, dropout is not applied and weights are used as is.

Dropout

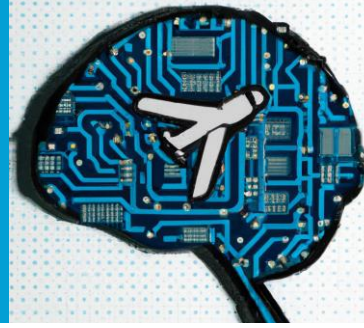


Can be maintained at inference time to get a probability distribution over the output.

Dropout is similar to ensemble learning, with as differences that:

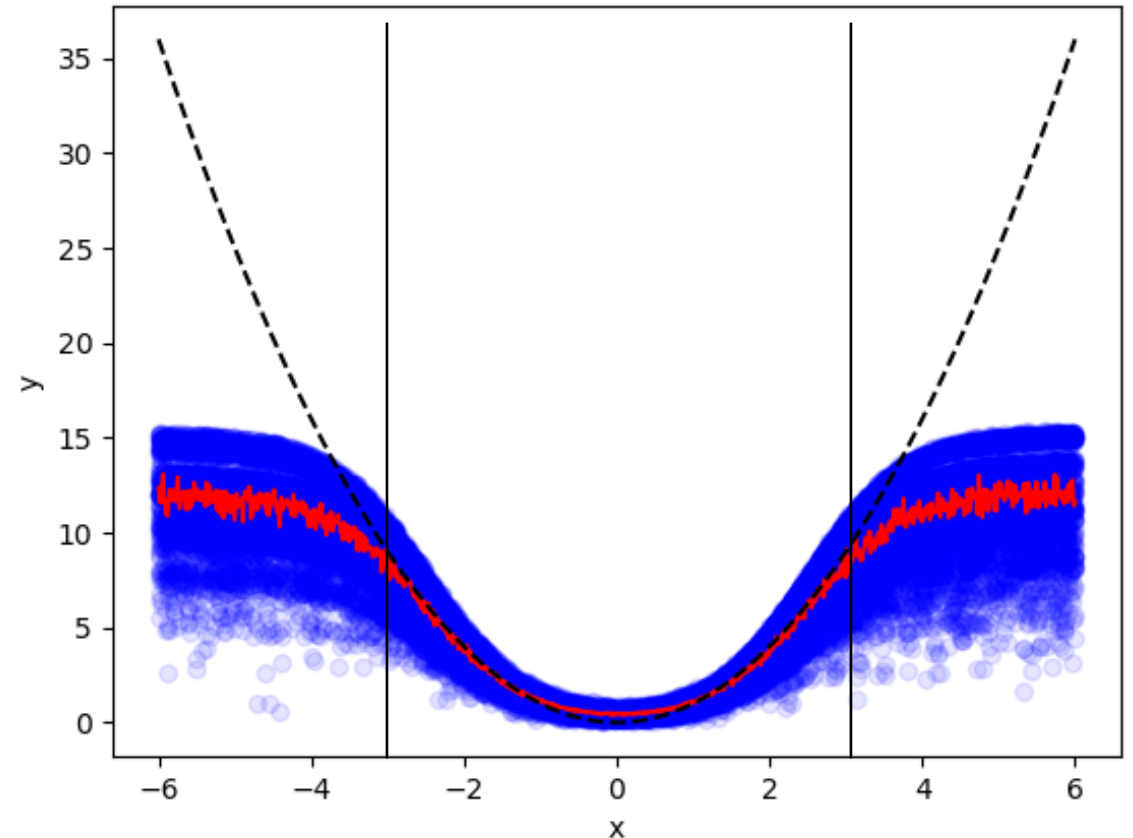
- All “networks” share the same weights
- All “networks” encounter the same data

Dropout

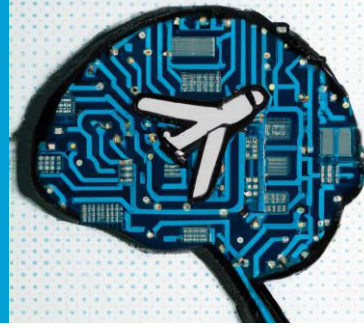


Uncertainty is larger for the intervals in which there is no training data.

Dropout with $p = 0.2$ of a network with 30 hidden neurons, when learning in the interval $x \in [-3, 3]$.



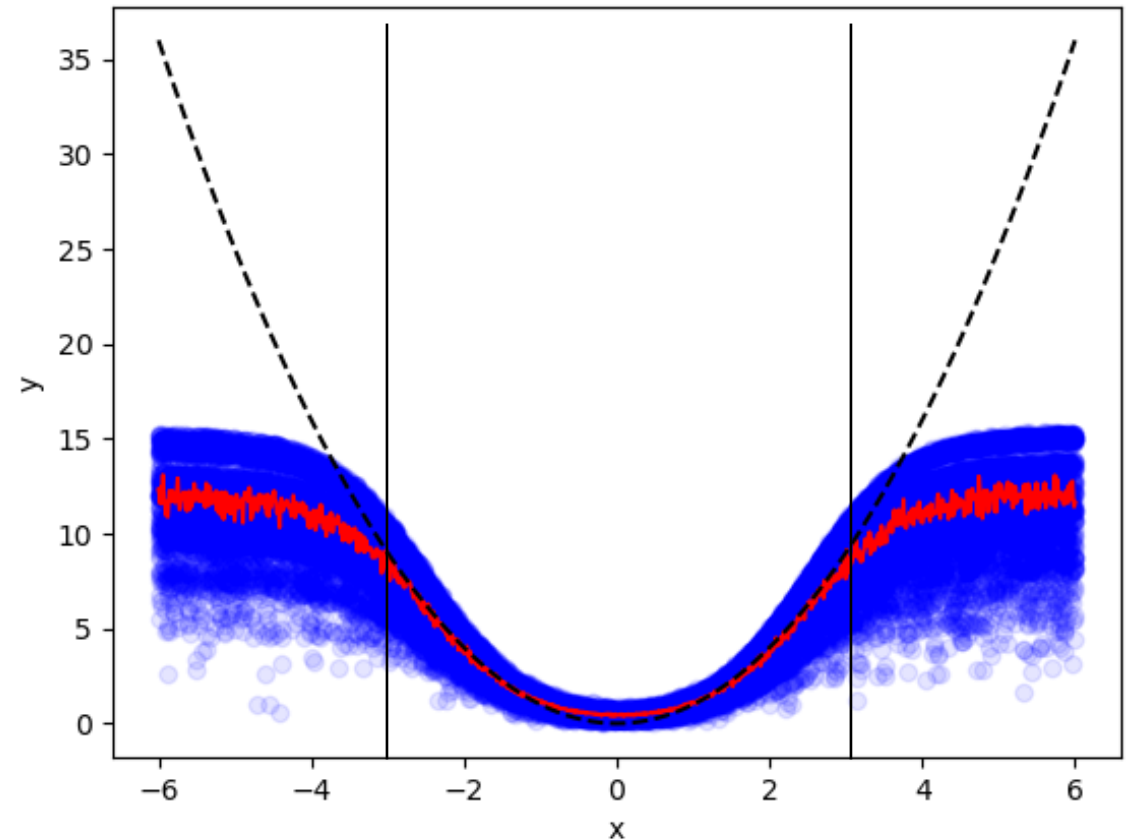
Dropout

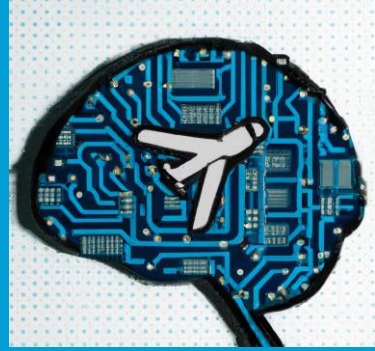


Uncertainty is larger for the intervals in which there is no training data.

Why do you think that the network underestimates the values outside $[-3,3]$?

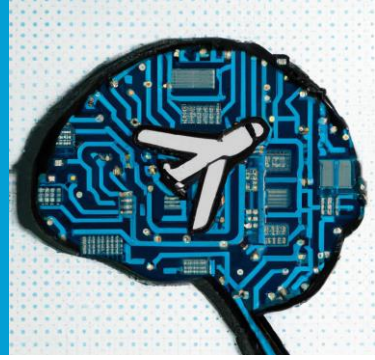
Dropout with $p = 0.2$ of a network with 30 hidden neurons, when learning in the interval $x \in [-3,3]$.



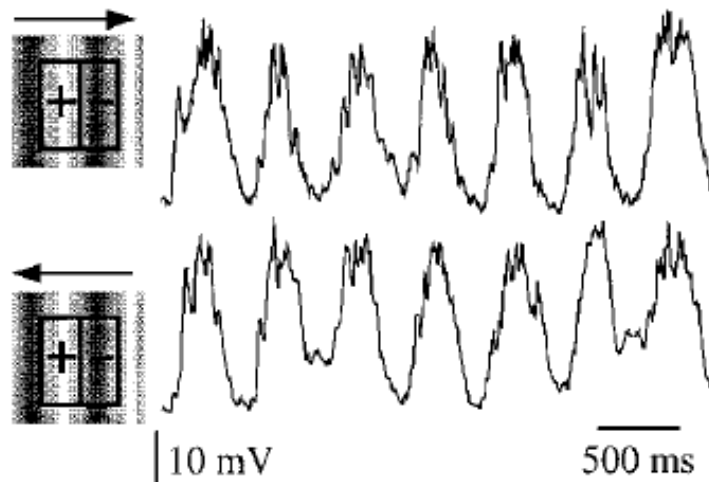


Convolutional neural networks

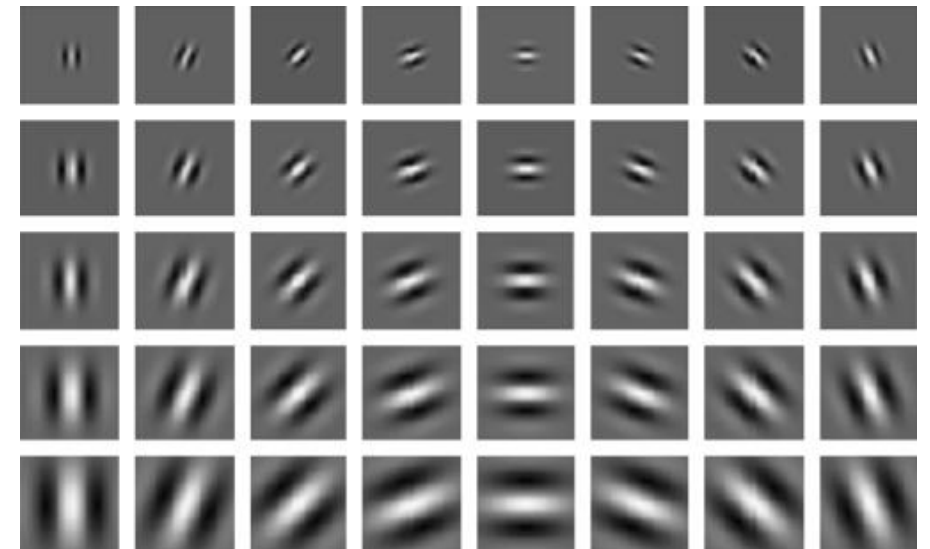
Cat visual system



Responses of “simple” cells in the cat visual system are similar to Gabor convolutional filters.



Response of a cat simple cell to a moving grating

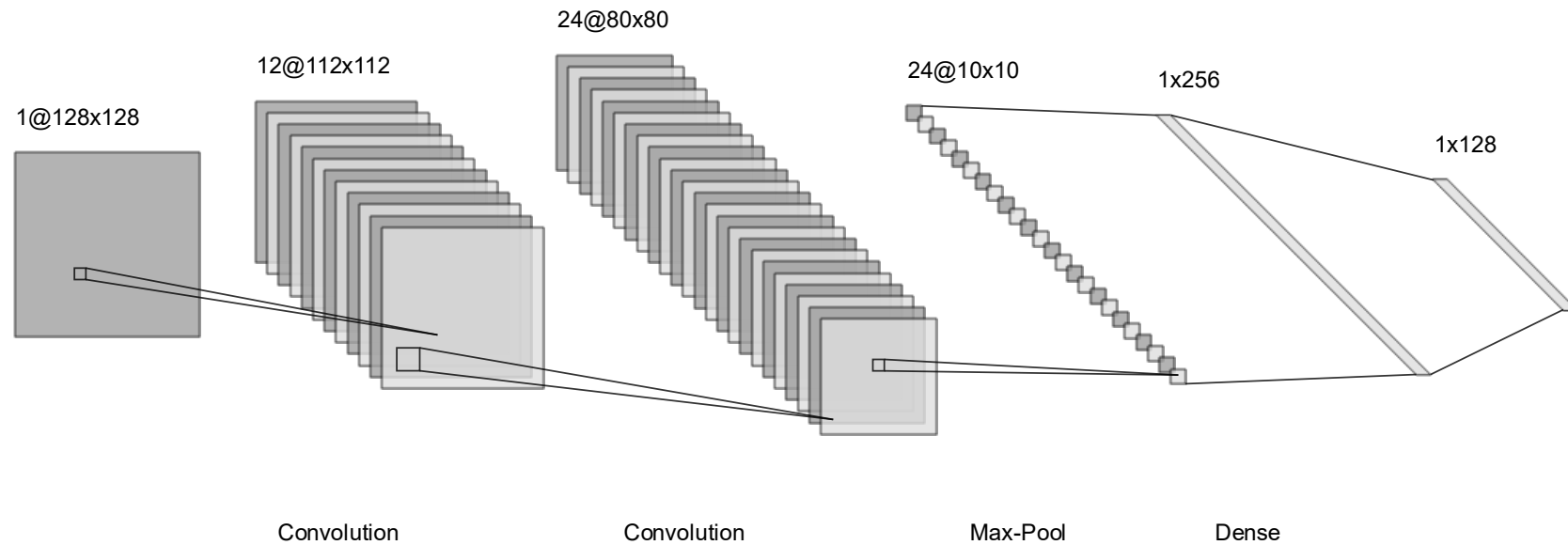
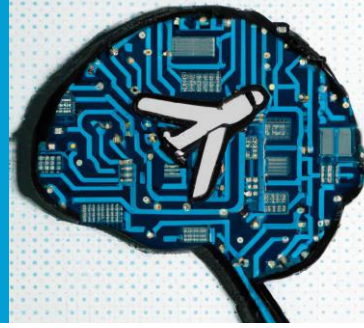


Gabor filters

From: Jagadeesh, B., Wheat, H. S., Kontsevich, L. L., Tyler, C. W., & Ferster, D. (1997). Direction selectivity of synaptic potentials in simple cells of the cat visual cortex. *Journal of neurophysiology*, 78(5), 2772-2789.

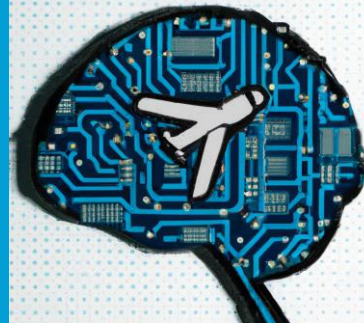
From: Li, H., Zhang, Q., & Sun, Z. (2017). Iris recognition on mobile devices using near-infrared images. In *Human Recognition in Unconstrained Environments* (pp. 103-117). Academic Press.

Convolutional neural networks



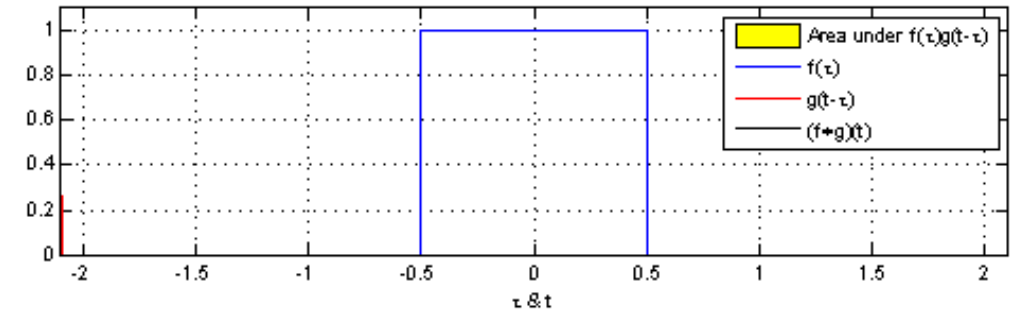
LeCun, Yann, and Yoshua Bengio. "Convolutional networks for images, speech, and time series."
The handbook of brain theory and neural networks 3361 (1995): 310.

Convolution



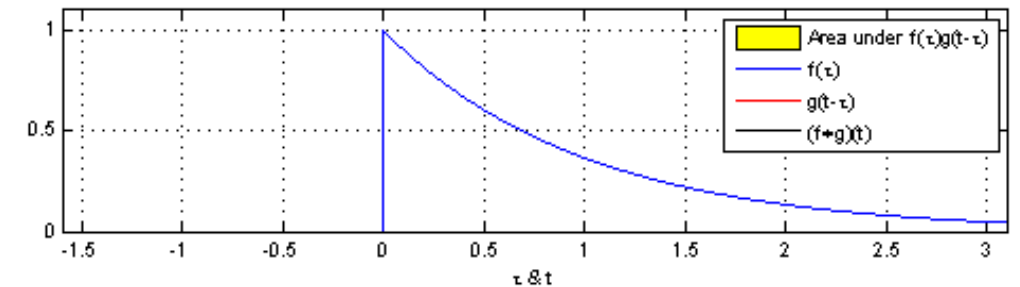
Common mathematical definition:

$$f(t) * g(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$



Discrete convolution:

$$f(t) * g(t) = \sum_{i=-\infty}^{\infty} f(i)g(t - i)$$

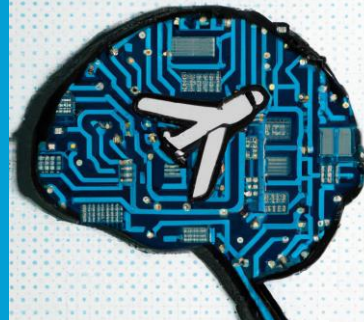


Animations from



WIKIPEDIA
The Free Encyclopedia

Convolution

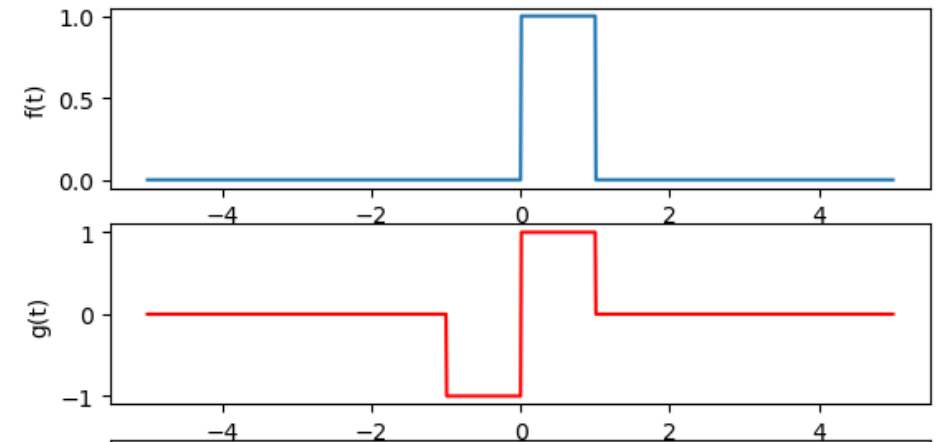


Common mathematical definition:

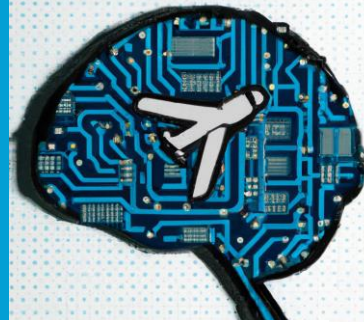
$$f(t) * g(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

Discrete convolution:

$$f(t) * g(t) = \sum_{i=-\infty}^{\infty} f(i)g(t - i)$$



Convolution

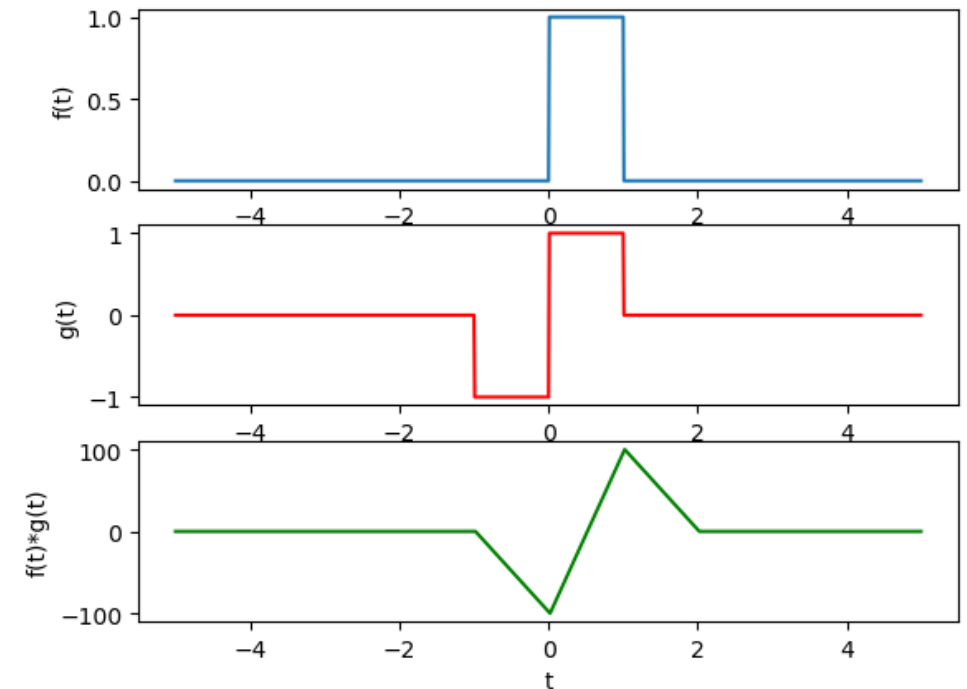


Common mathematical definition:

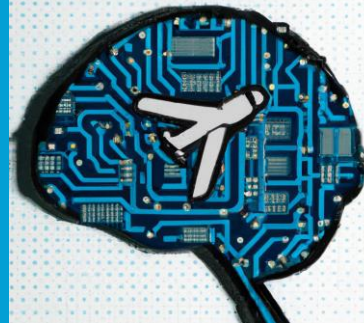
$$f(t) * g(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

Discrete convolution:

$$f(t) * g(t) = \sum_{i=-\infty}^{\infty} f(i)g(t - i)$$



Cross-correlation

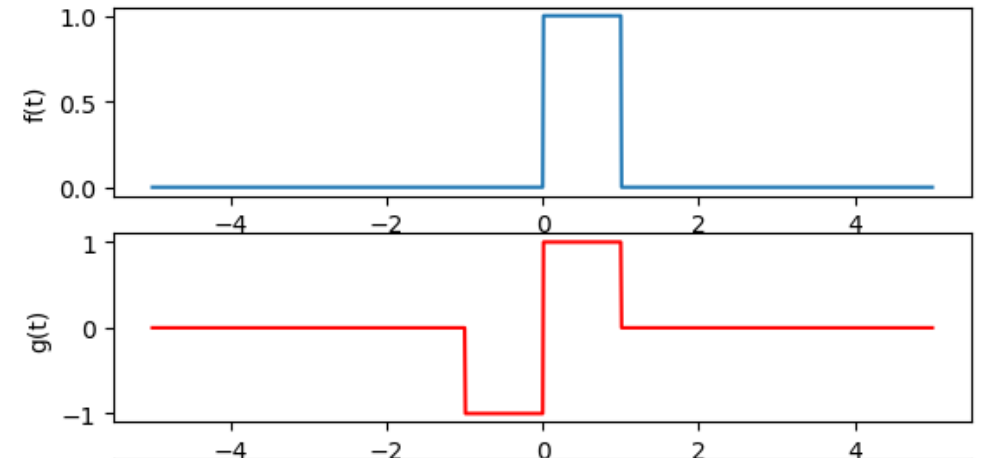


Common mathematical definition:

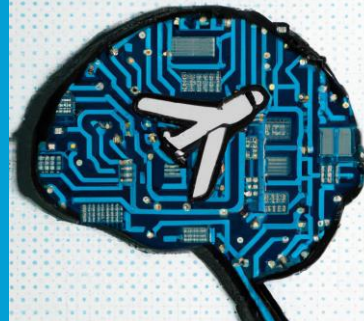
$$f(t) \star g(t) = \int_{-\infty}^{\infty} f(\tau)g(t + \tau)d\tau$$

Discrete cross-correlation:

$$f(t) \star g(t) = \sum_{i=-\infty}^{\infty} f(i)g(t + i)$$



Cross-correlation

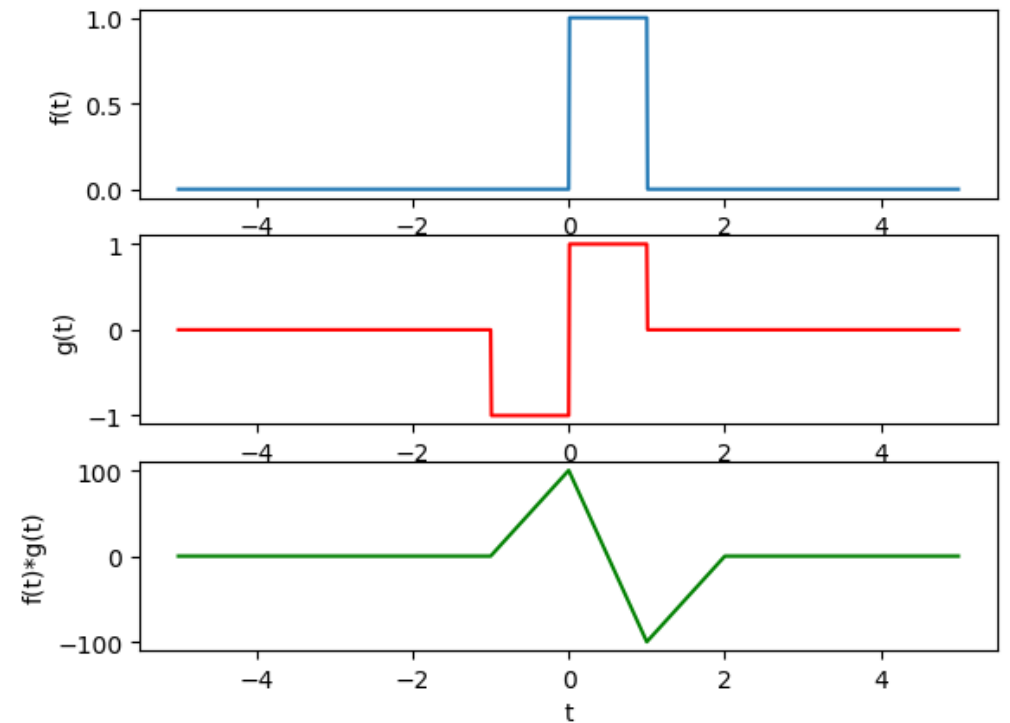


Common mathematical definition:

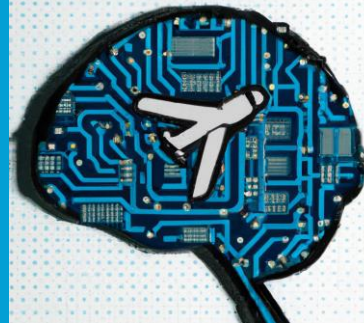
$$f(t) \star g(t) = \int_{-\infty}^{\infty} f(\tau)g(t + \tau)d\tau$$

Discrete cross-correlation:

$$f(t) \star g(t) = \sum_{i=-\infty}^{\infty} f(i)g(t + i)$$

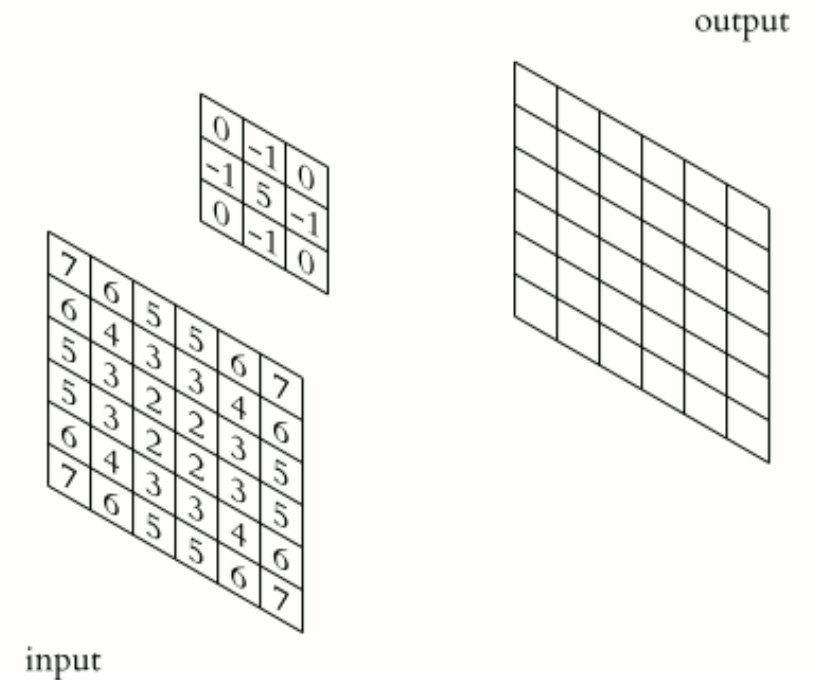


2D “convolution”

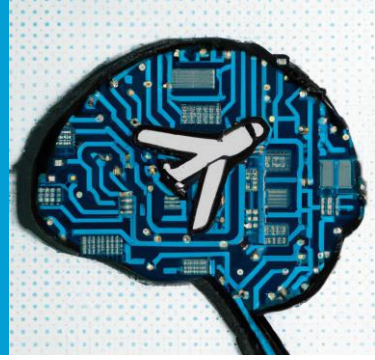


Common implementation in neural network libraries:

$$\begin{aligned} S(i, j) &= (K * I)(i, j) \\ &= \sum_m \sum_n I(i + m, j + n) K(m, n) \end{aligned}$$



2D “convolution”

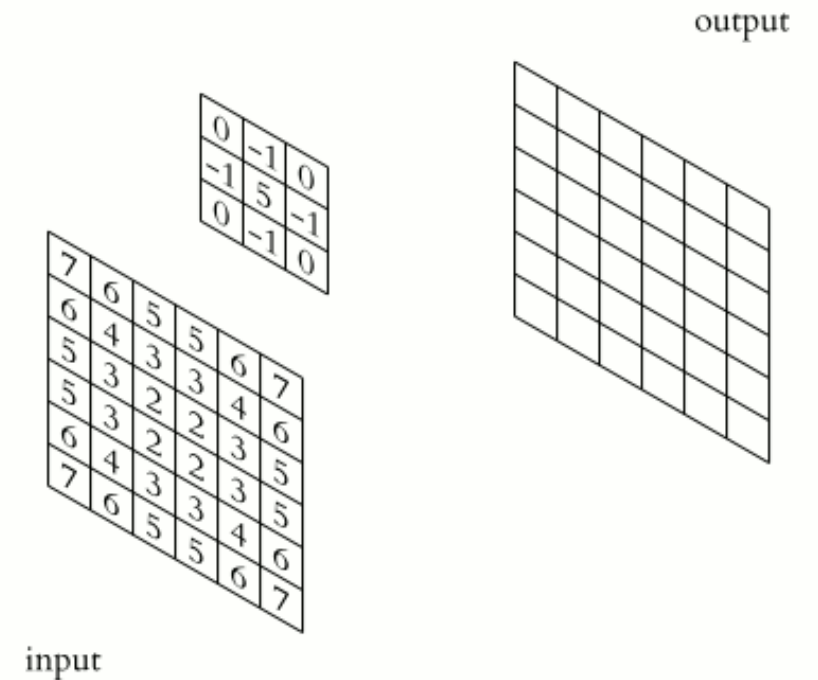


Note:

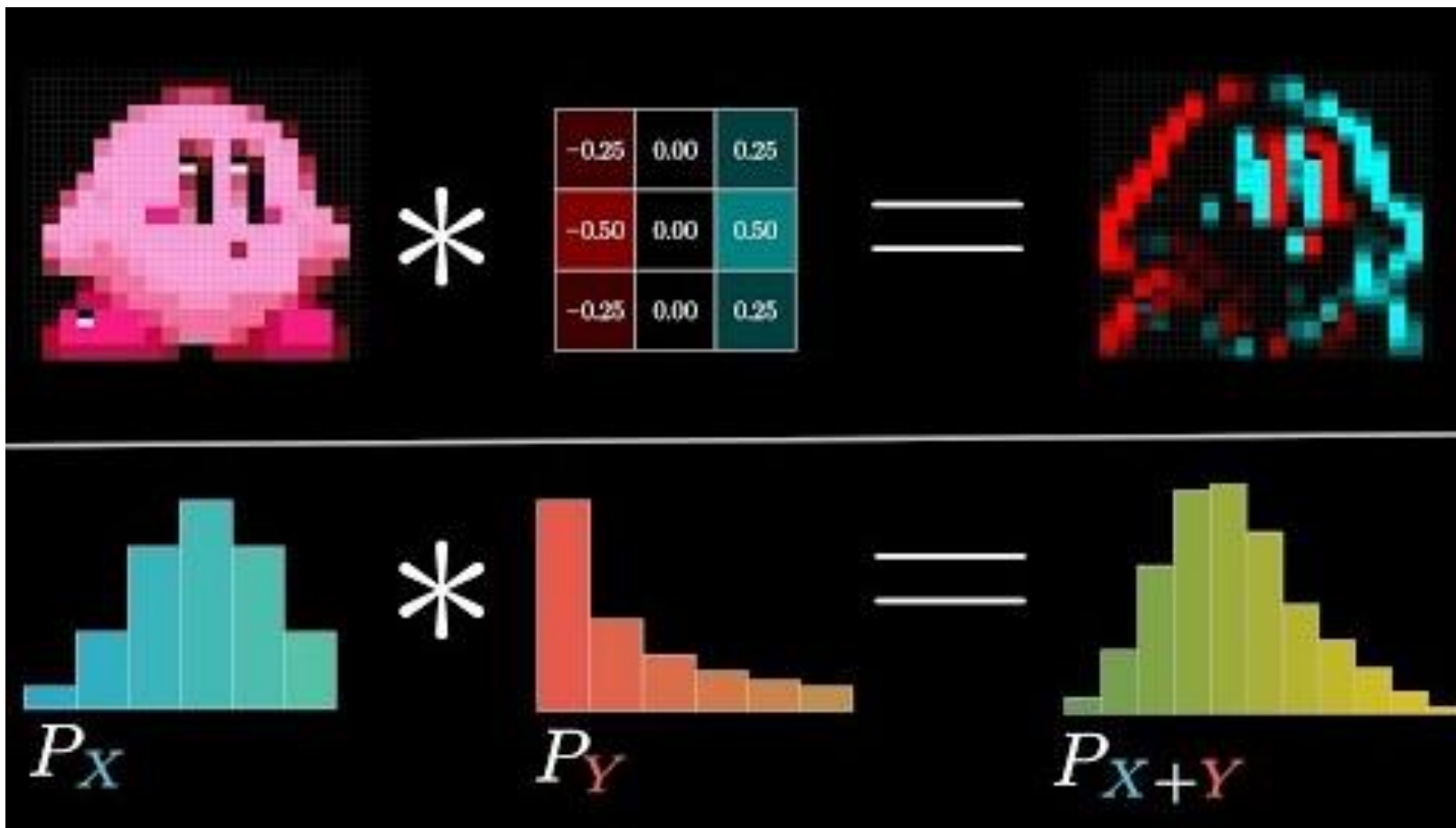
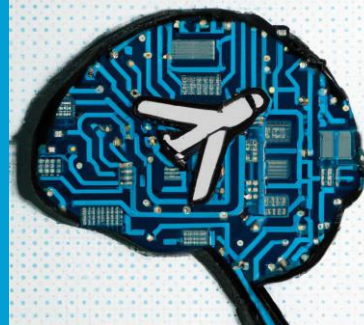
In the animation, values outside are assumed to be constant, in order to keep the same size as the input. This is called “constant padding” (‘replicate’ in PyTorch), with convolution mode ‘same’.

Other mode: ‘valid’.

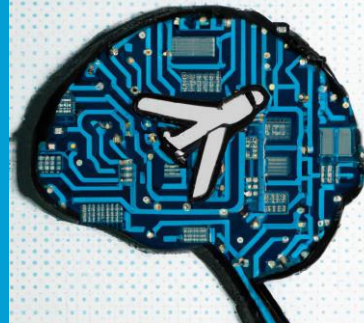
Other padding: ‘zeros’, ‘reflect’, ‘circular’.



2D convolution

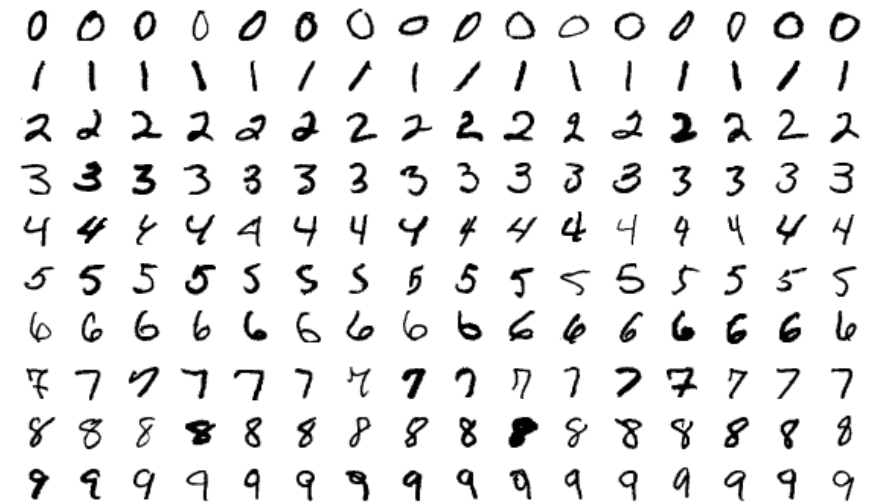


CNN for MNIST data set



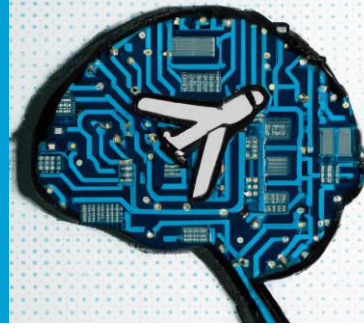
An early application of convolutional neural networks (CNNs) was for optical character recognition (OCR).

OCR is extensively used, e.g., for automatic sorting of physical mail.

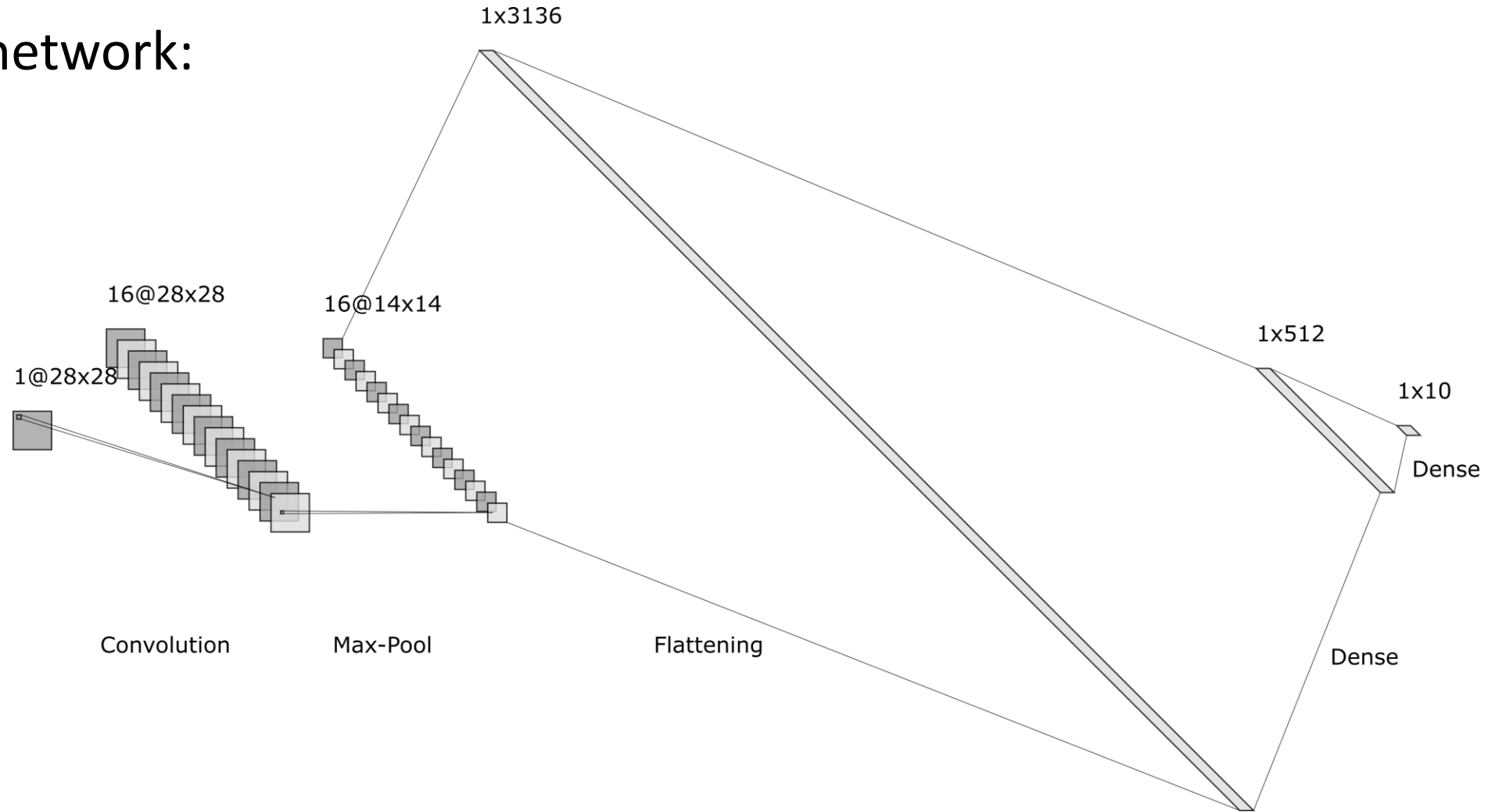


"THE MNIST DATABASE of handwritten digits". Yann LeCun, Courant Institute, NYU Corinna Cortes, Google Labs, New York Christopher J.C. Burges, Microsoft Research, Redmond.

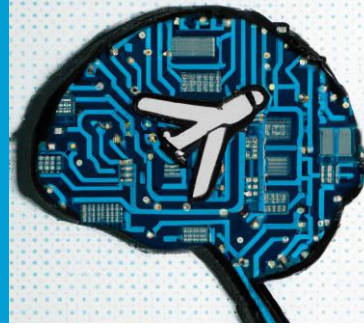
CNN for MNIST data set



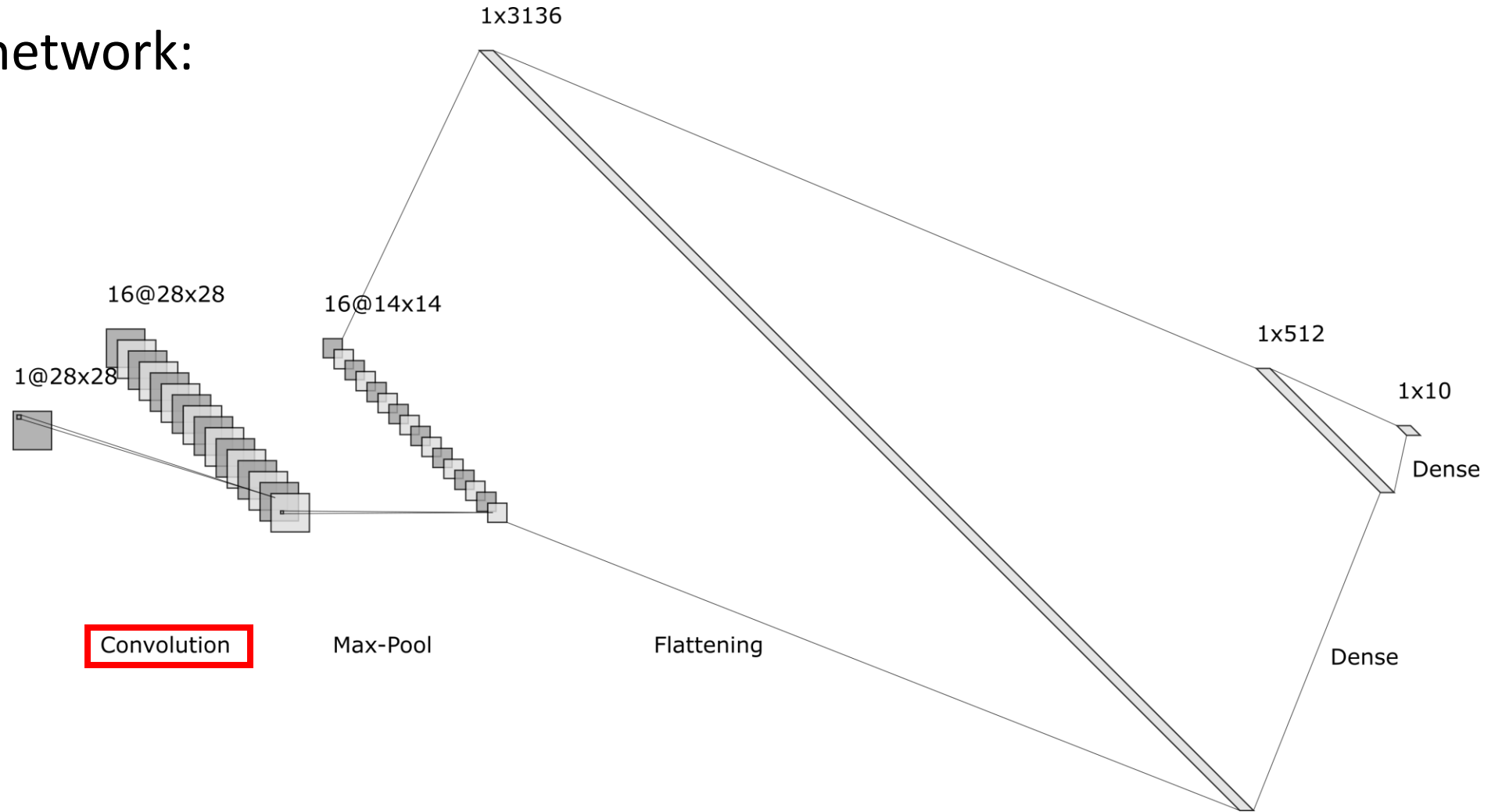
Example network:



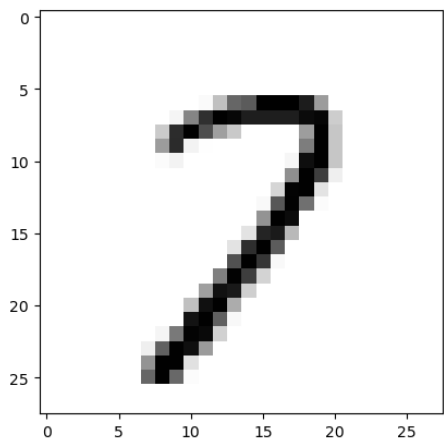
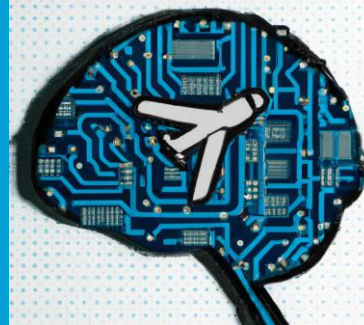
CNN for MNIST data set



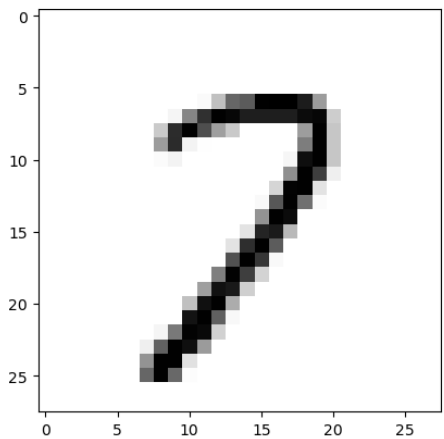
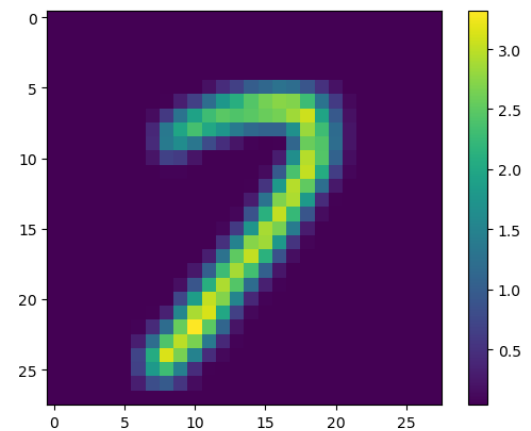
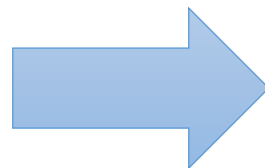
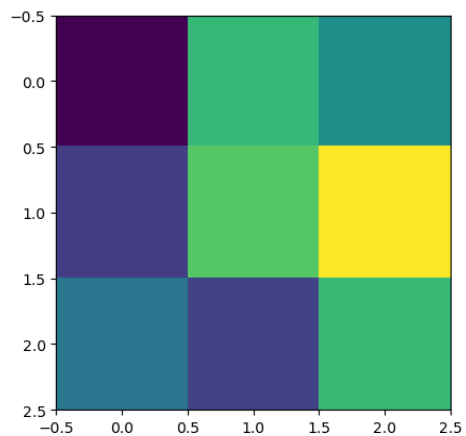
Example network:



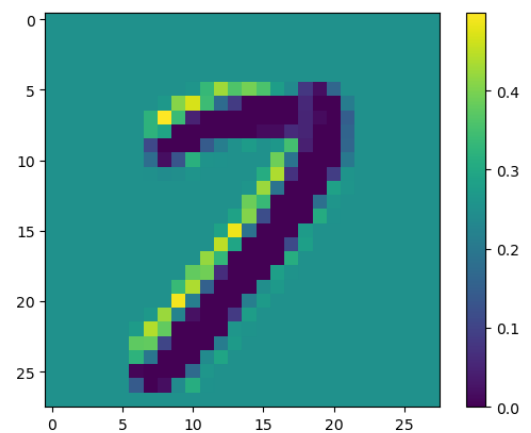
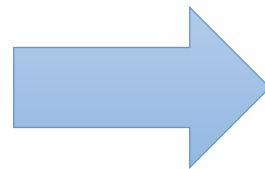
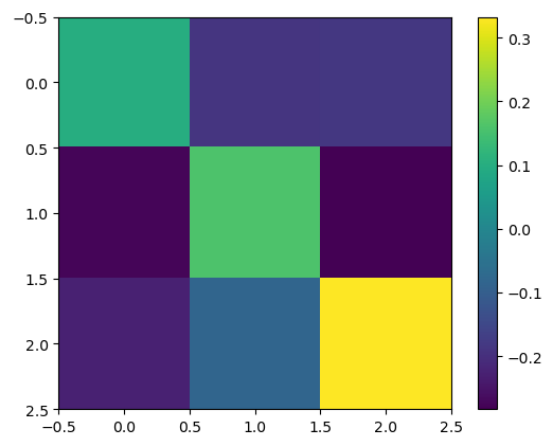
Convolution



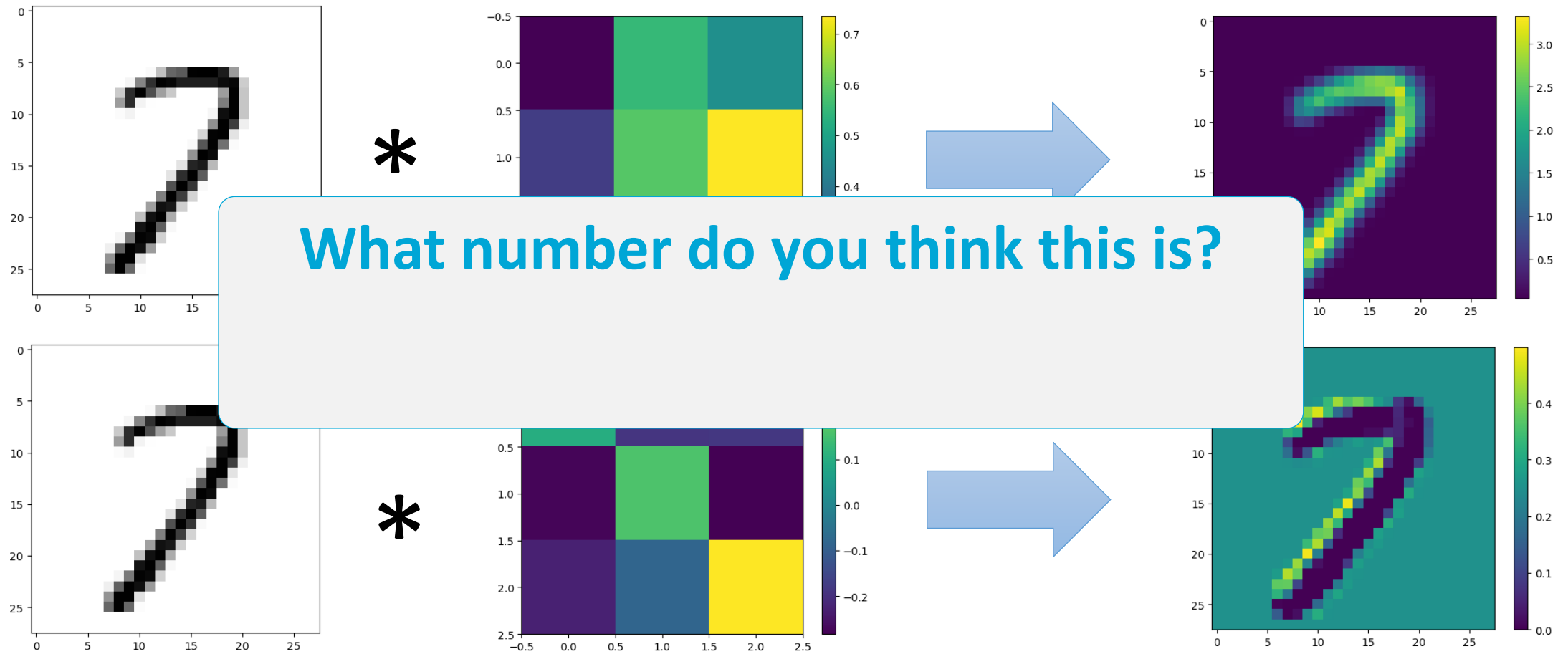
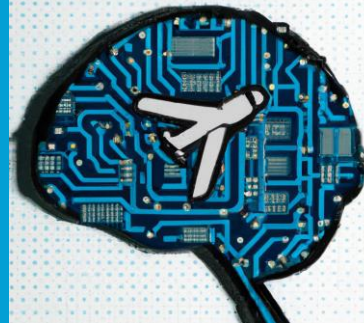
*



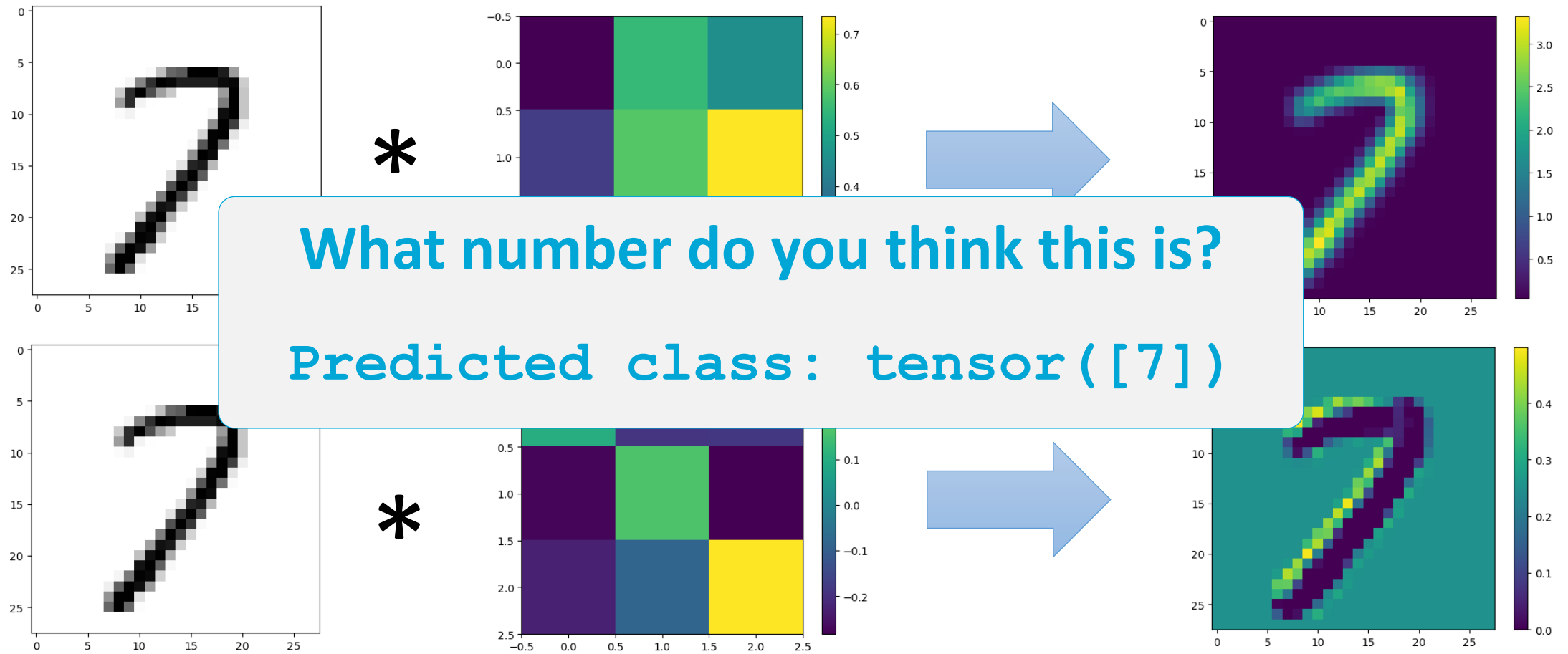
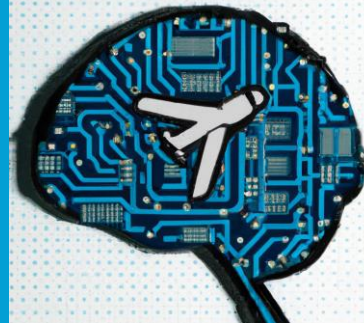
*



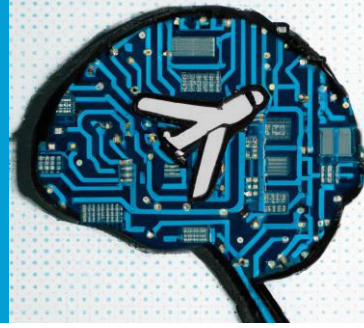
Convolution



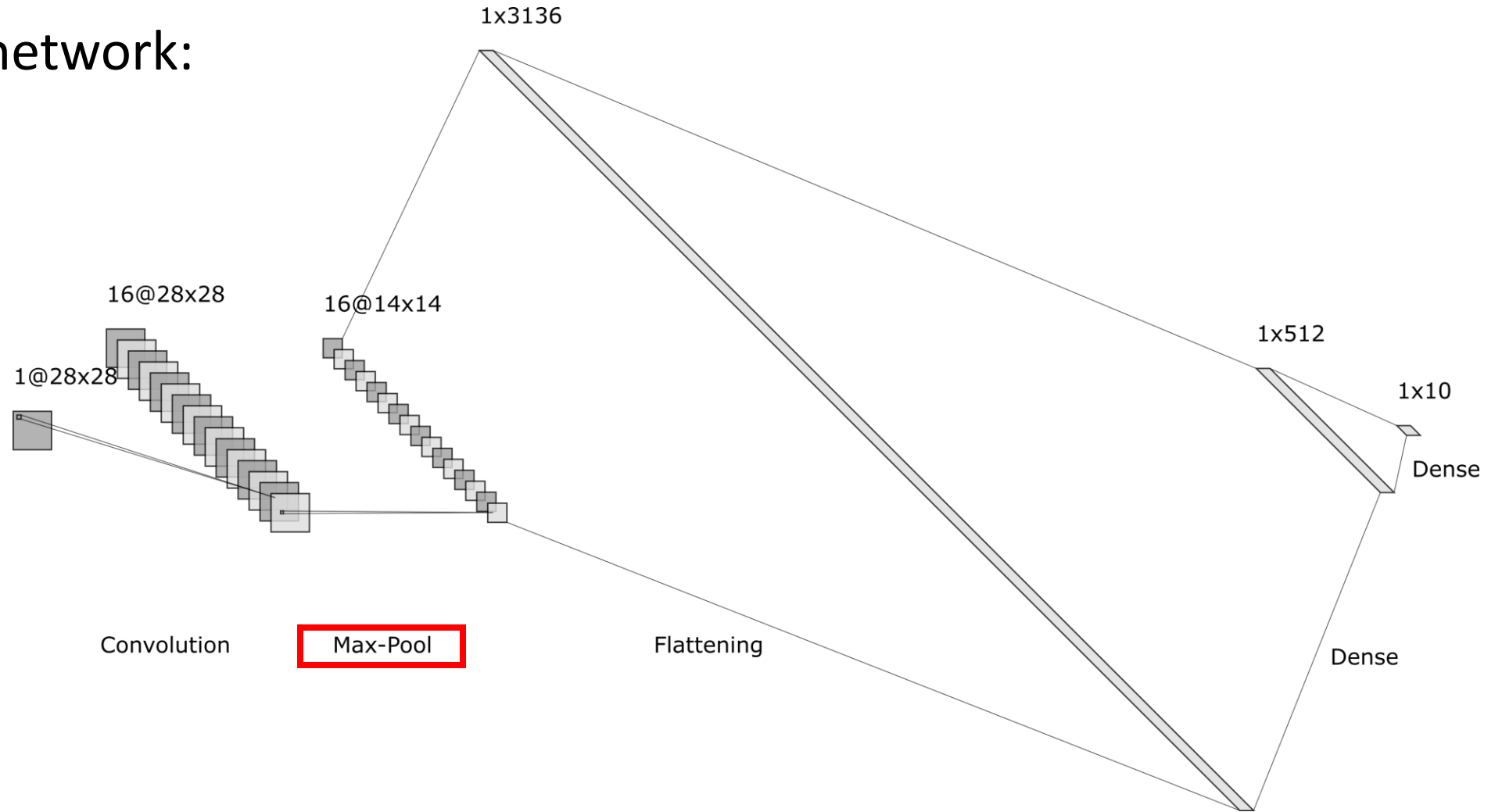
Convolution



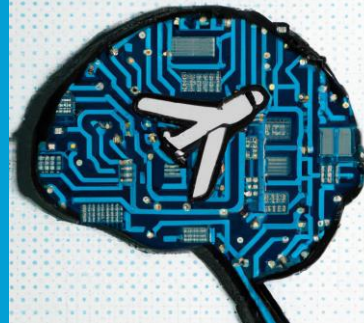
CNN for MNIST data set



Example network:

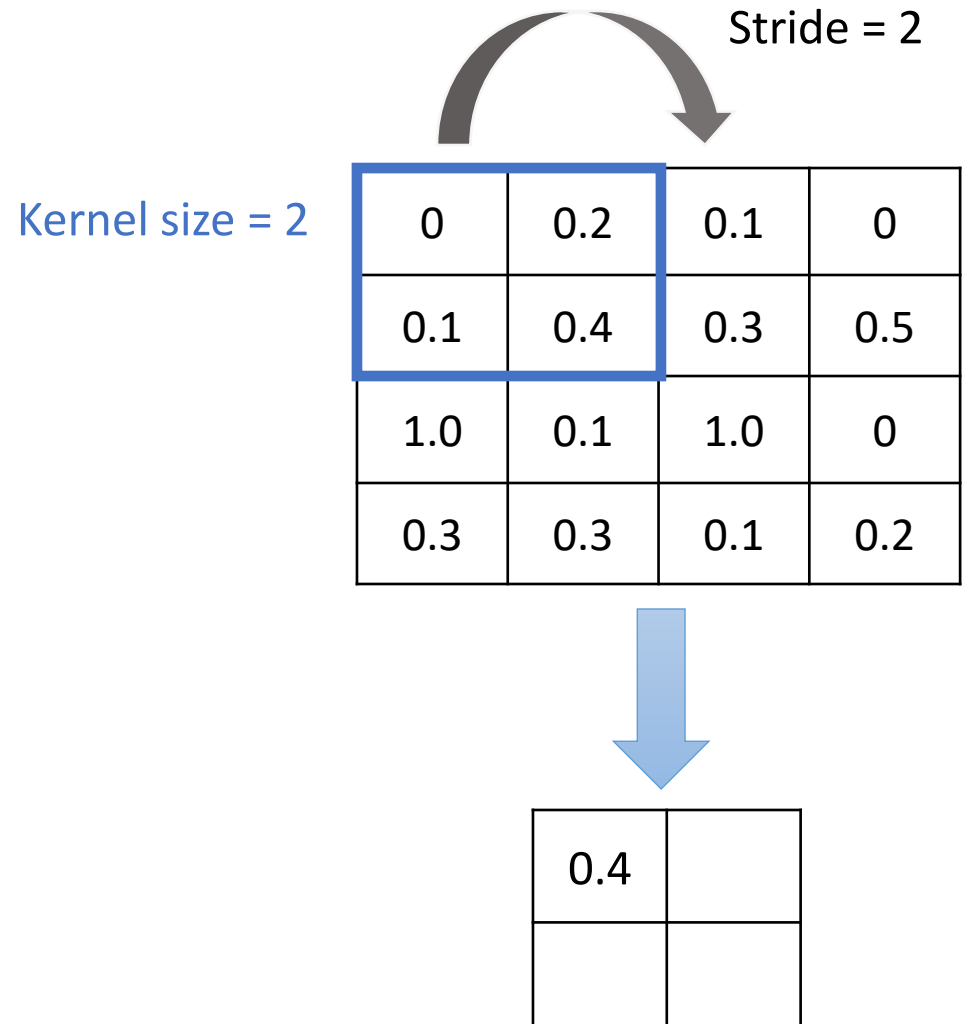


Max-pooling

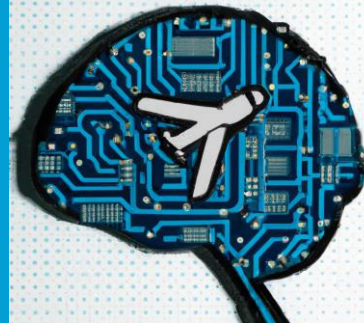


Goals:

- Reduce dimensionality
- Translation invariance

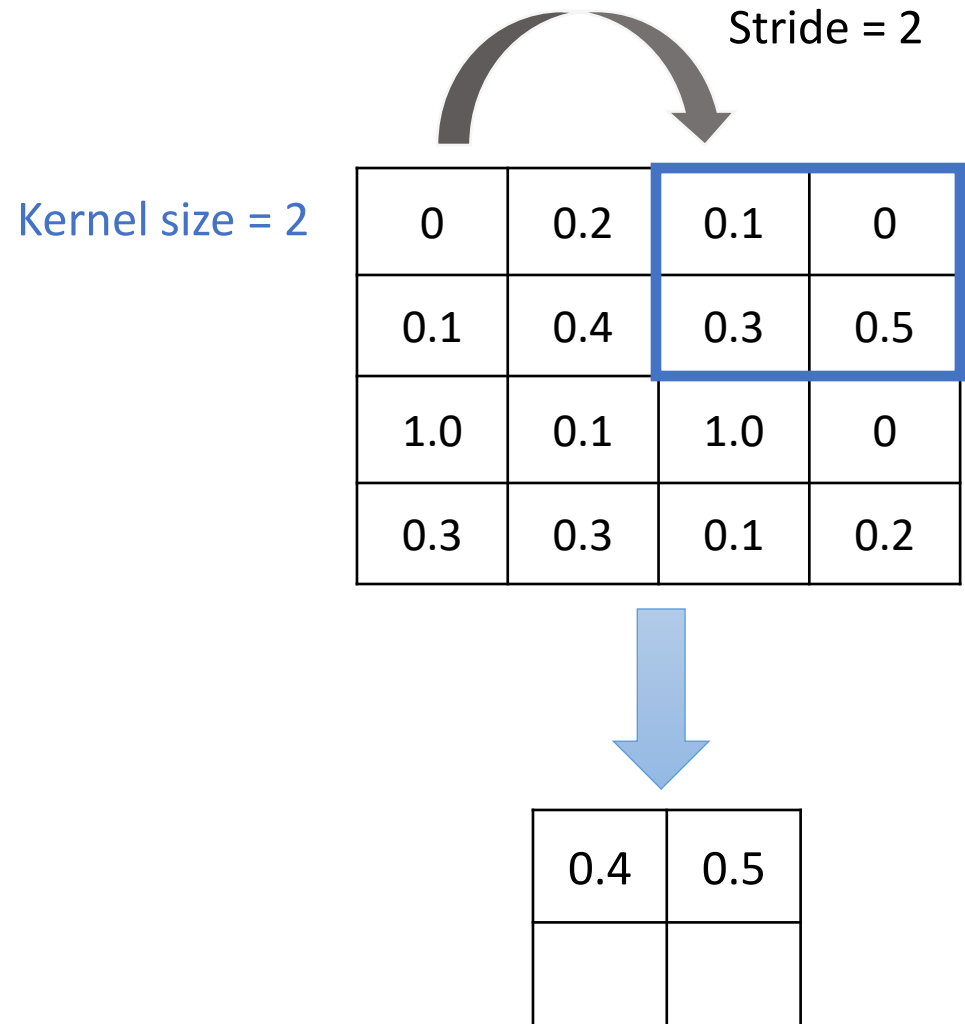


Max-pooling

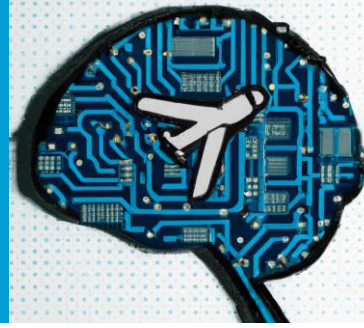


Goals:

- Reduce dimensionality
- Translation invariance

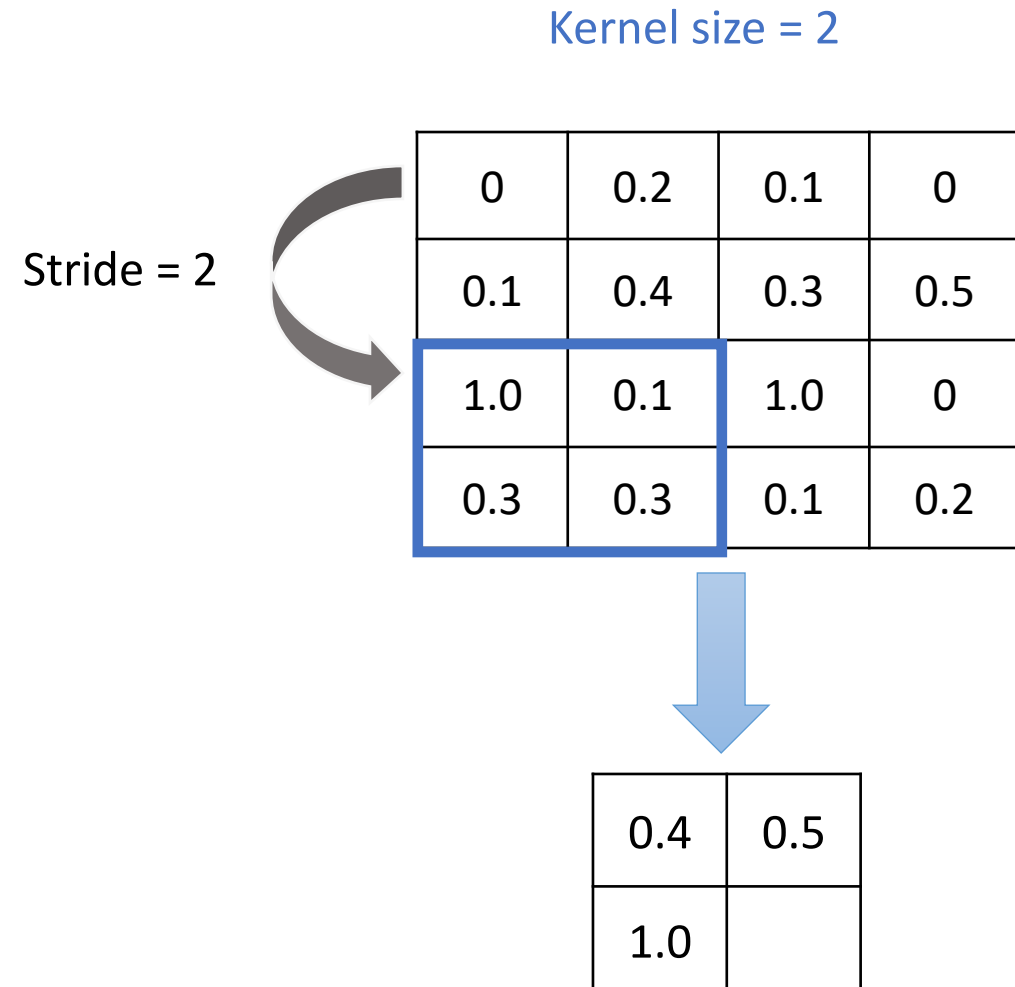


Max-pooling

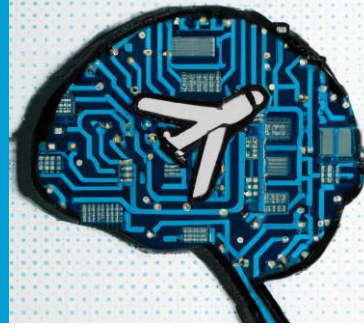


Goals:

- Reduce dimensionality
- Translation invariance

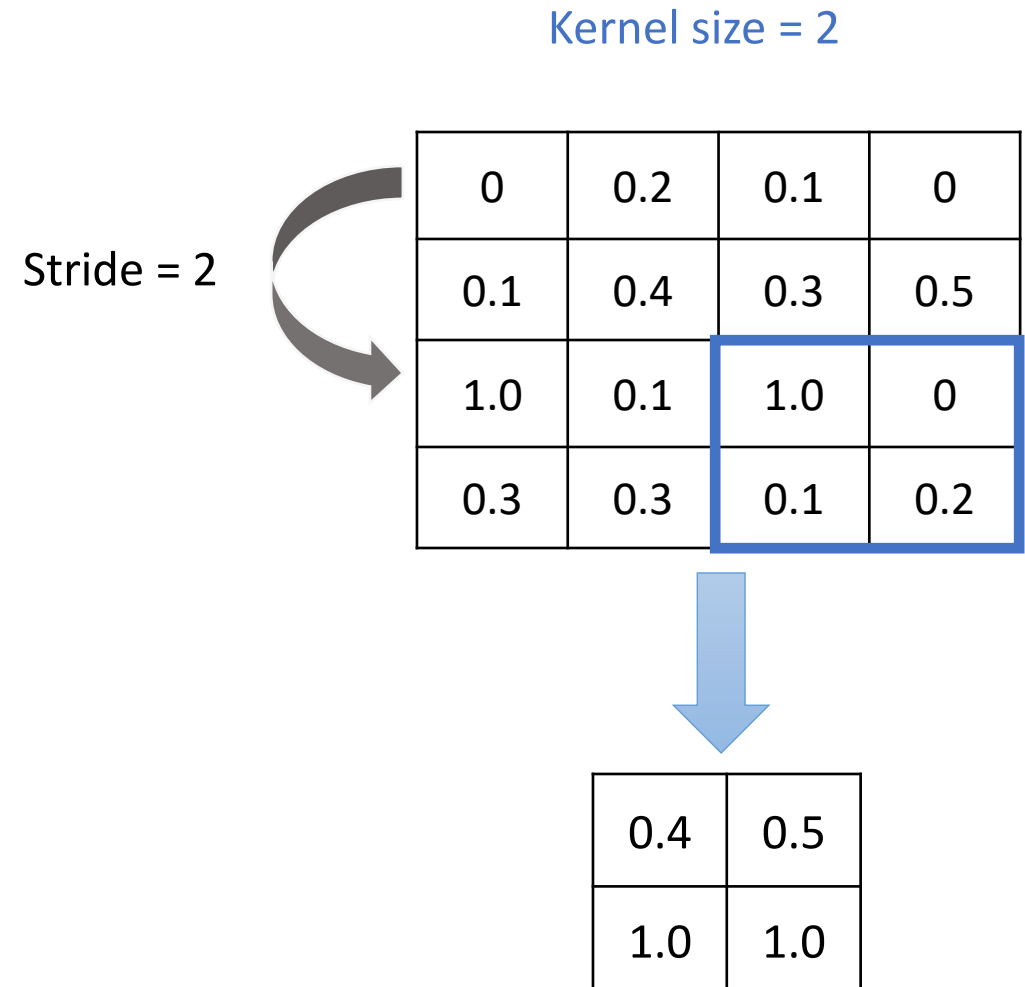


Max-pooling

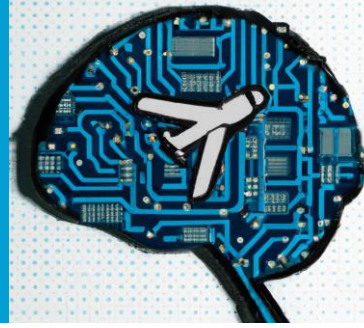


Goals:

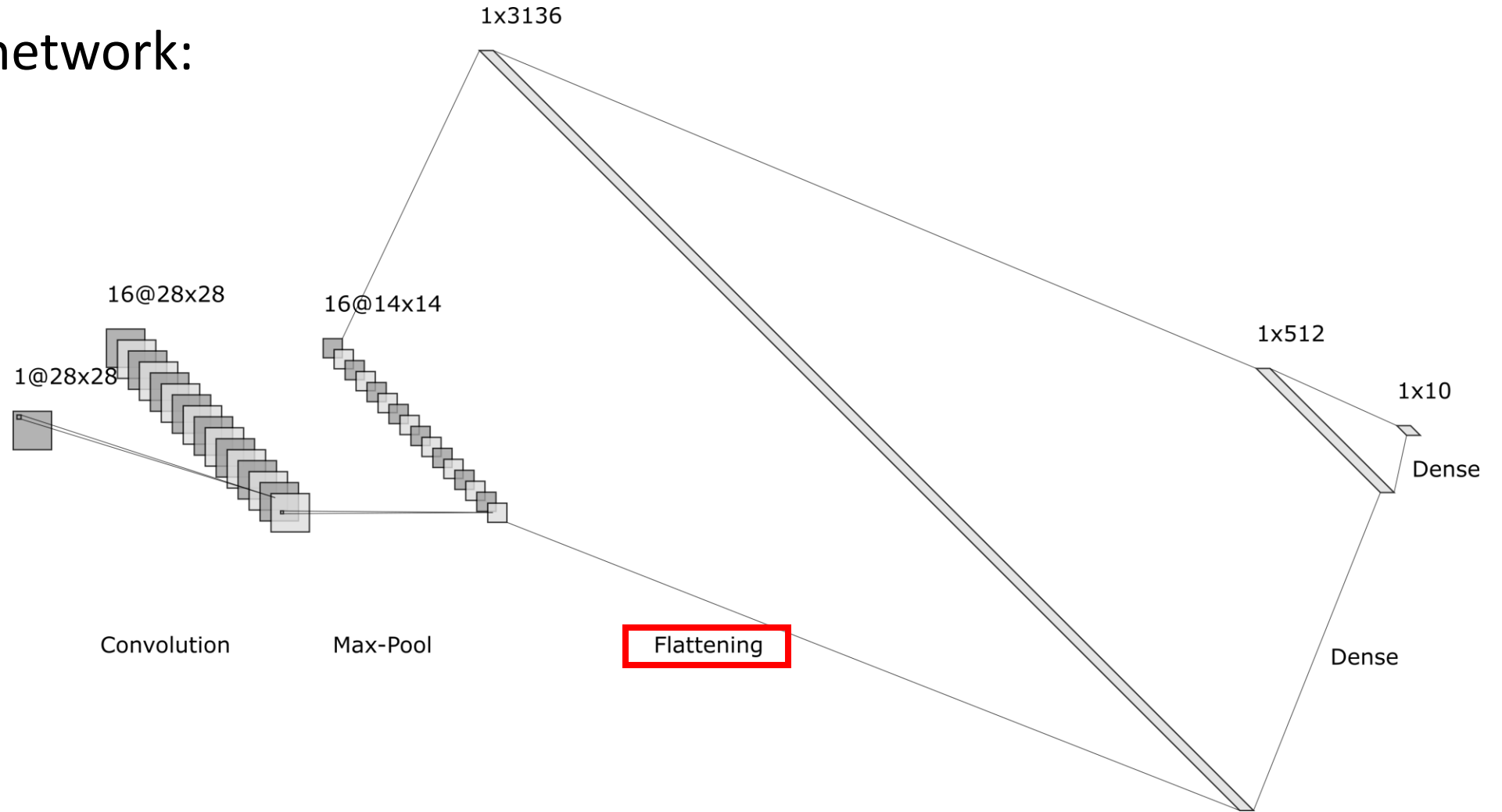
- Reduce dimensionality
- Translation invariance



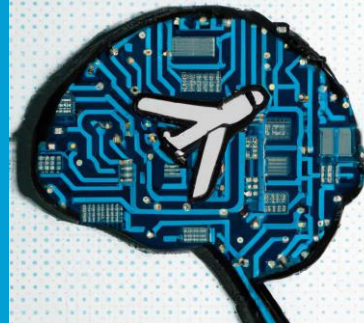
CNN for MNIST data set



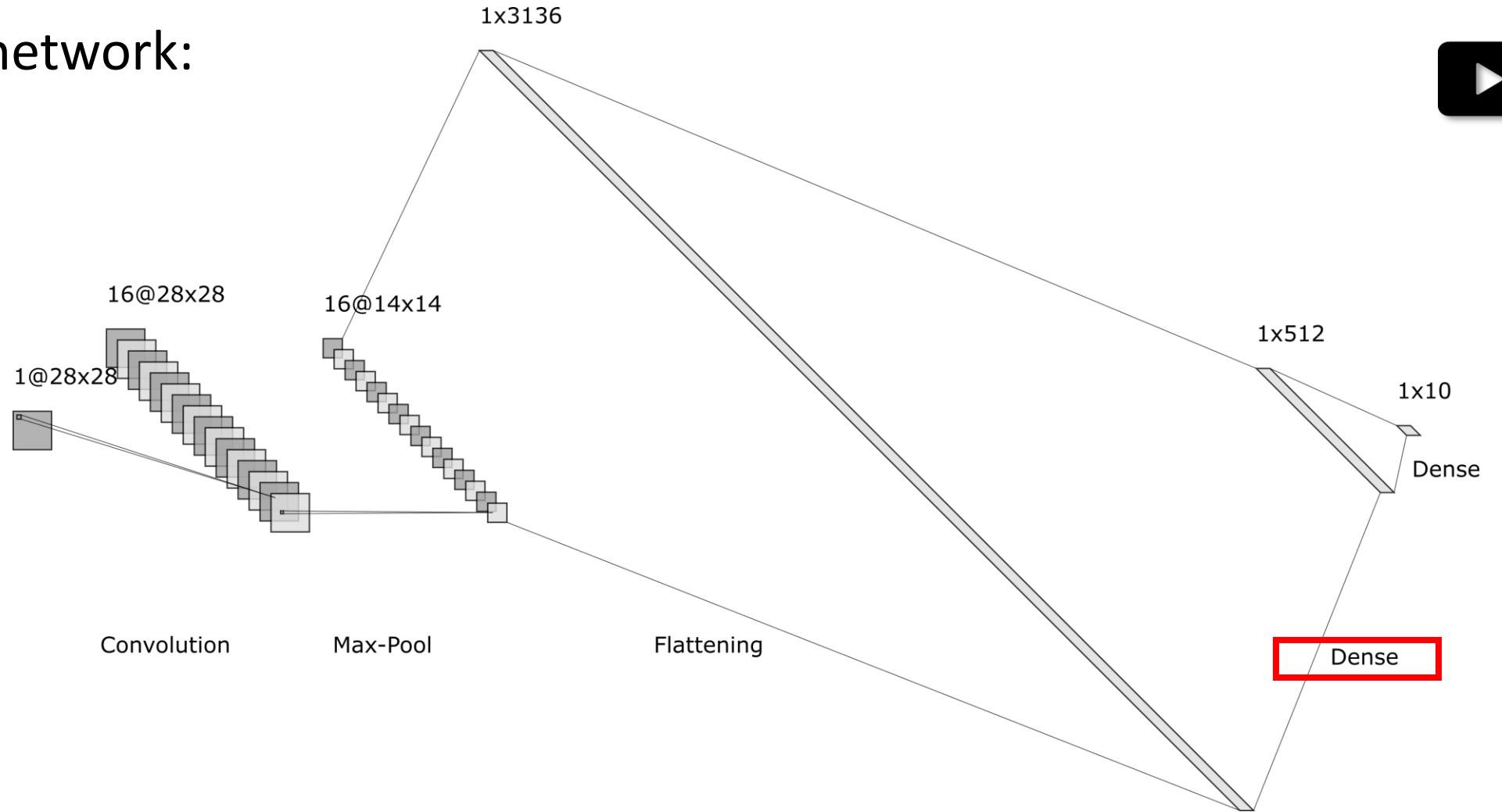
Example network:



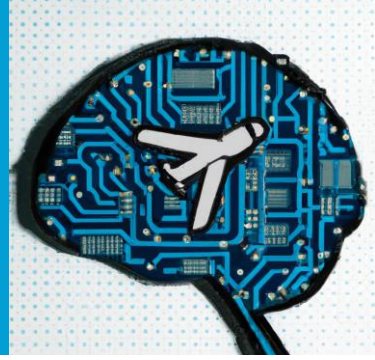
CNN for MNIST data set



Example network:



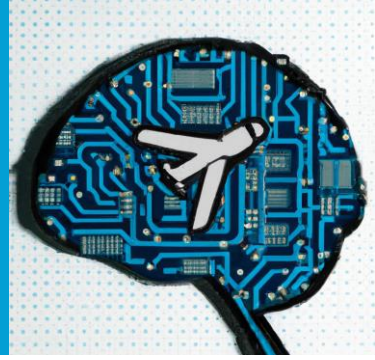
Why have CNNs been so important?



CNNs ... :

- simplify the learning problem by means of *weight sharing*

Why have CNNs been so important?



CNNs ... :

- simplification of the task

Illustration with our tiny MNIST CNN:

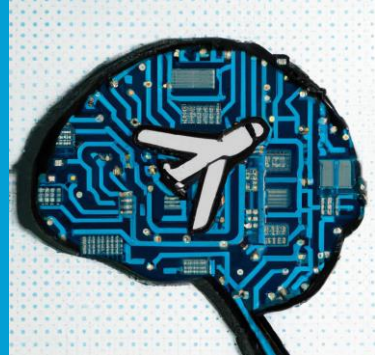
Convolution:

Kernel size of 3x3, 16 features = 9 x 16 weights (144)

Dense connections:

Image size of 28x28 = 784 inputs and 16 feature maps of 28 x 28 hidden neurons = 12.544 hidden neurons, gives 9.834.496 dense connections.

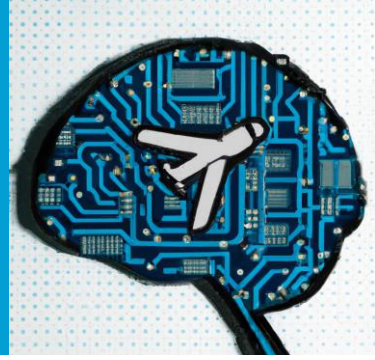
Why have CNNs been so important?



CNNs ... :

- simplify the learning problem by means of *weight sharing*
- allow to learn features of features, i.e., a *feature hierarchy*

Feature hierarchy



Subsequent convolutional layers learn increasingly complex features at increasingly larger spatial scales.

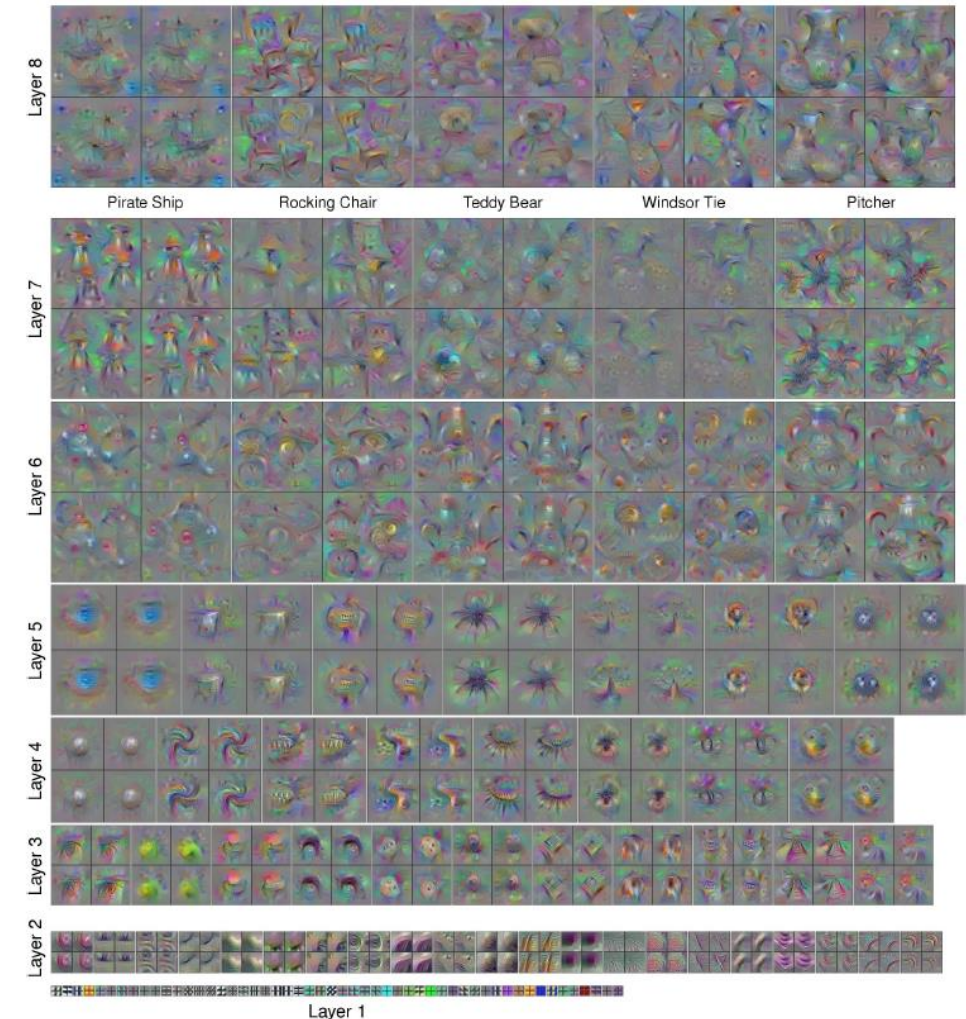
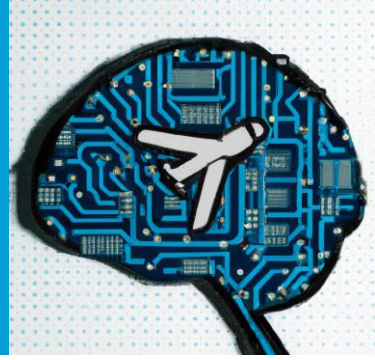


Figure from:

Yosinski, J., Clune, J., Nguyen, A., Fuchs, T., & Lipson, H. (2015). Understanding neural networks through deep visualization. arXiv preprint arXiv:1506.06579.

Receptive field size



Larger spatial scales:

The receptive field size of features increases for higher layers. The exact size can be calculated based on the type of padding, kernel size, and stride.

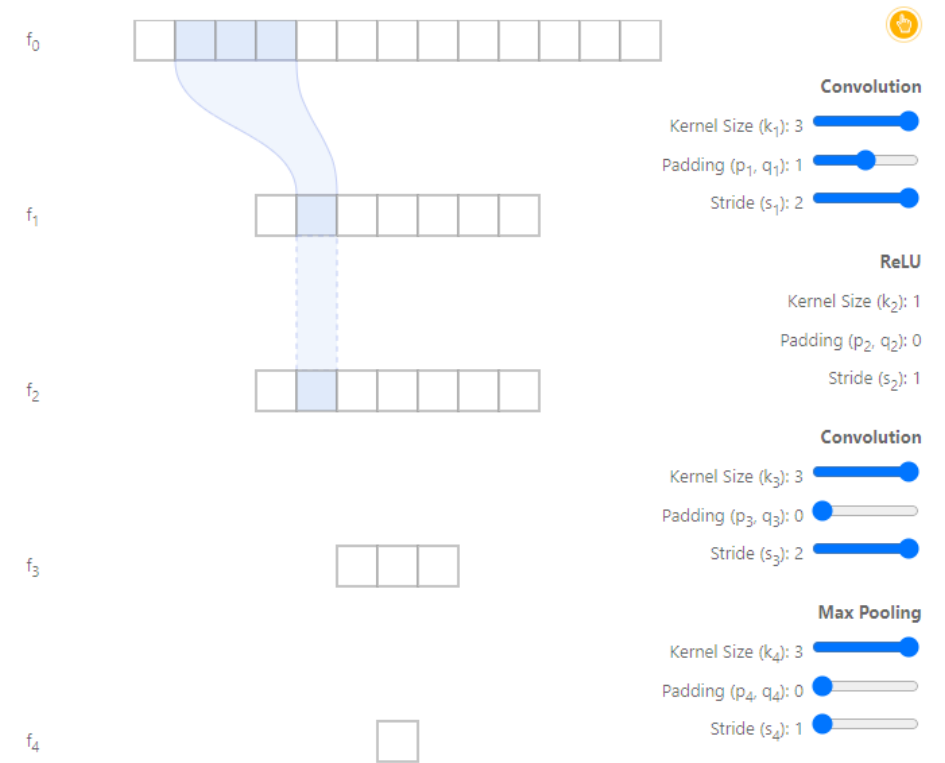
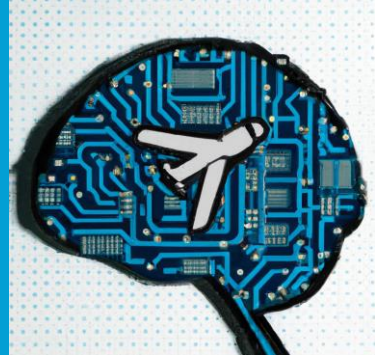


Figure from:

<https://distill.pub/2019/computing-receptive-fields/>

Receptive field size



Larger spatial scales:

The receptive field size of features increases for higher layers. The exact size can be calculated based on the type of padding, kernel size, and stride.

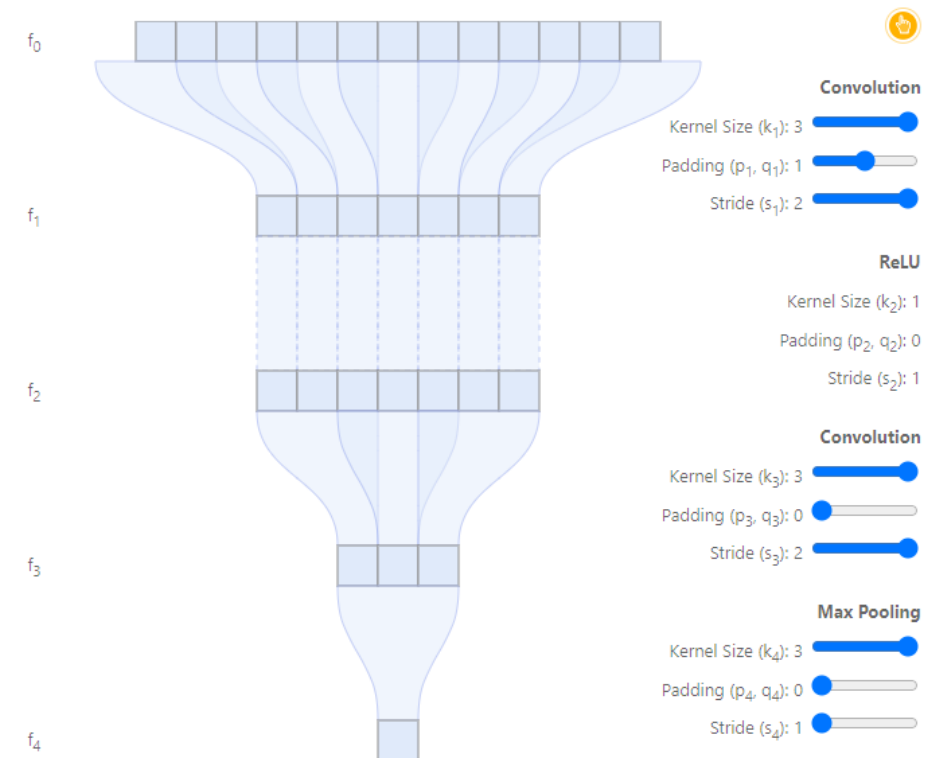
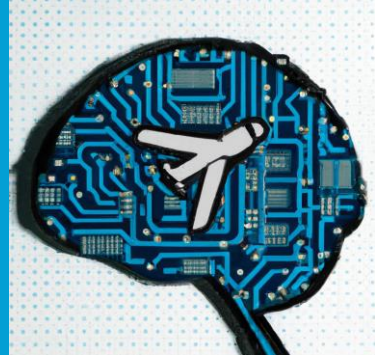


Figure from:

<https://distill.pub/2019/computing-receptive-fields/>

Fully convolutional networks



Auto-encoder:

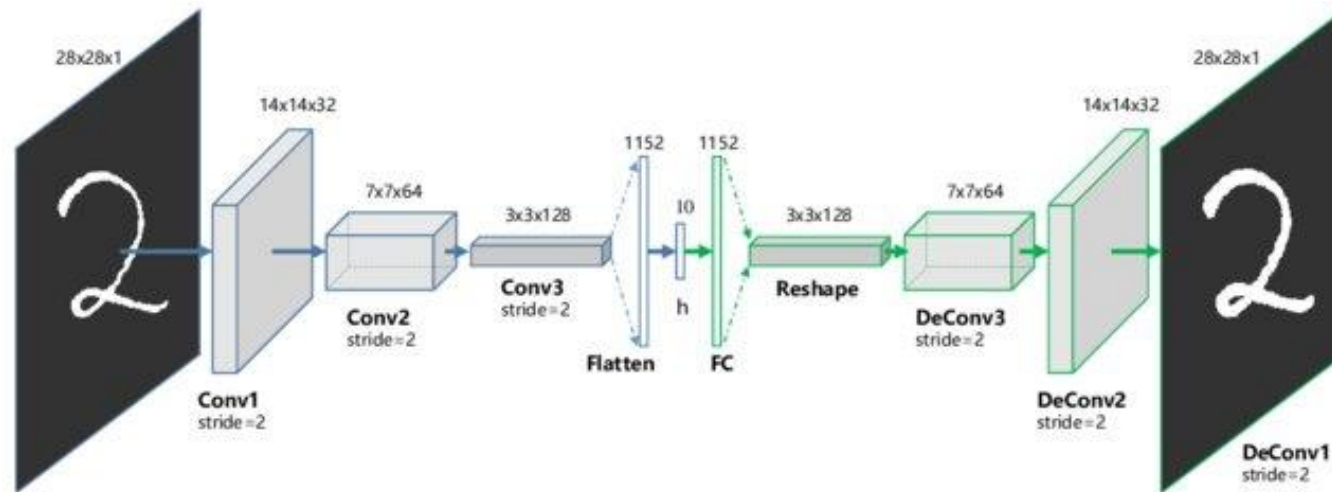
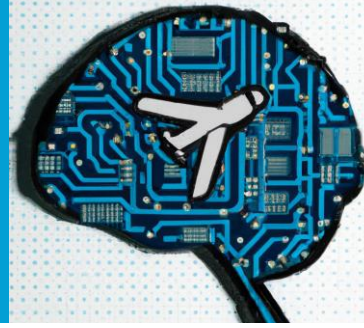
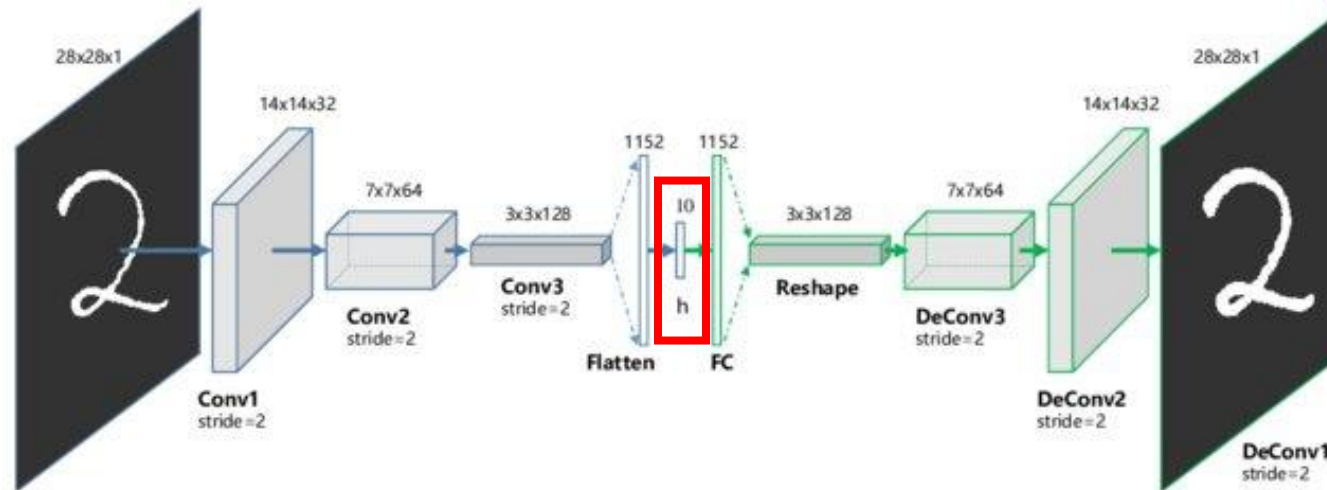


Image from: Guo, X., Liu, X., Zhu, E., & Yin, J. (2017, November). Deep clustering with convolutional autoencoders. In *International conference on neural information processing* (pp. 373-382). Springer, Cham.

Fully convolutional networks



Auto-encoder:



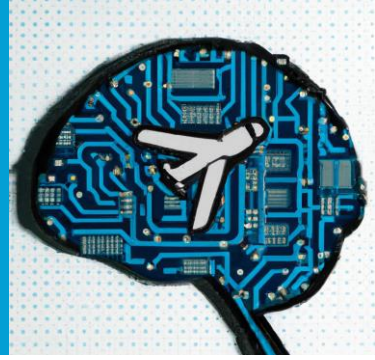
Note:

The loss function is defined as the difference between the output image and the input image.

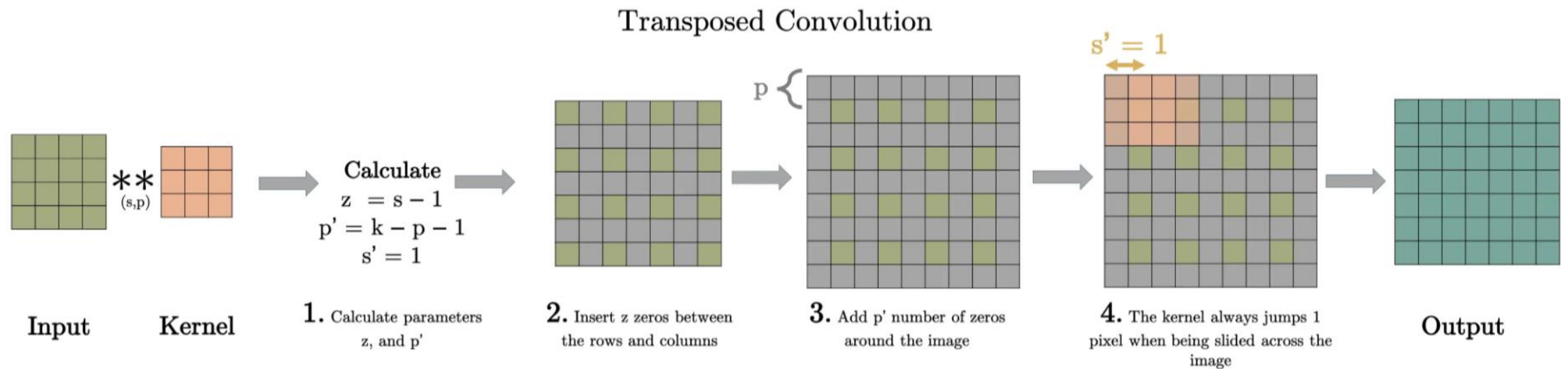
*The **inner hidden layer** is a highly compressed version of the image.*

Image from: Guo, X., Liu, X., Zhu, E., & Yin, J. (2017, November). Deep clustering with convolutional autoencoders. In *International conference on neural information processing* (pp. 373-382). Springer, Cham.

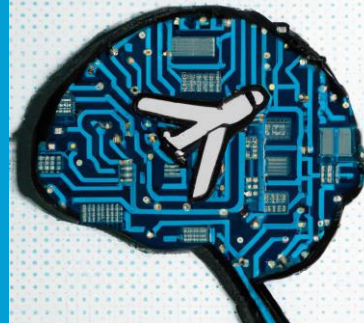
Deconvolution



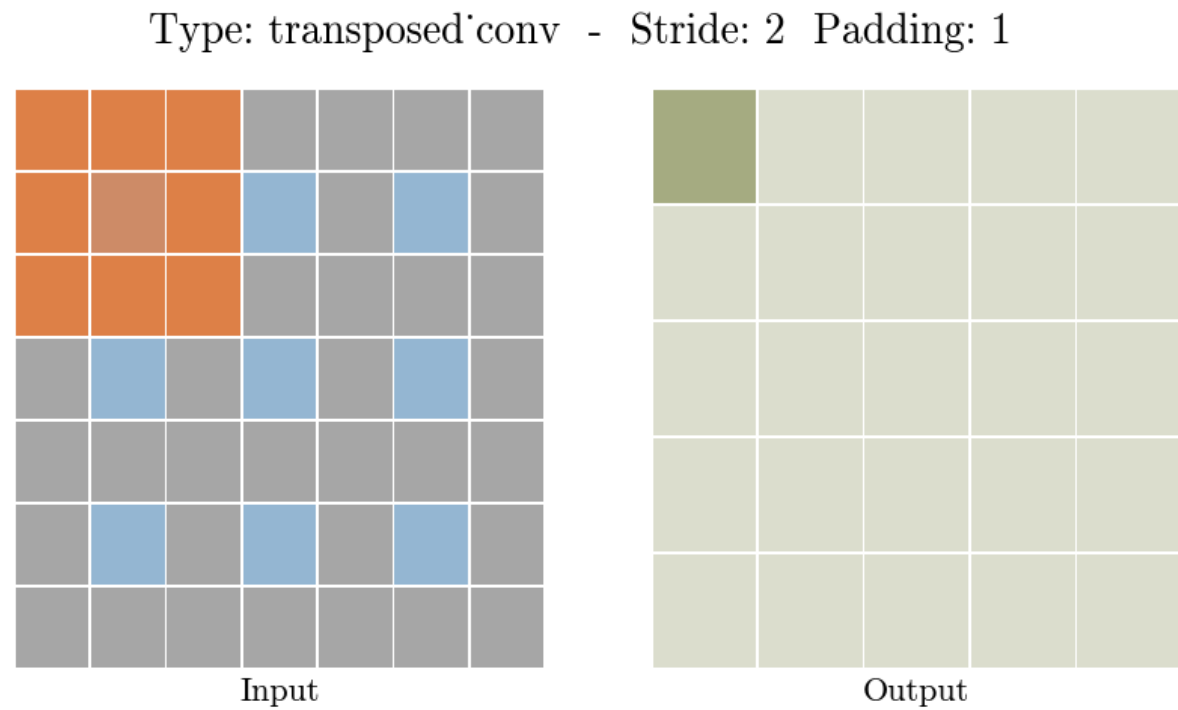
Also called “transposed convolution”:



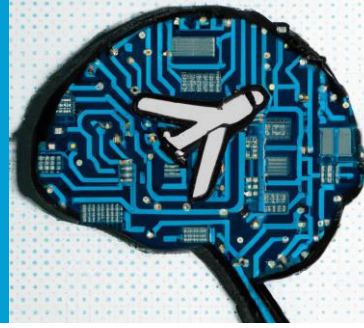
Deconvolution



Also called “transposed convolution”:



UNet



U-Net is a fully convolutional network with skip connections, originally invented for image segmentation.

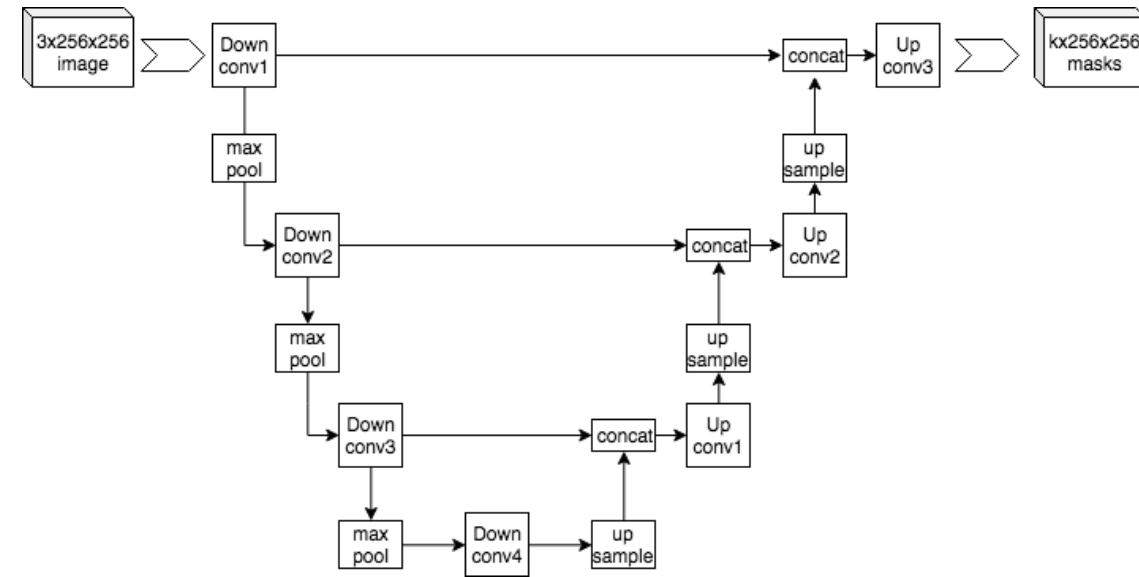
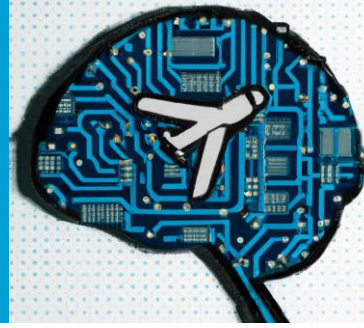


Image from  WIKIPEDIA
The Free Encyclopedia

UNet



U-Net is a fully convolutional network with **skip connections**, originally invented for image segmentation.

Can you think of an advantage of skip connections in such an image processing task?

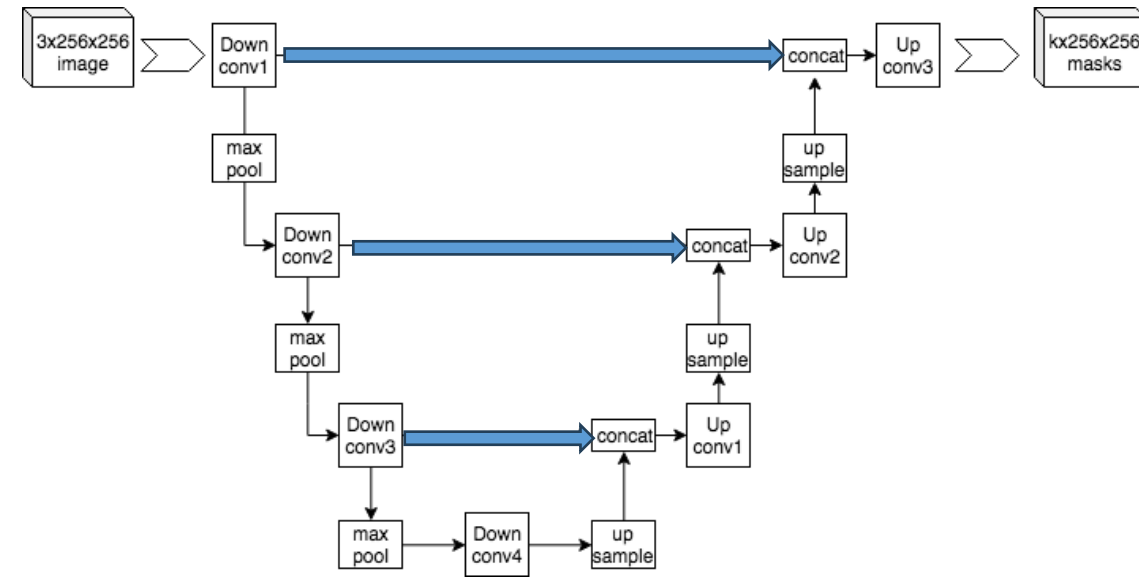
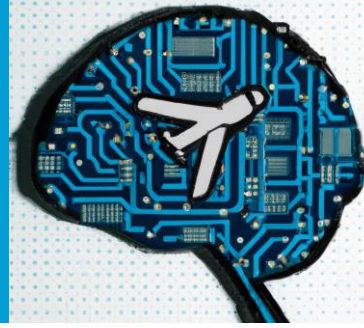


Image from  WIKIPEDIA
The Free Encyclopedia

UNet



U-Net is a fully convolutional network with skip connections, originally invented for image segmentation.

Can you think of an advantage of skip connections in such an image processing task?

Skip connections help retain *resolution*.

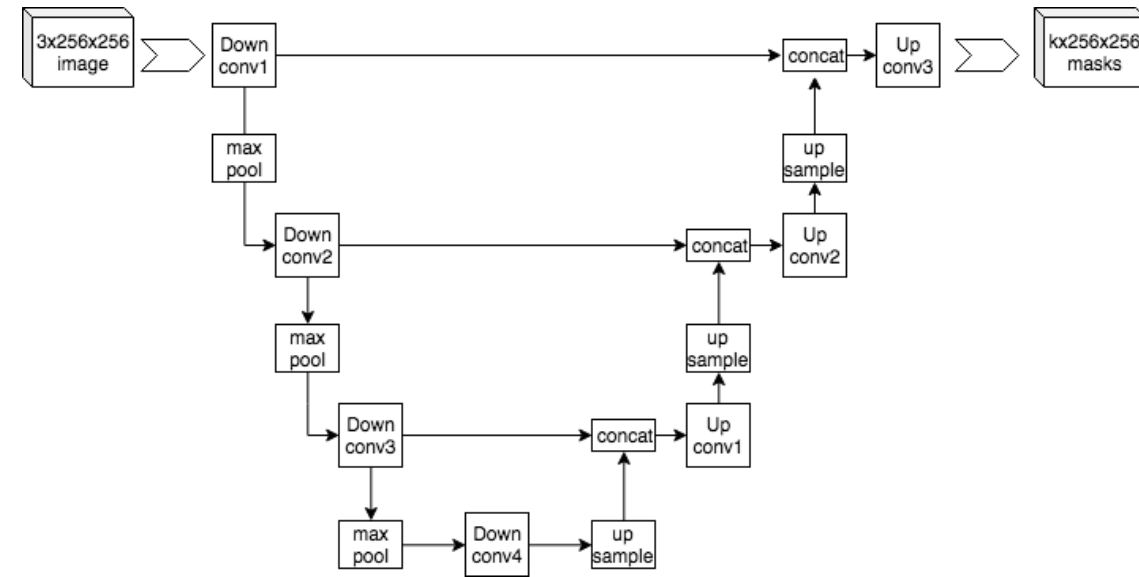
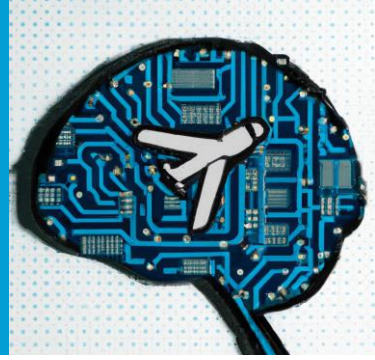
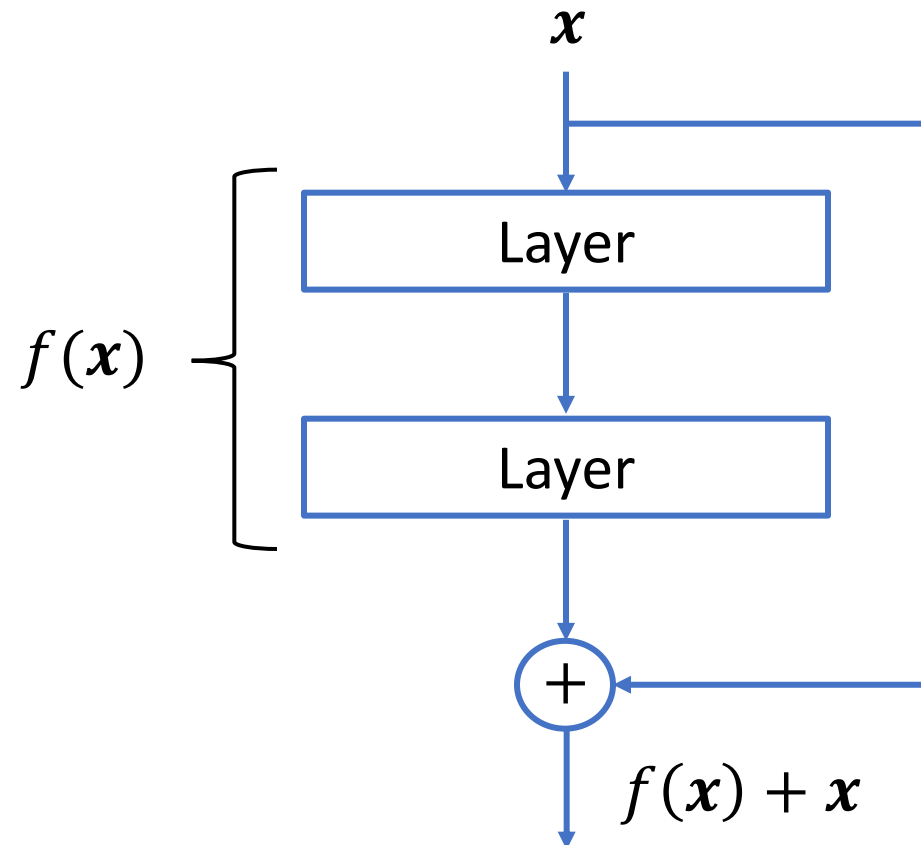


Image from  WIKIPEDIA
The Free Encyclopedia

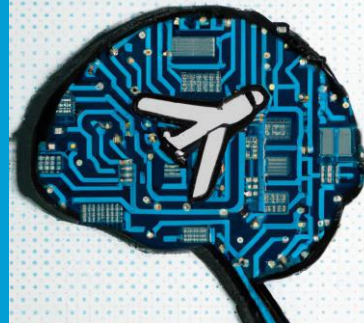
Residual network



Neural network that includes *residual / skip* connections:

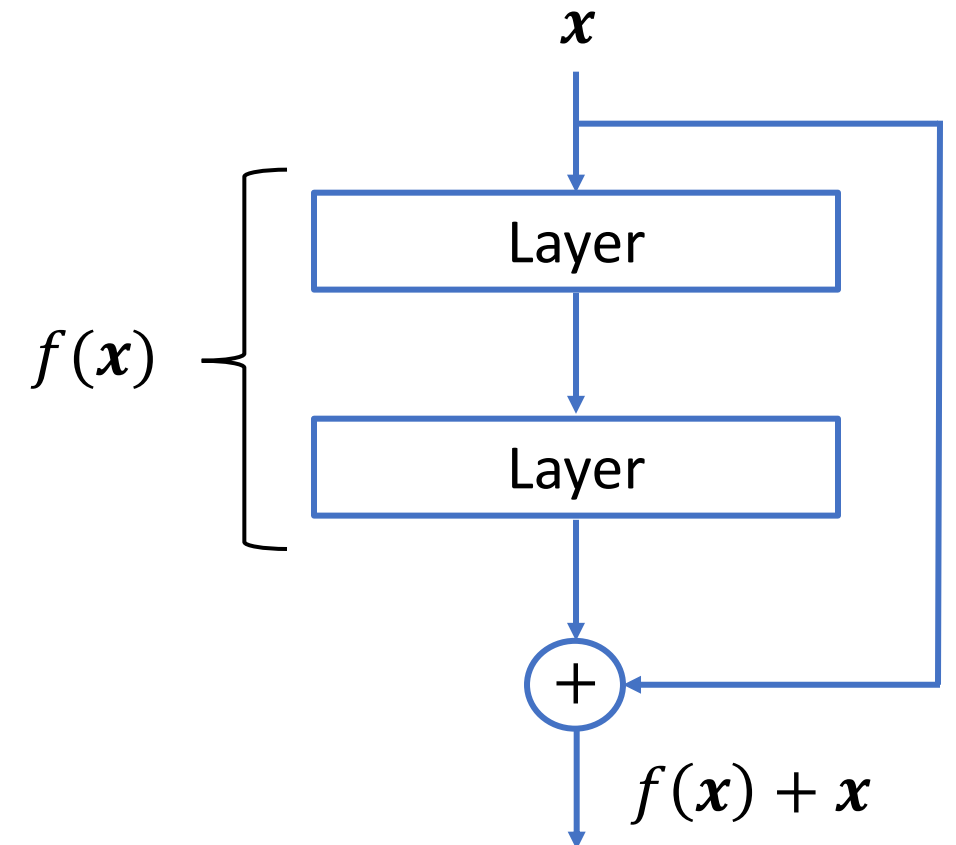


Residual network

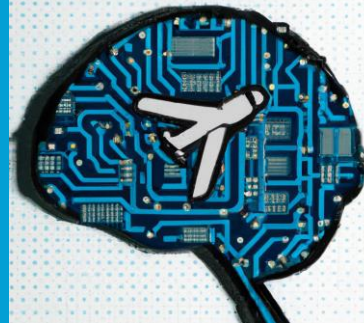


Neural network that includes *residual / skip* connections:

Can you think of an additional advantage of skip connections?



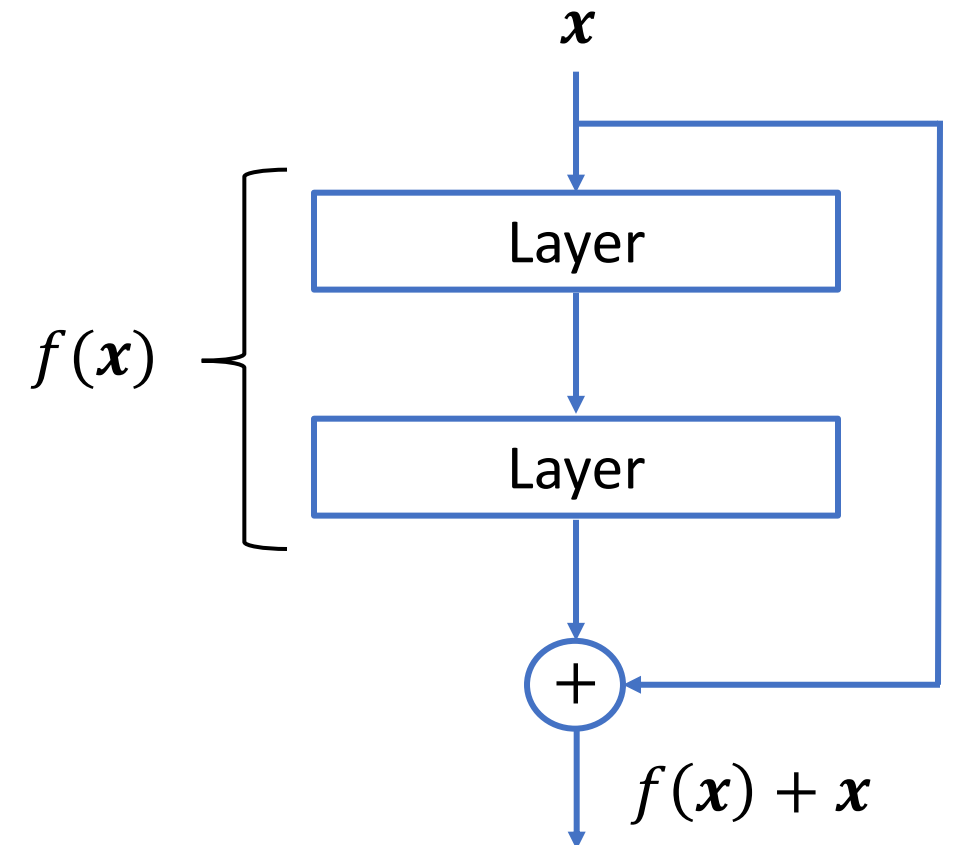
Residual network

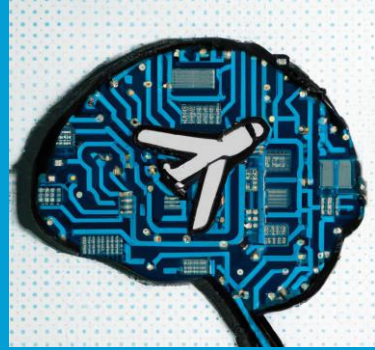


Neural network that includes *residual / skip* connections:

Can you think of an additional advantage of skip connections?

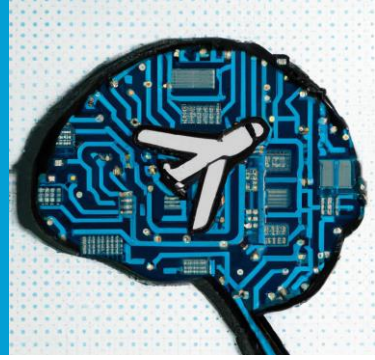
It helps combat vanishing gradient.





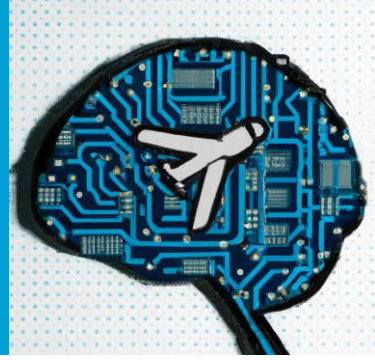
Quiz

Question 1



Why is overfitting especially problematic for deep neural networks?

Question 1

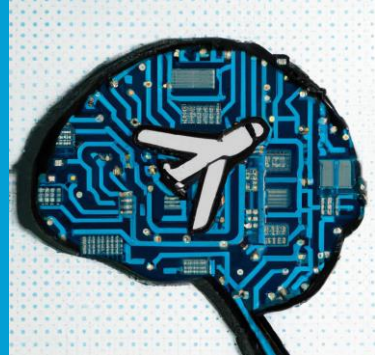


Why is overfitting especially problematic for deep neural networks?

Because deep neural networks can represent very complex functions (have a high capacity), they are more prone to overfitting on the training data.

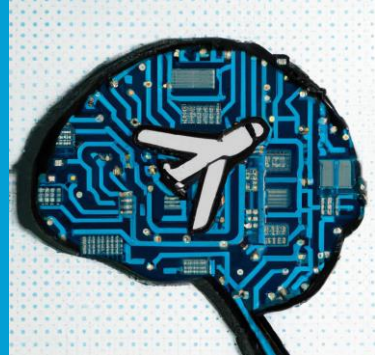
A perceptron has a lower capacity (can only model linear functions), and is less prone to overfitting.

Question 2



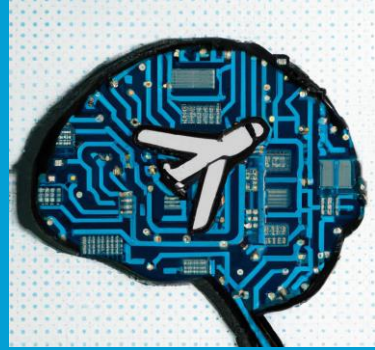
Why is the use of convolutional kernels called “weight” sharing? And why is this useful?

Question 2

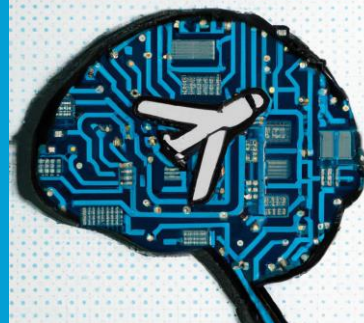


Why is the use of convolutional kernels called “weight” sharing? And why is this useful?

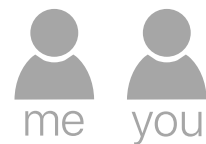
- Because all neurons in the feature response map covering the visual field share the same (kernel) weights.
- This substantially reduces the number of required weights for processing high-dimensional inputs.



Questions?



Note:
A nice text box



Tutor



More info

